
BOCLOAK: Optimal Transport-Guided Adversarial Attacks on Graph Neural Network-Based Bot Detection

Kunal Mukherjee¹ Zulfikar Alom² Tran Gia Bao Ngo³ Cuneys Gurcan Akcora⁴ Murat Kantarcioglu¹

Abstract

The rise of bot accounts on social media poses significant risks to public discourse. To address this threat, modern bot detectors increasingly rely on Graph Neural Networks (GNNs). However, the effectiveness of these GNN-based detectors in real-world settings remains poorly understood. In practice, attackers continuously adapt their strategies as well as must operate under domain-specific and temporal constraints, which can fundamentally limit the applicability of existing attack methods. As a result, there is a critical need for robust GNN-based bot detection methods under realistic, constraint-aware attack scenarios. We introduce BOCLOAK¹ to systematically evaluate the robustness of GNN-based bot detection via both edge editing and node injection adversarial attacks under realistic constraints. BOCLOAK constructs a probability measure over spatio-temporal neighbor features and learns an optimal transport (OT) geometry that separates human and bot behaviors. It then decodes transport plans into sparse, plausible edge edits that evade detection while obeying real-world constraints. We evaluate BOCLOAK across three social bot datasets, five state-of-the-art bot detectors, three adversarial defenses, and compare it against four leading graph adversarial attack baselines. BOCLOAK achieves up to 80.13% higher attack success rates while using 99.80% less GPU memory under real-world constraints. BOCLOAK shows that OT provides a lightweight, principled framework for bridging adversarial attacks and real-world bot detection.

¹Department of Computer Science, Virginia Tech, Virginia, USA ²Department of Computer Science, University of Toledo, Ohio, USA ³Department of Computer Science, Manitoba, Canada ⁴AI Initiative, University of Central Florida, Florida, USA. Correspondence to: Kunal Mukherjee <mkunal@vt.edu>, Murat Kantarcioglu <muratk@vt.edu>.

1. Introduction

Bot accounts that mimic human behavior pose a persistent threat on social platforms. They distort engagement, amplify misinformation, and manipulate public discourse at scale (Liu et al., 2025). To mitigate this, GNN-based detectors are widely deployed, combining user profiles, content, and social connectivity (Varol et al., 2017; Kudugunta & Ferrara, 2018; Feng et al., 2021b; 2022a).

Despite strong performance in controlled settings, the robustness of these detectors under realistic adversarial conditions remains poorly understood. Existing graph attacks (Zügner et al., 2018; Geisler et al., 2021; Alom et al., 2025) typically assume an attacker can perturb node features or flip edges arbitrarily, subject only to a fixed budget. These assumptions collapse in real deployments. Bot cannot rewire arbitrary users or gain instant access to high-reputation connections. Instead, they must create new accounts, learn from the demise of previously flagged bots, and establish plausible follow patterns in order to evade detection.

This setting introduces core constraints. First, the social graph is large, and probing all possible nodes is infeasible due to computational costs, API limits, and detection risk. Second, the number of feasible edge configurations grows combinatorially with the size of the graph. Third, while bots must connect to amplify content, overt coordination exposes them to cluster-based detection. Once one bot account is flagged, its connections can trigger cascading bans. Effective attacks must therefore operate under partial observability, behavioral plausibility, and limited connectivity.

Prior work is fragmented: feature-space attacks (Cresci et al., 2019; 2021) ignore network structure, recent node-editing methods introduce helper bots for evasion (Wang et al., 2023; 2025; Liu et al., 2024) but do not characterize vulnerable local neighborhoods or how to construct them under temporal and domain constraints. As a result, State-Of-The-Art (SOTA) detection methods are fragile; our results in Figure 1 show that even a single human follower can mislead a SOTA bot detector in 81.50% of the cases. This striking fragility highlights a gap in existing defenses: despite these vulnerabilities, prior work has not connected bot evasion to a structured geometric attack formulation.

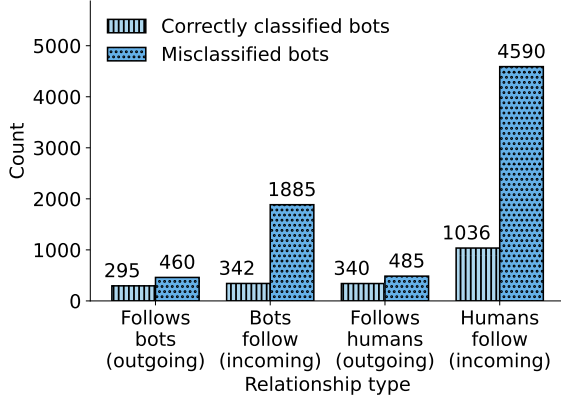


Figure 1. Relationship counts for classified bots in TwiBot-22 by BotRGCN. Counts are computed from the perspective of bot accounts. Each category indicates whether a bot has at least one outgoing or incoming relationship of the specified type. Categories are not mutually exclusive: a single bot may contribute to multiple bars. Misclassified bots are more likely to have edges incident to humans, especially incoming follows.

We argue that optimal transport (OT) (Petric Maretic et al., 2019), with its roots in measure theory and convex optimization, provides a principled and efficient way to reason in such constrained settings. Our proposed method, BOCLOAK, uses OT to compare probability distributions over spatio-temporal features of nodes and their neighbors, and finds the minimal cost of transforming one local neighborhood distribution (e.g., from a bot) into another (e.g., from a human account). OT plans are low-cost, interpretable transformations (example in App. W) that can be computed efficiently with partial network access or model gradients. These properties make OT ideal for generating sparse, plausible edge edits for attacking under realistic constraints.

We evaluate BOCLOAK across three social bot datasets, five bot detectors, and three defense strategies, comparing it against four SOTA graph adversarial attack baselines. BOCLOAK consistently achieves 80.13% higher attack success rates while requiring negligible resource overhead and maintaining social plausibility. Notably, it performs equally well in unconstrained settings with no domain limitations. Our results push OT-based adversarial attacks to the forefront for efficient, model-agnostic bot detection.

Our contributions are as follows:

- We present the first application of optimal transport theory in social bot detection and attacks on GNNs.
- We develop a learnable OT geometry that aligns spatio-temporal graph transport costs with adversarial objectives and realistic domain constraints.
- We demonstrate that BOCLOAK evades SOTA bot detectors under both constrained and unconstrained attacks, achieving 80.13% higher attack success. BOCLOAK bypasses five prominent defenses and sustains high evasion

rates under three adversarial hardening techniques.

- We demonstrate that BOCLOAK has significantly lower computational overhead than SOTA attacks. Across three datasets, it achieves up to 99.80% lower GPU memory usage than PR-BCD and FGA, and runs up to 20 $\times$ faster than SOTA adversarial attacks (e.g., Netstack and GOTack), without gradient access or model-specific tuning.

2. Related Works

Social Bot Detection. Early work on social bot detection relied on supervised classifiers (Ferrara et al., 2016; Varol et al., 2017; Kudugunta & Ferrara, 2018). As bots evolved from simple spammers to coordinated, human-like accounts, research shifted toward graph-based detectors that more explicitly use social structure. The release of social graph datasets for bot detection, such as Cresci-2015, TwiBot-20, and TwiBot-22 (Cresci et al., 2015; Feng et al., 2021a; 2022b) standardized evaluation across user attributes, node, and edge types. To address the growing gap in realism between benchmarks and emerging adversaries, the BotSim-24 (Qiao et al., 2025) dataset demonstrates that LLM-driven bots substantially degrade the performance of existing detectors. These datasets catalyzed the development of Graph Neural Network (GNN)-based bot detectors such as BotRGCN (Feng et al., 2021b), S-HGN (Lv et al., 2021), and RGT (Feng et al., 2022a).

We treat these detectors as victim models to be attacked and study how an adaptive adversary can inject new bots with carefully engineered neighborhoods that remain undetected.

GNN-based Adversarial Attacks. Adversarial vulnerabilities of GNNs have been widely studied in graph benchmarks. Netstack (Zügner et al., 2018) introduced targeted poisoning and evasion attacks by greedily perturbing features and edges around a victim node, while Fast Gradient Attack (FGA) (Chen et al., 2018) used gradient saliency to select high-impact edge rewiring operations. Subsequent work GOTack (Alom et al., 2025) learned universal perturbation patterns by exploiting the orbit structure of nodes, achieved strong cross-architecture transfer, and PR-BCD (Geisler et al., 2021) formulated topology attacks as continuous optimization over an adjacency-perturbation matrix and solved it via projected randomized block coordinate descent, scaling to large graphs under a global edge budget. More recent graph attacks include LR-BCD, which adds local node-wise constraints to randomized block-coordinate attacks (Gosch et al., 2023), partial graph attack (PGA), which allocates a global attack budget to selected vulnerable nodes (Zhu et al., 2024), and EvA, which directly optimizes discrete graph perturbations with an evolutionary black-box procedure (Akhondzadeh et al., 2026).

BOCLOAK differs from these works along two axes: (i)

we formulate bot evasion directly in the space of domain and time-aware neighborhood distributions and learn an OT ground cost that shapes an explicit human-bot geometry; and (ii) our attack is optimized to mimic misclassified bots while respecting strict node-injection, degree, and temporal constraints. LR-BCD and EvA are not designed around social-network feasibility constraints such as no forced human follow-backs, while PGA is a global budget-allocation attack rather than a per-target test-time evasion attack. These differences motivate a domain-specific attack design rather than a direct reuse of general graph attacks.

3. Preliminaries

We model a social network as a directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{X})$, where $\mathcal{V}$ is the set of nodes (accounts), $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is the set of directed edges (follow relations), and $\mathbf{X} \in \mathbb{R}^{|\mathcal{V}| \times d}$ denotes node features (profile, posts, tweets). For a node $v \in \mathcal{V}$, let $\mathcal{N}_k(v) \subseteq \mathcal{V}$ denote the set of nodes in its k -hop ego-neighborhood. We use $\mathcal{Z}_v = \{z_i\}_{i=1}^{m_v}$ for the corresponding multiset of neighbor feature vectors, where each z_i is constructed from $\mathbf{X}$ and graph structure and encodes attribute, relational, and temporal information. This separates the node set $\mathcal{N}_k(v)$ from the feature vectors on which OT is computed.

We now summarize the optimal transport (OT) formulation (Petric Maretic et al., 2019; Peyré & Cuturi, 2019) used to compare two empirical distributions over $\mathbb{R}^d$. Given two discrete distributions $\mu_1 = \sum_{i=1}^m a_i \delta_{z_i}$ and $\mu_2 = \sum_{j=1}^n b_j \delta_{\tilde{z}_j}$, where δ_z denotes the Dirac measure concentrated at $z \in \mathbb{R}^d$ (Tao, 2011), with nonnegative weights a_i and b_j satisfying $\sum_i a_i = 1$ and $\sum_j b_j = 1$, and a nonnegative ground cost $c(z_i, \tilde{z}_j)$ between feature vectors, we define the cost matrix $C \in \mathbb{R}^{m \times n}$ by $C_{ij} = c(z_i, \tilde{z}_j)$. The (unregularized) OT distance, $W_c(\mu_1, \mu_2)$ is: $\min_{P \in \mathbb{R}_{\geq 0}^{m \times n}} \langle P, C \rangle_F$ s.t. $P \mathbf{1}_n = a$, $P^\top \mathbf{1}_m = b$, where $\langle P, C \rangle_F = \sum_{i,j} P_{ij} C_{ij}$ is the Frobenius inner product between P and C , and $\mathbf{1}_m$ is the ones column vector in $\mathbb{R}^m$.

OT can be entropy-regularized for computational stability and scalability. Throughout the paper, $\log$ denotes the natural logarithm and we use the standard convention $0 \log 0 = 0$. For $\varepsilon > 0$, the regularized problem adds the term $\varepsilon \sum_{i,j} P_{ij} (\log P_{ij} - 1)$ to the objective. This term may be negative because $P_{ij} \in [0, 1]$ and because $\sum_{i,j} P_{ij} = 1$; the “ -1 ” contributes only the additive constant $-\varepsilon$, independent of P , but yields the convenient derivative $\varepsilon \log P_{ij}$. The resulting problem is strictly convex and has a unique solution P^* that can be efficiently computed via Sinkhorn (Cuturi, 2013) iterations. The corresponding transport cost $D(\mu_1, \mu_2) = \sum_{i,j} P_{ij}^* c(z_i, \tilde{z}_j)$ defines a distance between empirical distributions. The optimal plan admits a closed-form scaling structure $P^* = \text{diag}(u) K \text{diag}(v)$, where

$u \in \mathbb{R}^m$ and $v \in \mathbb{R}^n$ are scaling vectors that enforce the marginal constraints, and $K_{ij} = \exp(-c(z_i, \tilde{z}_j)/\varepsilon)$ is the Gibbs kernel derived from the cost matrix.

Semi-Supervised Node Classification. Let a node subset $\mathcal{V}_{\text{train}} \subset \mathcal{V}$ be labeled with class labels $y_v \in \mathcal{Y}$ (e.g., human vs. bot), while the remaining nodes are unlabeled. A GNN-based classifier f_Θ maps each node v to a predictive distribution over labels: $f_\Theta : (\mathcal{G}, \mathbf{X}) \mapsto (p_\Theta(y | v))_{v \in \mathcal{V}}$.

Adversarial Attack. An adversarial attack perturbs the graph $\mathcal{G}$ to induce incorrect predictions by a trained classifier f_Θ . We focus on test-time evasion attacks that modify graph inputs while keeping the model fixed.

The attacker may change the graph structure and node features under a fixed perturbation budget Δ . Let $\mathbf{A}$ and $\mathbf{A}'$ be the adjacency matrices of the original and perturbed graphs, and let $\mathbf{X}$ and $\mathbf{X}'$ be the corresponding node feature matrices. The total perturbation is constrained as:

$$\sum_{u,v} |\mathbf{A}_{uv} - \mathbf{A}'_{uv}| + \|\mathbf{X} - \mathbf{X}'\|_0 \leq \Delta, \quad (1)$$

where $\|\cdot\|_0$ denotes the number of feature entries changed. The resulting perturbed graph $\mathcal{G}' = (\mathcal{V}, \mathcal{E}', \mathbf{X}')$ is used at inference time, and the attacker’s goal is to cause target nodes to be misclassified with the modified input.

3.1. Problem Formulation

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{X})$ be a directed social graph, where $\mathcal{V}$ denotes user accounts, $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ directed follow relations, and $\mathbf{X} \in \mathbb{R}^{|\mathcal{V}| \times d}$ node features. A subset $\mathcal{V}_L \subset \mathcal{V}$ is labeled for semi-supervised bot detection, with $y_v \in \{\text{human}, \text{bot}\}$. A deployed bot detector f is trained on $\mathcal{G}$ and outputs class probabilities $p_f(y | v; \mathcal{G})$ for any node v .

The attacker operates at test time and introduces a new bot node $v_t \notin \mathcal{V}$ with features $\mathbf{x}_t$. The attacker selects a set of incident edges, $\Delta\mathcal{E}(v_t) \subseteq (v_t \times \mathcal{V}) \cup (\mathcal{V} \times v_t)$, resulting in the modified graph, $\mathcal{G}' = (\mathcal{V} \cup \{v_t\}, \mathcal{E} \cup \Delta\mathcal{E}(v_t), \mathbf{X} \cup \{\mathbf{x}_t\})$. Outgoing follow edges from v_t are freely chosen. Incoming edges (follow-backs) are not directly controllable for humans and must respect domain constraints.

Domain Constraints. All constrained results use the same admissible-edit set for every attack. Let $\mathcal{V}_{\text{hum}}$ be the human nodes. Edits are constrained to a feasible set $\mathcal{F}(B)$ defined by a per-target budget B and hard social-plausibility rules:

$$\begin{aligned} \mathcal{F}(B) = \{ \Delta\mathcal{E} : |\Delta\mathcal{E}| \leq B, \Delta\mathcal{E} \subseteq (\{v_t\} \times \mathcal{V}) \cup (\mathcal{V} \times \{v_t\}), \\ \Delta\mathcal{E} \cap (\mathcal{V}_{\text{hum}} \times \{v_t\}) = \emptyset, \\ \Psi_{\text{dir}}(\Delta\mathcal{E}) \leq 0, \Psi_{\text{time}}(\Delta\mathcal{E}) \leq 0 \}. \end{aligned}$$

The first condition enforces the edge budget; the second makes every edit incident to the target bot; the third prohibits forced human $\rightarrow$ bot follow-backs; and Ψ_{dir} and Ψ_{time} reject

directionally invalid or temporally inconsistent edits. Thus the attacker cannot rewire unrelated graph regions, create arbitrary human follow-backs, or modify features, labels, or metadata of any non-target node.

Attacker’s Capability. The attacker performs targeted evasion by injecting or editing a bot v_t so that it is misclassified as human. Edits are restricted to edges incident to v_t and must respect a hard edge budget B . The attacker can freely choose outgoing follow edges from v_t to existing nodes, but cannot force incoming follow-backs from existing users (e.g., cannot make an arbitrary human follow its bot). No changes are permitted to the edges between any other pairs of nodes, nor to the features, labels, or metadata (e.g., account age) of any node other than v_t .

Attacker’s Knowledge. We assume a black-box setting. The attacker has access to the training graph and its node labels, as well as the feature schema (profile, content, and edges), but *no knowledge of the victim model’s architecture, parameters, gradients, or output scores*. The attacker cannot query the model or observe logits. Instead, the attacker observes which accounts are flagged or removed by the platform and which accounts remain active over time, as reflected through observable metadata such as account age. For injected bots, the attacker may assign non-temporal profile and content features (e.g., username, bio).

We do not consider poisoning (training-time) or availability (global degradation) attacks. The *focus is on inserting bots that evade detection under realistic deployment conditions*.

Victim Models. We consider two categories of victim models (Bacciu et al., 2020): vanilla detectors, which are trained without explicit adversarial robustness objectives, and defended detectors, which integrate adversarial defense. Attacks on vanilla models establish a baseline vulnerability level, whereas evaluations on defended models assess the effectiveness of robustness strategies. Notably, these defenses typically operate without prior knowledge (i.e., non-adaptive) of the specific attack techniques employed.

4. BOCLOAK: Constrained Graph Attacks via Optimal Transport

We present BOCLOAK (shown in Figure 2), a framework for constrained adversarial attacks on social graphs that generate evasive bot accounts through node editing or injection.

Challenges. Issues that shape the attack design: search and feasibility. First, the neighborhood around a bot is combinatorial because a target bot can connect to a large candidate pool with many possible degree patterns, interaction roles, and temporal profiles. Direct search over edge sets $\Delta\mathcal{E}$ is not feasible at the scale of social graphs. Second, bot edits must respect behavioral and structural constraints. The attacker

cannot rewire the rest of the graph or force follow-backs. *These constraints exclude many perturbations assumed in standard adversarial settings.*

BOCLOAK Philosophy. BOCLOAK is a geometry-first method that treats local neighborhoods, rather than individual edges, as first-class objects of adversarial manipulation. Each node is represented by a probability measure over its ego-neighborhood in a spatio-temporal feature space, and optimal transport compares these distributions under a learned ground cost. This induces a metric geometry in which bots and humans occupy distinct regions, with misclassified bots near the decision boundary. BOCLOAK selects such boundary bots as cloak templates to derive sparse, feasible edge edits by decoding transport plans.

Why OT rather than scalar matching? A homophily or embedding similarity score can say that two neighborhoods are close, but it does not specify which neighbor mass should be aligned under a budget. OT is a mapping that returns both a scalar distance and a coupling P^* : the distance selects a template region to imitate, while the coupling identifies the few high-mass neighbor correspondences that can be decoded into feasible target-incident edits. Thus BOCLOAK reconstructs a plausible local neighborhood rather than copying an entire ego-network or greedily searching over all edges.

4.1. Neighborhood and Boundary Definitions

Let $\mathcal{N}_k(v)$ denote the node set in the k -hop ego-neighborhood of node v ; in experiments we use $k = 1$ unless otherwise stated. For each neighbor $\eta \in \mathcal{N}_k(v)$ we define a feature vector $\phi_v(\eta) \in \mathbb{R}^d$ that summarizes attributes of η and its relation to v . The construction of $\phi_v(\eta)$ is domain dependent, but the method only requires that $\phi_v(\eta)$ be a fixed-dimensional representation that can encode static attributes, interaction role features, content signals, and temporal signals.

Neighborhood Distribution and Cost Measures. We represent the neighborhood of v as an empirical probability measure (as shown in 1 of Figure 2)

$$\mu_v = \sum_{\eta \in \mathcal{N}_k(v)} w_v(\eta) \delta_{\phi_v(\eta)},$$

where δ_x is a point mass at x and $w_v(\eta)$ is a normalized importance weight. The weighting scheme lets the measure emphasize neighbors that carry a stronger behavioral signal for detection. We set $w_v(\eta)$ by normalizing an importance score $a_v(\eta) > 0$ (details in Appendix §C.2, Equation 9):

$$w_v(\eta) = \frac{a_v(\eta)}{\sum_{\eta' \in \mathcal{N}_k(v)} a_v(\eta')}.$$

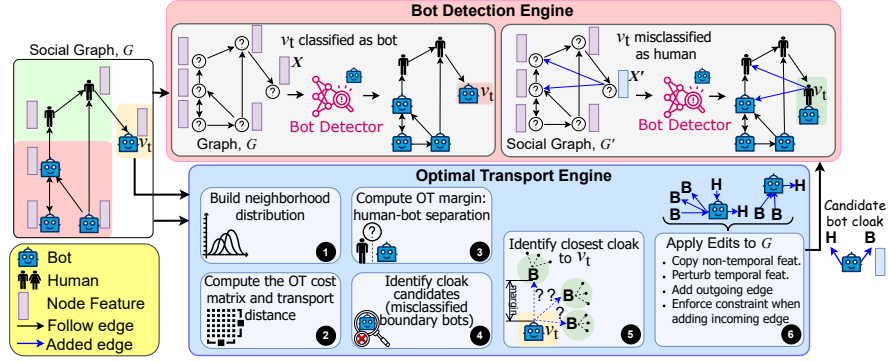


Figure 2. Overview of BOCLOAK. The method learns an optimal transport geometry over k -hop neighborhood distributions and uses transport plans to guide sparse, plausible local edge edits for evasion.

The score $a_v(\eta)$ is defined as the product of a structural term and a temporal term: $a_v(\eta) = g_{\text{str}}(s(\eta)) g_{\text{temp}}(t(\eta))$, where $s(\eta)$ is a scalar structural statistic and $t(\eta)$ is a scalar temporal statistic. The product acts as an “and” gate: a neighbor receives high mass only when it is both structurally informative and temporally plausible, rather than merely satisfying one criterion. The functions g_{str} and g_{temp} are monotone and chosen to emphasize informative neighbors while avoiding extreme weight concentration.

What topology is retained? BOCLOAK does not require the full platform graph at attack time. Instead, topology enters through the ego-neighborhood node set, directed edge-role features, in/out-degree statistics, and feasibility checks on candidate edits. This deliberately encodes the structural information available to an attacker with partial observability while avoiding assumptions that the attacker can inspect or rewire arbitrary distant graph regions.

We define a ground cost between neighbor feature vectors through a learnable embedding. Let $h_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^{d_{\text{emb}}}$ be a neural map, and let $\mathbf{M} \succeq 0$ be a learnable positive semidefinite matrix. For neighbor features $z = \phi_v(\eta)$ and $z' = \phi_\xi(\eta')$, with $\eta \in \mathcal{N}_k(v)$ and $\eta' \in \mathcal{N}(\xi)$, we define $c_\theta(z, z') = \|h_\theta(z) - h_\theta(z')\|_{\mathbf{M}}^2$ where $\|u\|_{\mathbf{M}}^2 \triangleq u^\top \mathbf{M} u$. This choice yields a smooth cost that correlates features and supports stable entropic OT computations.

4.2. Optimal Transport Distance Between Nodes

Let $c_\theta : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}_{\geq 0}$ be a learned ground cost between neighboring feature vectors. Using the entropy-regularized optimal transport formulation from Section 3, we define the distance between two nodes v and ξ via their neighborhood measures, (as shown in 2 of Figure 2).

Given neighborhood measures $\mu_v = \sum_{i=1}^m a_i \delta_{z_i}$ and $\mu_\xi = \sum_{j=1}^n b_j \delta_{z'_j}$, we construct the cost matrix $C_{ij} = c_\theta(z_i, z'_j)$ and solve the following entropic optimal transport problem:

$$P_{v\xi}^* \in \arg \min_{P \in \mathcal{U}(a,b)} \underbrace{\sum_{i,j} P_{ij} C_{ij}}_{\text{core}} + \varepsilon \underbrace{\sum_{i,j} P_{ij} (\log P_{ij} - 1)}_{\text{regularizer}}, \quad (2)$$

where the transport polytope is $\mathcal{U}(a,b) = \{P \in \mathbb{R}_{\geq 0}^{m \times n} : P \mathbf{1}_n = a, P^\top \mathbf{1}_m = b\}$. Here $\mathbf{1}_m$ and $\mathbf{1}_n$ enforce that P has row sums a and column sums b , further details in Appendix §C.3 and §C.4.

We define the OT distance between nodes v and ξ as the transport cost induced by the optimal plan:

$$D_\theta(v, \xi) \triangleq \langle P_{v\xi}^*, C_{v\xi} \rangle. \quad (3)$$

We compute the plan $P_{v\xi}^*$ using Sinkhorn iterations, detailed derivation in Appendix D. With $K = \exp(-C/\varepsilon)$, Sinkhorn alternates $u \leftarrow a \odot (Kv)$ and $v \leftarrow b \odot (K^\top u)$ for T_{sink} iterations and returns $P^* \approx \text{diag}(u) K \text{diag}(v)$. The distance compares neighborhoods by transporting mass between their features under the learned cost c_θ .

Learning a Label-Aware Geometry.

We train θ offline with a multi-term loss that shapes margins and favors sparse and plausible alignments. For each training bot v , the compact objective is

$$\mathcal{L}_{\text{BOCLOAK}}(\theta) = \mathbb{E}_{v \in \mathcal{V}_{\text{train}}} [\lambda_{\text{BCE}} L_{\text{BCE}}(v; \theta) + \lambda_{\text{sp}} L_{\text{sp}}(v; \theta) + \lambda_{\text{pl}} L_{\text{pl}}(v; \theta)]. \quad (4)$$

Here L_{BCE} shapes the OT margin $m(v)$, L_{sp} penalizes diffuse transport plans so that decoded edits are sparse, and L_{pl} penalizes transport mass assigned to feature mismatches that violate degree or temporal plausibility rules. Appendix §C.6 gives the full definitions and Algorithm 1 gives the training procedure.

4.3. Boundary Cloak Templates

Let $\mathcal{V}_{\text{bot}}$ and $\mathcal{V}_{\text{hum}}$ denote the sets of bot and human nodes as identified by the ground truth. For $v \in \mathcal{V}_{\text{bot}}$, define the closest human and bot distances by $d_{\text{hum}}(v) = \min_{h \in \mathcal{V}_{\text{hum}}} D_{\theta}(v, h)$, and $d_{\text{bot}}(v) = \min_{\xi \in \mathcal{V}_{\text{bot}} \setminus \{v\}} D_{\theta}(v, \xi)$, and let the OT margin be $m(v) = d_{\text{hum}}(v) - d_{\text{bot}}(v)$, (details in Appendix §C.5 and ③ of Figure 2). Large positive $m(v)$ means v is closer to other bots than to humans in OT space; small or negative values indicate proximity to the human region.

We then enforce a margin threshold $m(v) \leq \tau_{\text{bdry}}$ to select bot nodes whose neighborhoods are closer to human nodes than to other bots under D_{θ} , further details in Appendix §C.7. We define the boundary set as $\mathcal{B}_{\text{bdry}} = \{v \in \mathcal{V}_{\text{bot}} : m(v) \leq \tau_{\text{bdry}}\}$, where τ_{bdry} is a validation-set threshold. Using a pre-trained node classifier f_{Θ} as a black-box, we define the set of misclassified bots that the classifier predicts as human: $\mathcal{B}_{\text{mis}} = \{v \in \mathcal{V} : y_v = \text{bot} \wedge \arg \max_y p_{\Theta}(y | v; \mathcal{G}) = \text{human}\}$.

The cloak candidate set in Equation 5 can be instantiated as $\mathcal{B}_{\text{cand}} = \mathcal{B}_{\text{bdry}} \cap \mathcal{B}_{\text{mis}}$ (as shown in ④ of Figure 2). These nodes serve as cloak templates because they already satisfy the detector decision rule on the current graph. After training the OT geometry (details in Appendix §C.6 and algorithm in Algorithm 1), we compute $D_{\theta}(v, \xi)$ for the pairs needed to evaluate $d_{\text{hum}}(v)$ and $d_{\text{bot}}(v)$ on the bot set. We then form $\mathcal{B}_{\text{bdry}}$ and $\mathcal{B}_{\text{mis}}$ and select a small pool of cloak templates for each target. This restriction limits the number of OT iterations required during attack generation.

4.4. Attack Objectives and Plans

We consider two attack scenarios: node injection and editing. In node injection (Figure 3a), the attacker introduces a new bot node $v_t \notin \mathcal{V}$ and adds a set of outgoing edges incident to v_t . In node editing (Figure 3b), the attacker selects an existing bot node $v_t \in \mathcal{V}$ and modifies its incident edges through $\Delta\mathcal{E}$. Node injection can be viewed as a special case of node editing in which v_t initially has no incident edges.

Conceptual Objective. The attacker aims to construct a neighborhood around v_t that resembles a cloak template while respecting feasibility (details in Appendix §C.8). Let μ_t denote the neighborhood measure of v_t after applying $\Delta\mathcal{E}$, where $\mu_t(\Delta\mathcal{E})$ is explicit dependency. BOCLOAK uses the following OT-guided objective as a target specification:

$$\min_{\Delta\mathcal{E} \in \mathcal{F}(B)} \min_{v_c \in \mathcal{B}} D_{\theta}(\mu_t(\Delta\mathcal{E}), \mu_c) + \lambda_{\text{pl}} \Phi(\Delta\mathcal{E}) + \lambda_{\text{sp}} |\Delta\mathcal{E}| \quad (5)$$

where $\mathcal{F}(B)$ encodes the edge budget and hard feasibility constraints, $\Phi(\Delta\mathcal{E})$ penalizes implausible edits under domain rules, and $|\Delta\mathcal{E}|$ enforces sparsity. The inner minimiza-

tion selects a cloak template v_c from bot candidates $\mathcal{B}_{\text{cand}}$.

4.5. Approximation Strategy

Direct optimization of Equation 5 is not tractable because $\Delta\mathcal{E}$ ranges over discrete edge sets and the objective depends on neighborhood measures induced by those edits. BOCLOAK first learns D_{θ} offline, then identifies a small candidate pool of cloak templates near the detector boundary, and finally converts a transport plan into concrete edge edits; full loss objective is described in Appendix §C.6, Equation 20. The decoding step maps an OT plan to $\Delta\mathcal{E}$ under the budget and plausibility constraints.

4.6. Decoding OT Plans into Discrete Edges

Given a target bot v_t , BOCLOAK selects a cloak v_c from the candidate set $\mathcal{B}_{\text{bdry}}$ (BOCLOAK algorithm described in Algorithm 2 and shown in ⑤ of Figure 2). Although v_c is itself a bot, the victim model predicts it as human, placing it near or within the human region of the learned OT geometry. To identify which parts of its neighborhood contribute most to this human-like appearance, we compare v_c to a nearby human $v_h \in \arg \min_{h \in \mathcal{V}_{\text{hum}}} D_{\theta}(v_c, h)$.

BOCLOAK computes the entropic OT plan P_{ch}^* between the neighborhood distributions μ_c and μ_h using Sinkhorn iterations under the learned cost c_{θ} (details in Appendix §C.4). This plan aligns neighbors of v_c with those of v_h according to structural, behavioral, and temporal similarity.

From this OT plan, BOCLOAK extracts a small set of high-mass cloak-side neighbors and converts them into the target bot’s incident edges, subject to the budget B and feasibility constraints. In node injection, these edges define the initial neighborhood of a new node. In node editing, they replace or augment a subset of existing incident edges (as shown in ⑥ of Figure 2). This decoding step yields a discrete set $\Delta\mathcal{E}$ that approximates the continuous objective in Equation 5.

Why Bot-Cloaks? We prefer misclassified bots over true humans as cloak templates because existing bots are more likely to maintain connections aligned with bot objectives, such as promotion or coordination. Mimicking arbitrary humans may yield bots that evade detection but fail to generate meaningful engagement.

Special Case. If no bot cloaks are available, BOCLOAK falls back to selecting a nearby human in OT space and constructs an evading bot by mimicking that human’s neighborhood.

Complexity. Let m and n denote the neighborhood sizes of μ_v and μ_{ξ} . One Sinkhorn OT solve between them costs $O(T_{\text{sink}}mn)$ time and $O(mn)$ space, which dominates BOCLOAK’s execution (details in Appendix §D.4). The dependence is on local ego-neighborhood density rather than the full graph size. For very large social graphs or high-degree

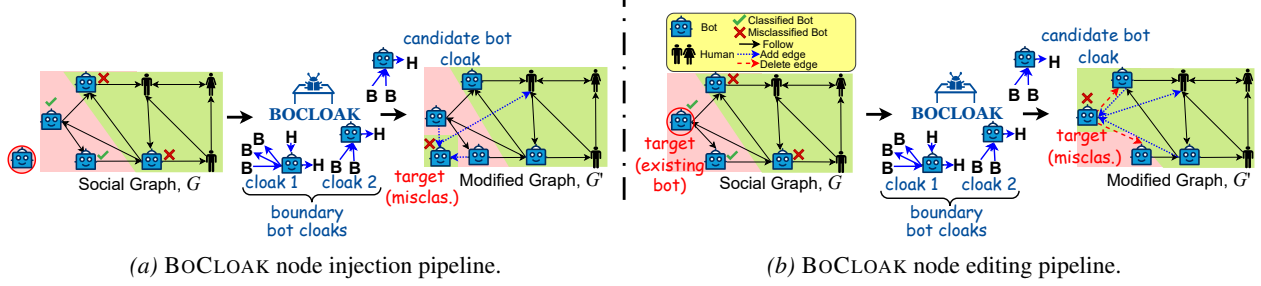


Figure 3. BOCLOAK pipeline for node injection and node editing. It operates on local neighborhoods rather than individual edges. Boundary bot cloaks are identified in the learned optimal transport geometry and used to guide sparse, plausible edge edits. (a) Injection constructs a new bot with a human-like neighborhood. (b) Editing transforms an existing bot’s neighborhood to induce misclassification.

nodes, BOCLOAK can use the same formulation with two practical truncations: retain only the top- K boundary templates and keep only the highest-weight neighbors when forming each empirical measure.

5. Experiments

In this section, we present experimental results that demonstrate the effectiveness and practicality of the BOCLOAK.

Dataset	#Nodes	#Edges	#Bots	#Humans
Cresci-15	5,301	14,220	3,351	1,950
Twibot-22	1,000,000	170,185,937	139,943	860,057
BotSim-24	2,907	46,518	1,000	1,907

Table 1. **Dataset Statistics:** Summary of bot detection datasets. Detailed statistics are in Appendix Table 9.

Datasets. We evaluate BOCLOAK on three widely used social bot detection datasets: TwiBot-22 (Feng et al., 2022b), a large-scale graph-based benchmark featuring heterogeneous relations and high-quality annotations; Cresci-2015 (Cresci et al., 2015), which includes early social spambots, fake followers, and genuine users with explicit interaction graphs; and BotSim-24 (Qiao et al., 2025), a recent benchmark that simulates highly human-like social botnets. The statistics of the datasets are summarized in Table 1.

Bot Detection Models. We employ the GNN-based node classifiers, such as GCN (Kipf & Welling, 2017) and GAT (Veličković et al., 2018) as baseline methods. We adopt the prominent BotRGCN (Feng et al., 2021b), SimpleHGNN (S-HGN) (Lv et al., 2021), and RGT (Feng et al., 2022a) models as SOTA bot detectors.

GNN Attack Models. We compare BOCLOAK with random baselines and SOTA adversarial attack frameworks, including Nettack (Zügner et al., 2018), FGA (Chen et al., 2018), PR-BCD (Geisler et al., 2021), GOttack (Alom et al., 2025), and the LR-BCD control (Gosch et al., 2023) across both vanilla bot detectors and their defended counterparts, including GNNGuard (Zhang & Zitnik, 2020), GRAND (Feng

et al., 2020), and RobustGCN (Zhu et al., 2019) for node editing attack. We additionally evaluate NoisyGNN (Ennadir et al., 2024) as a robustness-control defense on the strongest BotRGCN setting. We compare BOCLOAK with random baselines for node injection attack since no prior node-injection attack for social graphs is directly comparable under our feasibility rules.

For each dataset, we follow the original training protocols for these detectors, including train/validation splits and early-stopping criteria. We further verify that our implementations match or exceed the reported clean accuracies before evaluating adversarial attacks, as summarized in Table 10. We follow the official train/validation/test as described by dataset authors (Feng et al., 2021a; 2022b; Qiao et al., 2025).

Experimental Setup. We evaluate BOCLOAK under two scenarios: *node editing* and *node injection*. In node editing, we uniformly sample 50 correctly classified bot nodes, reset the graph to the original (unperturbed) graph before attacking each target, and allow both edge additions and deletions, but no human follow-back. This follows the standard protocol used in prior works (both Nettack and GOttack), but we use 50 targets to improve statistical stability. In node injection, we follow the same regime as editing, but we introduce a new bot into the graph and connect it either randomly (since there are no competitors) or using a bot cloak from BOCLOAK; only edge additions are allowed, and no human-follow-back edge can be created.

Common constrained protocol. For the constrained columns, every attack receives the same per-target budget Δ and the same feasible set $\mathcal{F}(\Delta)$ from §3.1. Baseline attacks produce their ranked perturbation proposals using their original optimization procedures and dataset loaders adapted to our graphs. We then apply a deterministic feasibility wrapper: invalid edits are rejected, and the baseline continues down its own ranked proposal list until either Δ admissible target-incident edits are applied or the list is exhausted. We do not rerank, hand-pick, or add BOCLOAK-specific information to baseline proposals. For FGA and PR-BCD, we

Dataset	Attack	GAT					BotRGCN					S-HGN				
		Unconstrained		Constrained			Unconstrained		Constrained			Unconstrained		Constrained		
		Vanilla	Vanilla	+GNNGuard	+GRAND	+RobustGCN	Vanilla	Vanilla	+GNNGuard	+GRAND	+RobustGCN	Vanilla	Vanilla	+GNNGuard	+GRAND	+RobustGCN
Cresci-15	Nettack	95.45$\pm$2.22	8.17 $\pm$ 3.23	6.59 $\pm$ 3.40	<u>7.54$\pm$2.07</u>	7.28 $\pm$ 3.19	94.67 $\pm$ 6.55	<u>4.32$\pm$2.10</u>	<u>3.78$\pm$1.95</u>	<u>3.21$\pm$1.88</u>	<u>2.67$\pm$1.72</u>	<u>94.11$\pm$2.45</u>	6.04 $\pm$ 3.00	4.81 $\pm$ 3.18	4.51 $\pm$ 3.60	4.88 $\pm$ 2.71
	FGA	94.75 $\pm$ 1.50	8.52 $\pm$ 1.92	5.74 $\pm$ 2.39	4.73 $\pm$ 3.67	<u>7.93$\pm$2.91</u>	96.89 $\pm$ 4.81	3.95 $\pm$ 2.44	3.40 $\pm$ 2.12	2.98 $\pm$ 1.76	2.15 $\pm$ 1.60	92.54 $\pm$ 1.23	5.22 $\pm$ 3.17	4.96 $\pm$ 3.09	4.31 $\pm$ 2.07	3.44 $\pm$ 2.01
	PR-BCD	95.02 $\pm$ 1.38	<u>8.79$\pm$1.91</u>	6.85 $\pm$ 2.35	7.48 $\pm$ 2.37	7.92 $\pm$ 3.24	96.36 $\pm$ 8.36	4.10 $\pm$ 3.05	3.60 $\pm$ 2.80	2.85 $\pm$ 2.20	1.90 $\pm$ 1.45	94.12 $\pm$ 1.23	7.57 $\pm$ 3.84	6.91 $\pm$ 2.43	6.70 $\pm$ 2.84	5.58 $\pm$ 2.45
	Gottack	94.98 $\pm$ 1.11	7.61 $\pm$ 2.91	<u>7.14$\pm$1.65</u>	5.08 $\pm$ 3.06	6.20 $\pm$ 1.94	<u>98.54$\pm$12.40</u>	<u>3.55$\pm$2.34</u>	3.41 $\pm$ 2.52	2.28 $\pm$ 1.22	2.66 $\pm$ 1.67	94.12 $\pm$ 1.21	<u>8.10$\pm$2.83</u>	<u>7.57$\pm$2.04</u>	<u>7.92$\pm$1.87</u>	<u>6.39$\pm$2.12</u>
	BOCLOAK (ours)	<u>95.25$\pm$1.11</u>	93.10$\pm$2.34	94.20$\pm$1.89	92.45$\pm$3.11	91.88$\pm$4.56	<u>99.34$\pm$0.29</u>	<u>99.34$\pm$0.29</u>	<u>98.91$\pm$0.54</u>	<u>99.06$\pm$0.41</u>	<u>99.72$\pm$0.18</u>	95.80$\pm$2.50	92.18$\pm$4.73	91.60$\pm$2.15	91.92$\pm$3.08	91.40$\pm$1.67
Twibot-22	Nettack	81.22 $\pm$ 1.23	13.51 $\pm$ 8.15	11.92 $\pm$ 9.38	8.59 $\pm$ 7.73	<u>12.91$\pm$8.91</u>	79.77 $\pm$ 6.77	9.33 $\pm$ 3.06	<u>8.67$\pm$4.62</u>	4.67 $\pm$ 5.03	8.00 $\pm$ 5.29	83.51 $\pm$ 1.22	5.03 $\pm$ 4.72	3.24 $\pm$ 9.20	3.72 $\pm$ 10.81	4.73 $\pm$ 7.85
	FGA	73.75 $\pm$ 1.22	12.32 $\pm$ 8.10	8.59 $\pm$ 4.52	9.94 $\pm$ 6.41	5.57 $\pm$ 10.78	71.90 $\pm$ 4.23	4.00 $\pm$ 2.00	7.33 $\pm$ 2.31	6.67 $\pm$ 4.16	1.33 $\pm$ 6.43	75.85 $\pm$ 2.54	5.79 $\pm$ 6.52	4.57 $\pm$ 4.96	3.41 $\pm$ 6.87	3.19 $\pm$ 9.89
	PR-BCD	73.86 $\pm$ 1.50	<u>13.51$\pm$4.52</u>	<u>12.52$\pm$9.38</u>	<u>11.32$\pm$6.41</u>	12.24 $\pm$ 4.52	75.77 $\pm$ 6.05	8.67 $\pm$ 2.31	8.67 $\pm$ 5.03	6.23 $\pm$ 4.16	7.33 $\pm$ 2.31	78.12 $\pm$ 1.98	4.58 $\pm$ 5.29	3.74 $\pm$ 3.34	3.61 $\pm$ 7.20	3.20 $\pm$ 7.92
	Gottack	<u>82.12$\pm$2.56</u>	11.41 $\pm$ 7.59	10.50 $\pm$ 6.38	9.78 $\pm$ 3.93	10.33 $\pm$ 5.34	<u>84.29$\pm$11.63</u>	6.89 $\pm$ 3.33	7.15 $\pm$ 4.25	6.01 $\pm$ 2.48	5.32 $\pm$ 2.58	80.85 $\pm$ 2.30	5.55 $\pm$ 7.98	4.04 $\pm$ 8.93	4.44 $\pm$ 7.82	4.95 $\pm$ 8.39
	BOCLOAK (ours)	<u>95.25$\pm$2.50</u>	94.38$\pm$3.41	93.72$\pm$1.66	93.90$\pm$4.28	95.12$\pm$3.33	86.67$\pm$2.31	86.67$\pm$2.31	84.67$\pm$11.02	83.67$\pm$3.06	87.33$\pm$1.15	98.12$\pm$1.75	97.42$\pm$1.39	96.15$\pm$3.74	95.90$\pm$2.06	95.11$\pm$4.28
BotSim-24	Nettack	53.77 $\pm$ 4.12	5.52 $\pm$ 1.92	3.74 $\pm$ 2.39	4.73 $\pm$ 3.67	4.93 $\pm$ 8.10	56.57 $\pm$ 6.63	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	3.00 $\pm$ 2.00	55.54 $\pm$ 5.55	4.71 $\pm$ 2.19	3.44 $\pm$ 3.09	3.35 $\pm$ 2.67	3.65 $\pm$ 7.44
	FGA	52.09 $\pm$ 1.59	5.79 $\pm$ 1.91	4.85 $\pm$ 2.35	4.48 $\pm$ 2.37	4.92 $\pm$ 3.24	55.67 $\pm$ 6.96	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	54.98 $\pm$ 1.33	0.00 $\pm$ 0.00	50.41 $\pm$ 2.50	5.72 $\pm$ 2.56	<u>4.63$\pm$2.12</u>	<u>5.06$\pm$3.15</u>	<u>4.54$\pm$3.73</u>
	PR-BCD	51.26 $\pm$ 4.11	<u>7.32$\pm$8.10</u>	<u>6.59$\pm$4.52</u>	6.65 $\pm$ 7.59	<u>7.14$\pm$3.65</u>	50.62 $\pm$ 8.04	<u>2.00$\pm$2.00</u>	<u>5.33$\pm$2.31</u>	<u>0.67$\pm$1.15</u>	0.00 $\pm$ 0.00	52.11 $\pm$ 2.50	5.20 $\pm$ 8.04	4.24 $\pm$ 4.63	4.43 $\pm$ 4.99	4.26 $\pm$ 1.88
	Gottack	91.50$\pm$1.50	7.08 $\pm$ 3.06	6.20 $\pm$ 1.94	6.67 $\pm$ 3.09	5.87 $\pm$ 2.27	<u>57.18$\pm$2.32</u>	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	<u>88.33$\pm$1.33</u>	4.42 $\pm$ 1.83	3.25 $\pm$ 3.38	3.65 $\pm$ 3.12	3.21 $\pm$ 2.45
	BOCLOAK (ours)	90.19 $\pm$ 1.50	88.74$\pm$3.12	90.05$\pm$1.47	89.30$\pm$4.66	88.40$\pm$2.29	58.22$\pm$7.20	58.22$\pm$7.20	52.68$\pm$1.15	54.98$\pm$1.45	55.10$\pm$3.98	90.75$\pm$2.33	89.16$\pm$4.25	90.44$\pm$2.11	89.78$\pm$1.63	88.90$\pm$3.74

Table 2. Node Editing Attack with Budget $\Delta = 1$: Misclassification rate ($\uparrow$, in %) for flipping fifty correctly classified bots by BOCLOAK and SOTA adversarial attacks against best SOTA bot detectors: GAT, BotRGCN, and S-HGN, with adversarial defenses. For results against other bot detectors, see App. 12, see App. 18 when the same bot cloak is reused, see App. 19 when no domain constraints are enforced. The best performance is shown in bold, and the second best is underlined.

also evaluate in-loop projection controls that project candidate perturbations onto the bot $\rightarrow$ human admissible edge set during optimization.

Hyper-parameters. Hyperparameters of bot detectors and their defense variants are provided in App. Tables 6 and 7; SOTA adversarial attack framework are provided in App. Table 8. BOCLOAK uses 1-hop ego-neighborhoods, standard in other domains for capturing behavioral signals (Dong et al., 2024; Mukherjee et al., 2025).

Computational Environment. Our implementation release: <https://github.com/kunmukh/bocloak>. Experiments were run on a Linux cluster with an AMD EPYC 7313 16-Core CPU, 1 TiB RAM, and 8 NVIDIA L40S 46 GB GPUs.

Evaluation Criteria. For a fixed black-box detector f , per-bot edge budget Δ , and attack $\mathcal{A}$, we focus on a set of target bots $\mathcal{V}_t$ that are correctly classified on the social graph G , i.e., $f(G, v_t) = \text{bot} \forall v_t \in \mathcal{V}_t$. In node editing, v_t is an *existing bot* and in node injection, v_t is a *new bot* to be injected by BOCLOAK. The perturbed graph G' contains both edge additions/deletions for bot editing but only edge addition for bot injection. Given a target $v_t \in \mathcal{V}_t$, the attack $\mathcal{A}$ constructs a perturbed graph G' by applying at most Δ edge edits incident to v_t . v_t is *flipped* if $f(G', v_t) = \text{human}$ while $f(G, v_t) = \text{bot}$, and define the indicator: $\text{BotFlip}(v_t) = \mathbb{I}[f(G, v_t) = \text{bot} \wedge f(G', v_t) = \text{human}]$. We use the misclassification rate: $\frac{1}{|\mathcal{V}_t|} \sum_{v \in \mathcal{V}_t} \text{BotFlip}(v)$, as the primary performance metric for evaluation. Higher misclassification values indicate poorer detector performance and greater attack effectiveness. In all the results, the best performance is in bold, and the second best is underlined.

5.1. Node Editing Results

We give the budget $\Delta = 1$ node editing results against GAT and two SOTA bot detectors in Table 2. For results with all

budgets and other bot detectors, see App. 12, and for results with reuse of the same bot cloak, see App. 18.

Table 2 shows that unconstrained SOTA attacks can be effective, but their performance drops sharply once we enforce social-domain constraints. This drop is expected because generic graph attacks often rely on perturbations that are high-impact mathematically but infeasible operationally, such as rewiring unrelated users, violating edge directionality, or inducing arbitrary human $\rightarrow$ bot follow-backs.

To verify that this gap is not simply caused by a weak post-hoc filtering implementation, we additionally evaluate constrained control variants in Appendix R. In particular, FGA-mod and PR-BCD-mod add an in-loop projection step onto the admissible bot $\rightarrow$ human edit set, LR-BCD adds a newer local-budget attack baseline, and Homophily replaces OT-guided coupling with embedding/homophily-based matching. As shown in Table 20, these controls improve some constrained baselines only modestly and remain far below BOCLOAK, indicating that the performance gap is driven by the mismatch between generic graph perturbation objectives and domain-faithful social-bot constraints. We further test NoisyGNN as an additional robustness-control defense on TwiBot-22/BotRGCN; results are reported in App. Table 11. NoisyGNN reduces constrained generic attacks to below 6% misclassification, but BOCLOAK still achieves 85.12%, showing that hidden-state noise alone does not remove the domain-feasible neighborhood vulnerability exploited by BOCLOAK.

Within the constrained setting, increasing budget improves SOTA only modestly (e.g., on Cresci-15 their rates rise from 3–9% at $\Delta=1$ to 12–23% at $\Delta=5$), but they remain far below BOCLOAK. By contrast, BOCLOAK consistently achieves **80.13%** higher success attack rates than the best-performing constrained SOTA.

An insight is that detector choice matters more for SOTA attacks than for BOCLOAK: SOTA can reach the low-20%

range on BotRGCN for some settings at $\Delta=5$, but still fails to approach BOCLOAK, suggesting BOCLOAK learns transferable “bot cloaks” rather than brittle, model-specific perturbations. Standard defenses only weakly affect BOCLOAK on Cresci-15 and TwiBot-22, with scores clustering tightly across vanilla and defended detectors, whereas constrained SOTA attacks stay uniformly low because the feasible edit space removes many of their exploitable directions. BOCLOAK is consistently the strongest attack model and remains highly effective even when constraints are active.

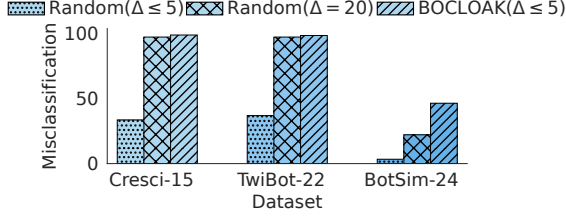


Figure 4. **Node Injection Attack:** Misclassification rate ($\uparrow$, in %) for fifty target nodes by BOCLOAK against best SOTA bot detector, BotRGCN. Extended results in App. Table 13 and Table 15.

Node Injection. Node injection results are shown in Figure 4. For extended results, see App. 13 and App. 15, and for results when the same bot cloak is reused, see App. 14 and App. 16. On Cresci-15 and TwiBot-22, BOCLOAK achieves near-perfect injection success with very small budgets (high-90% with 1-3 edges), and additional edges yield only marginal gains, suggesting the decision boundary is easy to cross once a minimally human-like ego-neighbourhood is formed.

Overall, injection robustness can look deceptively “solved” on easier benchmarks (where even random succeeds at high budget), whereas BOCLOAK’s OT geometry- and plausibility-aware construction is most revealing on harder datasets like BotSim-24 where random search fails and success scales smoothly with budget.

Dataset	$-\lambda_{\text{BCE}}$	$-\lambda_{\text{sp}}$	$-\lambda_{\text{pl}}$	$-\alpha_{\text{deg}}$	$-\alpha_{\text{age}}$	BOCLOAK
Cresci-17	38.21 $\pm$ 5.12	96.88 $\pm$ 5.89	57.99 $\pm$ 3.38	73.23 $\pm$ 0.23	87.23 $\pm$ 1.78	99.67 $\pm$ 0.20
TwiBot-22	22.45 $\pm$ 6.44	92.12 $\pm$ 7.65	55.23 $\pm$ 6.77	70.98 $\pm$ 4.29	83.88 $\pm$ 6.63	94.00 $\pm$ 2.00
BotSim-24	32.45 $\pm$ 5.64	96.55 $\pm$ 0.66	60.34 $\pm$ 5.09	69.22 $\pm$ 2.15	85.24 $\pm$ 1.38	99.33 $\pm$ 1.15

Table 3. **Ablation Study:** Misclassification rate ($\uparrow$, in %) for node editing attack against vanilla BotRGCN with budget Δ of 5. Extended result in App. 21.

Ablation Studies. Table 3 (extended in App. 21) shows that the OT-margin BCE surrogate is the primary contributor: removing it leads to a pronounced success drop, confirming its role in enforcing a separable OT margin that keeps misclassified boundary cloaks close to the human region.

Sensitivity Analysis. Figure 5 illustrates that BOCLOAK is most effective at intermediate OT regularization and margin threshold. With very small ϵ (e.g., 0.01), transport plans

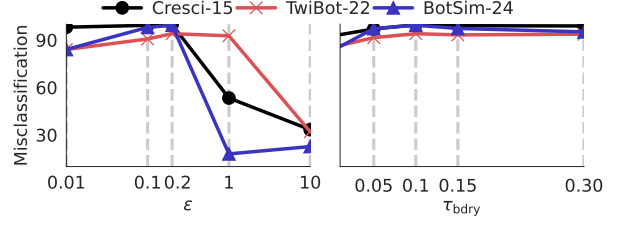


Figure 5. **Sensitivity Analysis of OT regularizer (ϵ) and margin threshold (τ_{bdry})** Misclassification rate ($\uparrow$, in %) for flipping fifty existing correctly classified bots as ϵ and τ_{bdry} is varied. Extended results in App. 22.

become sharp and unstable, leading to higher variance and moderate flips (about 83–85%), and large (≥ 10), the plan becomes diffuse, blurring the OT-induced margin. Varying the margin threshold τ_{bdry} shows a similar optimal value: around 0.10, where it maximizes flip rates by enforcing a meaningful OT-margin without over-constraining the search.

Attack	BotSim-24		
	Time (sec) ($\downarrow$)	RAM (MB) ($\downarrow$)	GPU (MB) ($\downarrow$)
Nettack	19.7 $\pm$ 1.65	2973 $\pm$ 628	156 $\pm$ 40
FGA	0.3 $\pm$ 1.57	1720 $\pm$ 13	286 $\pm$ 2
PR-BCD	2.1 $\pm$ 0.75	1857 $\pm$ 21	286 $\pm$ 1
GOTack	7.5 $\pm$ 1.75	2069 $\pm$ 623	240 $\pm$ 2
BOCLOAK	0.1 $\pm$ 0.13	1450 $\pm$ 2	11 $\pm$ 1

Table 4. **System Overhead:** System resource usage by BOCLOAK and SOTA adversarial attacks against vanilla BotRGCN with budget Δ of 1. Extended result in App. 23.

Overhead Analysis. Table 4 describes the resource usage. BOCLOAK incurs 99.80% less GPU footprint (13MB) because it does not require GPU computation and runs up to 20 $\times$ faster than SOTA attacks, extended in App 23.

Limitation. We assume fixed feature vectors throughout the study. In practice, social network content, such as tweets, changes over time and can affect both detection and evasion. However, modeling these effects requires temporally resolved content and metadata, which are not yet available.

6. Conclusion

We introduced BOCLOAK, which constructs realistic k -hop neighbourhood under directional, social, and temporal constraints. BOCLOAK is a geometry-first attack framework that learns a label-aware optimal transport ground cost that explicitly separates human and bot behavioral regions. Across three datasets, five victim detectors with three defenses, and compared against four SOTA adversarial attacks, BOCLOAK consistently achieves 80.13% higher evasion success under realistic constraints and using 99.80% less GPU memory. This work closes a key realism gap in the robustness evaluation of social bot detectors.

Acknowledgements

We thank the anonymous reviewers for their helpful feedback. The research reported here in were supported in part by NSF awards DMS-2204795, OAC-2115094, CNS-2331424, ITE-2452833, ARL/Army Research Office awards W911NF-24-1-0202 and W911NF-24-2-0114, and Virginia Commonwealth Cyber Initiative grants.

Impact Statement

This paper presents research whose goal is to advance the field of machine learning, with a focus on understanding the robustness and limitations of graph-based bot detection systems. Our work studies constrained adversarial attacks in order to characterize failure modes of existing detectors and to motivate the design of more reliable and resilient defenses.

While the techniques introduced here could, in principle, be misused to evade deployed detection systems, the intent of this work is diagnostic rather than operational. By revealing how bot accounts can exploit neighborhood-level vulnerabilities, our results provide insights that can help platform designers and researchers strengthen detection models and improve trustworthiness. We do not release tools for large-scale misuse, and the attacks studied operate under restrictive assumptions intended to mirror realistic constraints.

Overall, we believe the primary societal impact of this work is to contribute to safer and more robust machine learning systems for online platforms. We do not foresee immediate negative societal consequences beyond those already well understood in the study of adversarial machine learning.

References

- Akhondzadeh, M. S., Zargarbashi, S. H., Cao, J., and Bojchevski, A. EvA: Evolutionary attacks on graphs. In *International Conference on Learning Representations (ICLR)*, 2026. URL <https://openreview.net/forum?id=EzXzGRngYb>.
- Alom, Z., Ngo, T. G. B., Kantarcioglu, M., and Akcora, C. G. GOTTack: Universal adversarial attacks on graph neural networks via graph orbits learning. In *The Thirteenth International Conference on Learning Representations (ICLR)*, Singapore, 2025.
- Bacciu, D., Errica, F., Micheli, A., and Podda, M. A gentle introduction to deep learning for graphs. *Neural Networks*, 129:203–221, 9 2020.
- Chen, J., Wu, T., Xu, Z., Chen, Y., Zheng, K., Li, Z., and Liu, J. Fast gradient attack on network embedding. In *arXiv preprint*, 2018. arXiv:1809.02797.
- Cresci, S., Di Pietro, R., Petrocchi, M., Spognardi, A., and Tesconi, M. Fame for sale: Efficient detection of fake twitter followers. *Decision Support Systems*, 80:56–71, 2015.
- Cresci, S., Petrocchi, M., Spognardi, A., and Tognazzi, S. Better safe than sorry: An adversarial approach to improve social bot detection. In *Proceedings of the 11th ACM Web Science Conference (WebSci’19)*, pp. 47–56, Boston, MA, USA, 2019. ACM.
- Cresci, S., Petrocchi, M., Spognardi, A., and Tognazzi, S. The coming age of adversarial social bot detection. *First Monday*, 26(6), 2021. doi: 10.5210/fm.v26i7.11474.
- Cuturi, M. Sinkhorn distances: Lightspeed computation of optimal transport. *Advances in neural information processing systems*, 26, 2013.
- Dong, K., Guo, Z., and Chawla, N. Pure message passing can estimate common neighbor for link prediction. *Advances in Neural Information Processing Systems*, 37: 73000–73035, 2024.
- Ennadir, S., Abbahaddou, Y., Lutzeyer, J. F., Vazirgiannis, M., and Boström, H. A simple and yet fairly effective defense for graph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 21063–21071, 2024. doi: 10.1609/aaai.v38i19.30098.
- Feng, S., Wan, H., Wang, N., Li, J., and Luo, M. Twibot-20: A comprehensive twitter bot detection benchmark. In *Proceedings of the 30th ACM International Conference on Information and Knowledge Management (CIKM)*, pp. 4485–4494, 2021a. doi: 10.1145/3459637.3482019.
- Feng, S., Wan, H., Wang, N., and Luo, M. Botrgcn: Twitter bot detection with relational graph convolutional networks. In *Proceedings of the IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM)*, pp. 236–243, 2021b.
- Feng, S., Tan, Z., Li, R., and Luo, M. Heterogeneity-aware twitter bot detection with relational graph transformers. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2022a.
- Feng, S., Tan, Z., Wan, H., Wang, N., Chen, Z., Zhang, B., Zheng, Q., Zhang, W., Lei, Z., Yang, S., et al. Twibot-22: Towards graph-based twitter bot detection. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2022b.
- Feng, W., Zhang, J., Dong, Y., Han, Y., Luan, H., Xu, Q., Yang, Q., Kharlamov, E., and Tang, J. Graph random neural network for semi-supervised learning on graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

- Ferrara, E., Varol, O., Davis, C. A., Menczer, F., and Flammini, A. The rise of social bots. *Communications of the ACM*, 59(7):96–104, 2016. doi: 10.1145/2818717.
- Flamary, R., Courty, N., Gramfort, A., Alaya, M. Z., Boissunon, A., Chambon, S., Chapel, L., Corenflos, A., Fatras, K., Fournier, N., et al. Pot: Python optimal transport. *Journal of Machine Learning Research*, 22(78):1–8, 2021.
- Geisler, S., Schmidt, T., Şirin, H., Zügner, D., Bojchevski, A., and Günnemann, S. Robustness of graph neural networks at scale. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- Gosch, L., Geisler, S., Sturm, D., Charpentier, B., Zügner, D., and Günnemann, S. Adversarial training for graph neural networks: Pitfalls, solutions, and new directions. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems*, volume 36, pp. 58088–58112. Curran Associates, Inc., 2023.
- Kipf, T. N. and Welling, M. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.
- Kudugunta, S. and Ferrara, E. Deep neural networks for bot detection. In *Proceedings of the 2018 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM)*, pp. 551–558, 2018. doi: 10.1109/ASONAM.2018.8508764.
- Liu, A., Xie, Y., Wang, L., Jin, G., Guo, J., and Li, J. Social bot detection on Twitter: Robustness evaluation and improvement. *Multimedia Systems*, 30(3), 2024. doi: 10.1007/s00530-024-01364-2.
- Liu, Y., Xu, S., Li, Y., and Yu, S. Evolution of malicious social bot detection: From individual profiling to group analysis and beyond. *Journal of Social Computing*, 6(3):258–284, September 2025. doi: 10.23919/JSC.2025.0017.
- Lv, Q., Ding, M., Liu, Q., Chen, Y., Feng, W., He, S., Zhou, C., Jiang, J., Dong, Y., and Tang, J. Are we really making much progress? revisiting, benchmarking and refining heterogeneous graph neural networks. In *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining*, pp. 1150–1160, 2021.
- Mahalanobis, P. C. On the generalised distance in statistics. *Proceedings of the National Institute of Sciences of India*, 2(1):49–55, 1936.
- Mukherjee, K., Harrison, Z., and Balaneshin, S. Z-rex: human-interpretable gnn explanations for real estate recommendations. *arXiv preprint arXiv:2503.18001*, 2025.
- Petric Maretic, H., El Gheche, M., Chierchia, G., and Frossard, P. Got: an optimal transport framework for graph comparison. *Advances in Neural Information Processing Systems*, 32, 2019.
- Peyré, G. and Cuturi, M. Computational optimal transport: With applications to data science. *Foundations and Trends® in Machine Learning*, 11(5–6):355–607, 2019. doi: 10.1561/22000000073. URL <https://doi.org/10.1561/22000000073>.
- Qiao, B., Li, K., Zhou, W., Li, S., Lu, Q., and Hu, S. Bot-sim: LLM-powered malicious social botnet simulation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025.
- Tao, T. *An Introduction to Measure Theory*, volume 126 of *Graduate Studies in Mathematics*. American Mathematical Society, Providence, RI, 2011. ISBN 978-0-8218-6919-2. URL <https://bookstore.ams.org/gsm-126>.
- Varol, O., Ferrara, E., Davis, C. A., Menczer, F., and Flammini, A. Online human-bot interactions: Detection, estimation, and characterization. In *Proceedings of the 11th International AAAI Conference on Web and Social Media (ICWSM)*, pp. 280–289, 2017.
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., and Bengio, Y. Graph attention networks. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=rJXMpikCZ>.
- Wang, L., Qiao, X., Xie, Y., Nie, W., Zhang, Y., and Liu, A. My brother helps me: Node injection based adversarial attack on social bot detection. In *Proceedings of the 31st ACM International Conference on Multimedia (MM’23)*, pp. 6705–6714. ACM, 2023.
- Wang, L., Qiao, X., Xie, Y., Nie, W., Zhang, Y., and Liu, A. Node injection-based adversarial attack and defense on social bot detection. *IEEE Transactions on Computational Social Systems*, 2025. doi: 10.1109/TCSS.2025.3599730.
- Zhang, C., He, Y., Cen, Y., Hou, Z., Feng, W., Dong, Y., Cheng, X., Cai, H., He, F., and Tang, J. Scr: Training graph neural networks with consistency regularization. *arXiv preprint arXiv:2112.04319*, 2021. doi: 10.48550/arXiv.2112.04319. URL <https://arxiv.org/abs/2112.04319>.
- Zhang, X. and Zitnik, M. GNNGuard: Defending graph neural networks against adversarial attacks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pp. 9263–9275, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/>

[690d83983a63aa1818423fd6edd3bfdb-Abstract.html](#).

Zhu, D., Zhang, Z., Cui, P., and Zhu, W. Robust graph convolutional networks against adversarial attacks. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2019.

Zhu, G., Chen, M., Yuan, C., and Huang, Y. Simple and efficient partial graph adversarial attack: A new perspective. *IEEE Transactions on Knowledge and Data Engineering*, 36(8):3683–3696, 2024. doi: 10.1109/TKDE.2024.3364972.

Zügner, D., Akbarnejad, A., and Günnemann, S. Adversarial attacks on neural networks for graph data. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pp. 2847–2856, 2018. doi: 10.1145/3219819.3220078.

Table 5. Summary of notations used in this paper.

Symbol	Description	Symbol	Description
Graph and Nodes		OT Margins and Cloak Templates	
$\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{X})$	Directed social graph with node set $\mathcal{V}$, edge set $\mathcal{E}$, and node features $\mathbf{X}$	$d_{\text{hum}}(v), d_H(v)$	Distance from bot v to nearest human in OT space
v	Generic node	$d_{\text{bot}}(v), d_B(v)$	Distance from bot v to nearest bot in OT space
v_t	Target bot node	$m(v)$	OT margin between human and bot regions for node v
ξ	Generic comparison/anchor node (e.g., template or human)	$\mathcal{B}_{\text{bdry}}$	Boundary bot set under OT geometry
$\mathcal{V}_{\text{bot}}, \mathcal{V}_{\text{hum}}$	Sets of bot and human nodes	τ_{bdry}	Margin threshold used to define boundary bots
$\mathcal{N}(v)$	k -hop ego neighborhood of node v	$\mathcal{B}_{\text{mis}}$	Bots misclassified as human by the detector
$N^{\text{out}}(v)$	Out-neighborhood (1-hop outgoing neighbors) of node v	$\mathcal{B}$	Candidate cloak template pool
u_i	The i -th neighbor (used in helper selection over $N^{\text{out}}(v_t)$)	Attack Objectives and Constraints	
Neighbor Features and Measures		$\Delta\mathcal{E}$	Set of edge edits incident to the target node
$\phi_v(\eta)$	Spatio-temporal feature vector of neighbor η relative to v	B	Edge budget for the attack
z_i	Shorthand for a neighbor feature vector (e.g., $z_i = \phi_v(u_i)$)	$\mathcal{F}(B)$	Feasible set of edge edits under budget and domain constraints
d	Dimensionality of neighbor feature vectors	$\Phi(\Delta\mathcal{E})$	Plausibility penalty for edge edits
$z_i \in \mathbb{R}^d$	Neighbor feature space and dimension	$\lambda_{\text{BCE}}, \lambda_{\text{pl}}, \lambda_{\text{sp}}$	BCE, plausibility and sparsity weights
$w_v(\eta)$	Normalized importance weight of neighbor η	Feature-Weighting Functions (Appendix)	
μ_v	Neighborhood probability measure of node v (weighted empirical distribution)	$\text{deg}_{\text{in}}(u), \text{deg}_{\text{out}}(u)$	In-/out-degree statistics for node u
a, b	Discrete marginal weight vectors for two neighborhoods (entries a_i, b_j)	$\text{deg}_{\text{raw}}(u)$	Raw degree used in weighting functions
m, n	Neighborhood sizes for μ_v and μ_w (numbers of atoms)	$\text{age_norm}(u)$	Normalized account age feature used in weighting
Optimal Transport Geometry (Entropic OT)		$g_{\text{deg}}(\cdot), g_{\text{time}}(\cdot)$	Degree-based and time-based importance functions
$h_{\theta}(\cdot)$	Neural embedding used to define the ground cost	$\alpha_{\text{deg}}, \alpha_{\text{time}}$	Hyperparameters controlling degree/time weighting strength
$c_{\theta}(z, z')$	Learned ground cost between neighbor feature vectors	Decoding, Construction, and Helper Selection	
C_{ij}	Ground-cost matrix entry, typically $C_{ij} = c_{\theta}(z_i, z'_j)$	v_c	Selected cloak template bot
$\mathbf{M}$	Positive semidefinite matrix defining Mahalanobis distance	v_h	Nearest human anchor in OT space
$\mathbf{L}$	Learnable factor to parameterize $\mathbf{M}$ (e.g., $\mathbf{M} = \mathbf{L}^{\top} \mathbf{L}$)	$h^*(v)$	Nearest-human selector in OT space
$\text{OT}_{\varepsilon}(\mu_v, \mu_w)$	Entropic-regularized OT objective between neighborhood measures	P_{ch}^*	OT plan between template bot and nearest human
$\mathcal{U}(a, b)$	Transportation polytope: couplings with marginals a and b	$\mu_t(\Delta\mathcal{E})$	Neighborhood measure of v_t after edits $\Delta\mathcal{E}$
P_{vw}^*	Entropic optimal transport plan between μ_v and μ_w	ρ_i	Row mass in P^* used for guided neighbor selection
P_{ij}	(i, j) entry of a coupling/plan matrix P	ρ	Vector of row masses $(\rho_i)_i$
$D_{\theta}(v, w)$	OT-based distance between nodes v and w	$\mathcal{U}$	Selected neighbor subset returned by helper routine
ε	Entropic regularization parameter	$\text{TopK}(\rho, K)$	Operator returning indices of the largest K entries of ρ
K_{ij}	Gibbs kernel entry: $K_{ij} = \exp(-C_{ij}/\varepsilon)$	K	Top- K selection size
u, v	Sinkhorn scaling vectors; $P \approx \text{diag}(u) K \text{diag}(v)$	Classifier, Training Pairs, and Surrogates	
$u^{(t)}, v^{(t)}$	Sinkhorn iterates at iteration t	f_{Θ}	Trained node classifier/detector (parameters Θ)
T_{sink}	Number of Sinkhorn iterations	$p_{\Theta}(y v; \mathcal{G})$	Predicted class probability for label y at node v in graph $\mathcal{G}$
$\odot$	Element-wise (Hadamard) product	y_v	Ground-truth node label (e.g., bot/human)
$\text{diag}(\cdot)$	Diagonal matrix formed from a vector	Π^+, Π^-	Positive and negative (different-class) training pair sets
$\mathbf{1}_k$	All-ones vector of length k	γ	Margin hyperparameter used in OT-geometry training loss
$\text{KL}(P K)$	KL divergence used in the entropic OT reformulation	$\sigma(\cdot)$	Logistic sigmoid function
		τ_{BCE}	Temperature hyperparameter to regulate sigmoid function
		$\Delta_{e, \text{subg}}, \Delta$	Edge budget, max number of modified edges by BOcloak

A. Appendix

B. Summary of Notations Used in This Paper

Table 5 summarizes the key notations used throughout the paper, covering graph structure, neighborhood representations, optimal transport quantities, and attack-specific objects.

C. Optimal Transport BOCLOAK-specific Extended Discussion

In this section, we present a detailed discussion of the Optimal Transport-based components of BOCLOAK, including formulation, and algorithmic steps.

C.1. Step 1: Spatio-Temporal Feature Extraction

For each node v , let $\mathcal{N}(v)$ denote its 1-hop neighborhood (all in- or out-neighbors that share at least one directed edge with v). For every neighbor $u \in \mathcal{N}(v)$, we build a structural and temporal feature vector $\phi_v(u) \in \mathbb{R}^d$, that captures node type, relational properties with respect to v , degree information, and temporal behavior.

Concretely, we organize $\phi_v(u)$ into four feature types: **static**, **behavioral**, **content**, and **temporal**:

- **Static features** $\mathbf{X}_{\text{stat}}(u)$ (intrinsic, time-invariant or slowly varying): $t(u) \in \{0, 1\}$ (human/bot indicator) and degree statistics $\deg_{\text{in}}(u), \deg_{\text{out}}(u)$ (raw and/or normalized).
- **Behavioral features** $\mathbf{X}_{\text{beh}}(u; v)$ (how u interacts with v in the ego graph): the directed edge role w.r.t. v , $r_v(u) \in \{0, 1, 2\} = \begin{cases} 1 & \text{if } u \rightarrow v \text{ (follower),} \\ 2 & \text{if } v \rightarrow u \text{ (followee),} \\ 0 & \text{if } u \leftrightarrow v \text{ (mutual follow).} \end{cases}$
- **Content features** $\mathbf{X}_{\text{cont}}(u)$ (what the account posts or describes): a fixed-length content vector (e.g., username, user bio, text/profile embeddings and content-derived attributes), denoted $\text{cont_feat}(u)$.
- **Temporal features** $\mathbf{X}_{\text{temp}}(u; v)$ (when the account exists/acts): normalized account age $\text{age_norm}(u) \in [0, 1]$ and relative age $\text{age_diff_norm}(u; v) = \text{age_norm}(u) - \text{age_norm}(v)$, capturing temporal alignment between u and v .

We collect these into a single spatio-temporal neighbor vector

$$\phi_v(u) = [\underbrace{\mathbf{X}_{\text{stat}}(u)}_{\text{Static}}, \underbrace{\mathbf{X}_{\text{beh}}(u; v)}_{\text{Behavioral}}, \underbrace{\mathbf{X}_{\text{cont}}(u)}_{\text{Content}}, \underbrace{\mathbf{X}_{\text{temp}}(u; v)}_{\text{Temporal}}] \in \mathbb{R}^d, \quad (6)$$

where, in our implementation, $\mathbf{X}_{\text{stat}}(u) = [t(u), \deg_{\text{in}}(u), \deg_{\text{out}}(u)]$, $\mathbf{X}_{\text{beh}}(u; v) = [r_v(u)]$, $\mathbf{X}_{\text{cont}}(u) = [\text{cont_feat}(u)]$, and $\mathbf{X}_{\text{temp}}(u; v) = [\text{age_norm}(u), \text{age_diff_norm}(u; v)]$. In this spatio-temporal feature space, neighbors with similar labels, structural roles, degrees, and temporal behavior lie close to each other.

C.2. Step 2: Local Neighborhood Probability Measures

We represent the 1-hop neighborhood of v as an empirical probability measure on the feature space. Let δ_z denote a point mass (Dirac measure (Tao, 2011)) at location $z \in \mathbb{R}^d$, i.e., for any bounded measurable test function f : $\int f(x) d\delta_z(x) = f(z)$.

Non-uniform, importance-weighted neighborhoods. Some neighbors are more informative about bot vs. human behavior than others. We therefore use a non-uniform weighting scheme. Let $a_v(u) > 0$ denote an unnormalized importance score for neighbor u and define the normalized weights:

$$w_v(u) = \frac{a_v(u)}{\sum_{u' \in \mathcal{N}(v)} a_v(u')}. \quad (7)$$

The importance score combines degree-based and time-based priorities:

$$a_v(u) = g_{\text{deg}}(\deg_{\text{raw}}(u)) g_{\text{time}}(\text{age_norm}(u)), \quad (8)$$

where, $g_{\text{deg}}(\deg_{\text{raw}}(u)) = 1 + \alpha_{\text{deg}} \log(1 + \deg_{\text{raw}}(u))$, with $\alpha_{\text{deg}} \geq 0$, to give higher but saturating importance to high-degree neighbors; likewise, $g_{\text{time}}(\text{age_norm}(u)) = 1 + \alpha_{\text{time}} \text{age_norm}(u)$, with $\alpha_{\text{time}} \geq 0$ to emphasize long-lived accounts that have persisted over the observation window.

We then define the neighborhood probability measure:

$$\mu_v = \sum_{u \in \mathcal{N}(v)} w_v(u) \delta_{\phi_v(u)} \quad (9)$$

This yields neighborhood distributions that concentrate more mass on structurally central and temporally persistent neighbors.

C.3. Step 3: Feature-based Neighbor Cost Computation

A *ground cost* c_θ between neighbor features in order to compare neighborhood distributions. Given two neighbor feature vectors $z, z' \in \mathbb{R}^d$, we first map each through a neural embedding network $h_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^{d_{\text{emb}}}$ and then compute a *squared Mahalanobis distance* in the embedding space.

Specifically, we let $\tilde{z} = h_\theta(z)$ and $\tilde{z}' = h_\theta(z')$ and define ground cost c_θ

$$c_\theta(z, z') = (\tilde{z} - \tilde{z}')^\top \mathbf{M} (\tilde{z} - \tilde{z}'), \quad (10)$$

where $\mathbf{M} = \mathbf{L}^\top \mathbf{L} \succeq 0$ is a positive-semidefinite matrix parameterized via a learnable matrix $\mathbf{L}$. Since this is the Mahalanobis distance (Mahalanobis, 1936) applied in a learned embedding space. It is expressive enough to reweight and correlate features, while remaining smooth, differentiable, and computationally efficient for entropic OT and Sinkhorn solvers (Flamary et al., 2021; Peyré & Cuturi, 2019).

C.4. Step 4: Entropic OT Between Neighborhoods

Given two neighborhoods $\mu_v = \sum_{i=1}^m a_i \delta_{z_i}$ and $\mu_w = \sum_{j=1}^n b_j \delta_{z'_j}$, we assemble a cost matrix $C \in \mathbb{R}^{m \times n}$ with entries $C_{ij} = c_\theta(z_i, z'_j)$, where z_i and z'_j are the neighbor feature vectors from v and w , respectively, and $a_i, b_j \geq 0$ are their corresponding normalized importance weights ($\sum_i a_i = \sum_j b_j = 1$). The discrete OT problem seeks a transport plan $P \in \mathbb{R}_{\geq 0}^{m \times n}$ with given marginals: $\mathcal{U}(a, b) = \left\{ P \in \mathbb{R}_{\geq 0}^{m \times n} : P \mathbf{1}_n = a, P^\top \mathbf{1}_m = b \right\}$, where $\mathbf{1}_k$ denotes an all-ones vector of length k .

For $\varepsilon > 0$, the entropic-regularized OT problem is

$$\text{OT}_\varepsilon(\mu_v, \mu_w) = \min_{P \in \mathcal{U}(a, b)} \left[\sum_{i,j} P_{ij} C_{ij} + \varepsilon \sum_{i,j} P_{ij} (\log P_{ij} - 1) \right]$$

Let P_{vw}^* denote an optimal entropic transport plan, i.e.,

$$P_{vw}^* \in \arg \min_{P \in \mathcal{U}(a, b)} \left[\sum_{i,j} P_{ij} C_{ij} + \varepsilon \sum_{i,j} P_{ij} (\log P_{ij} - 1) \right]$$

The entropic regularizer, ε , encourages diffuse plans and yields a unique minimizer P^* ; moreover, after introducing the Gibbs kernel $K = \exp(-C/\varepsilon)$, the objective can be expressed (up to an additive constant) as a KL divergence, making ε a *temperature* that controls plan sharpness (small ε : near-permutation, large ε : diffuse). The ideal plan lies between a completely diffuse plan and a brittle near-permutation plan, stabilizing the OT geometry by providing smooth, well-conditioned transport maps that capture multiple plausible alignments among similar neighbors, rather than completely diffuse or a brittle, permutational matching plan.

As is standard in entropic OT, the objective can be rewritten as a Kullback-Leibler (KL) divergence between P and a Gibbs kernel constructed from the cost matrix C ; see Appendix D for a detailed derivation. In particular, introducing the Gibbs kernel $K = \exp(-C/\varepsilon)$, the optimal plan P^* can be written in a scaling form $P^* = \text{diag}(u) K \text{diag}(v)$ where the scaling vectors u and v are obtained via Sinkhorn iterations that enforce the marginal constraints $P^* \mathbf{1}_n = a$ and $(P^*)^\top \mathbf{1}_m = b$.

We define OT distance as the transport cost induced by the entropic plan:

$$D_\theta(v, w) \triangleq \langle P_{vw}^*, C_{vw} \rangle = \sum_{i,j} (P_{vw}^*)_{ij} (C_{vw})_{ij}. \quad (11)$$

Entropic regularization is used only to obtain a stable and unique plan P_{vw}^* via Sinkhorn.

C.5. Step 5: OT-Margin Between Human And Bot

Let $\mathcal{B}$ denote the set of bot accounts and $\mathcal{H}$ the set of human accounts. Using the OT distance $D_\theta(\cdot, \cdot)$ from Equation 11, for each bot $v \in \mathcal{B}$, the minimum distance to any human account is $d_H(v) = \min_{h \in \mathcal{H}} D_\theta(v, h)$, while the minimum distance to another bot account is $d_B(v) = \min_{b \in \mathcal{B} \setminus \{v\}} D_\theta(v, b)$. The OT-based margin is then given by $m(v) = d_H(v) - d_B(v)$.

This OT-based margin ensures:

- if $m(v)$ is large and positive, then v is much closer (in OT space) to other bots than to any human, indicating a robust bot-like neighborhood;
- if $m(v)$ is small or negative, v lies close to (or inside) the “human region” of OT space, suggesting that small structural perturbations could push v across the classifier’s decision boundary.

We also introduce a margin threshold τ_{bdry} and label a bot v as a boundary bot when $m(v) \leq \tau_{\text{bdry}}$, meaning its neighborhood is closer to the human manifold than to the bot manifold under D_θ .

This OT-based margin with margin threshold, τ_{bdry} ensures:

- if $m(v) > \tau_{\text{bdry}}$, then v is well inside the bot manifold (robustly bot-like), since it is closer to bots than to humans by a comfortable margin;
- if $m(v) \leq \tau_{\text{bdry}}$, then v is a boundary bot whose neighborhood is comparatively human-close in OT space, so small structural perturbations may push it across the decision boundary (especially when $m(v) \leq 0$).

Adversarial Interpretation. Viewing OT space as the *attack space*, the adversary seeks a neighborhood distribution μ'_v for the target bot such that the classifier labels it as human:

$$\min_{\mu'_v: f(\mu'_v)=\text{human}} D_\theta(\mu_v, \mu'_v). \quad (12)$$

The optimal value of Equation 12 reflects the robustness of the bot detector around v : a large minimal perturbation implies that the current neighborhood is robust to realistic edits, while a small one indicates vulnerability.

In BOCLOAK, we do not optimize Equation 12 directly. Instead, we use existing bots as *cloak* templates for μ'_v . Choosing a mimic bot with small $d_H(\cdot)$ or negative margin $m(\cdot)$ yields a blueprint for the *minimal* structural changes (edge edits) needed to make the target bot’s neighborhood distribution human-like.

Under this formulation, misclassified bots naturally reveal vulnerable regions of the decision boundary in OT space, and the OT distance quantifies how far a correctly classified bot lies from such regions. Bots with small positive margin are our candidate *boundary bots*, whereas bots with negative or near-zero margin are already evasive.

C.6. Step 6: OT Geometry Training Objective

The OT geometry (parameters θ and $\mathbf{M} = \mathbf{L}^\top \mathbf{L}$) is trained offline using a multi-task optimization loss (refer to Equation 20 and Algorithm 1). We use three complementary loss components accounting for margin, sparsity and plausibility.

Binary Cross-Entropy (BCE) Loss via OT-margin surrogate. Rather than differentiating through discrete edge edits and the classifier, we directly shape the OT margin. For each bot v , let $y_v \in \{0, 1\}$ denote a label that encodes whether v is currently misclassified ($y_v = 1$) or correctly classified ($y_v = 0$). We apply a logistic surrogate to the margin:

$$L_{\text{BCE}}(v; \theta) = \text{BCE}\left(\sigma(-m(v)/\tau_{\text{BCE}}), y_v\right), \quad (13)$$

where σ is the logistic sigmoid, $\tau_{\text{BCE}} > 0$ is a sigmoid temperature hyperparameter, and BCE is the standard binary cross-entropy. This encourages misclassified bots to have a small or negative margin and correctly classified bots to have a large positive margin, thereby sharpening the decision boundary in OT space.

Sparsity loss via OT plan entropy. To encourage attack sparsity (few edge changes for the target bot), we regularize the entropy of the OT plan between a bot v and its nearest human $h^*(v)$. For each bot v , the nearest human is:

$$h^*(v) = \arg \min_{h \in \mathcal{H}} D_\theta(v, h), \quad (14)$$

where $D_\theta(v, h)$ is the OT distance between their neighborhood measures.

Let $P_{vh}^* \in \mathbb{R}_{\geq 0}^{m \times n}$ be the Sinkhorn plan between μ_v and $\mu_{h^*(v)}$, where rows $i = 1, \dots, m$ index neighbors $u_i \in \mathcal{N}(v)$ of the bot v and columns $j = 1, \dots, n$ index neighbors $u'_j \in \mathcal{N}(h^*(v))$ of its closest human. Thus P_{ij} denotes how much probability mass from bot-side neighbor u_i is transported to human-side neighbor u'_j .

We define row-wise and column-wise conditional entropies that measure how concentrated the transport mass is within each row/column of P (i.e., whether each neighbor aligns to only a few counterparts versus spreading its mass broadly), where $p_{j|i} = P_{ij}/a_i$ denotes the row-conditional and $q_{i|j} = P_{ij}/b_j$ denotes the column-conditional distribution:

$$\begin{aligned} H_{\text{row}}(P) &= \sum_{i=1}^m a_i \left(- \sum_{j=1}^n p_{j|i} \log p_{j|i} \right) = - \sum_{i=1}^m \sum_{j=1}^n P_{ij} \log \frac{P_{ij}}{a_i}, \\ H_{\text{col}}(P) &= \sum_{j=1}^n b_j \left(- \sum_{i=1}^m q_{i|j} \log q_{i|j} \right) = - \sum_{i=1}^m \sum_{j=1}^n P_{ij} \log \frac{P_{ij}}{b_j}, \end{aligned}$$

Here, $p_{j|i} = P_{ij}/a_i$ describes how the mass of bot-side neighbor u_i is distributed across human-side neighbors, and $q_{i|j} = P_{ij}/b_j$ describes how the mass received by human-side neighbor u'_j is distributed across bot-side neighbors. When $H_{\text{row}}(P)$ is small, each bot-side neighbor concentrates its transport on only a few human-side neighbors; when $H_{\text{col}}(P)$ is small, each human-side neighbor receives mass from only a few bot-side neighbors.

The sparsity loss for bot v as the sum of these two entropies:

$$L_{\text{sp}}(v; \theta) = H_{\text{row}}(P_{vh}^*) + H_{\text{col}}(P_{vh}^*). \quad (15)$$

By construction, $L_{\text{sp}}(v; \theta)$ is large when the transport mass in P_{vh}^* is distributed broadly across many bot-side and human-side neighbors, resulting in high row- and column-wise conditional entropies. Minimizing $L_{\text{sp}}(v; \theta)$ therefore encourages OT plans in which the transport concentrates on a small set of high-confidence template matches between bot-side neighbors and human-side neighbors (i.e., a localized, few-to-few alignment). In the context of BOCLOAK, these template matches can be interpreted as a small set of “bot templates” (human-like local patterns) that the target bot must mimic, which in turn corresponds to attack strategies requiring only a few edge edits around the target bot.

Derivation and Properties. Using Equation 15 and the marginal constraint $P\mathbf{1}_n = a$,

$$H_{\text{row}}(P) = - \sum_{i,j} P_{ij} \log P_{ij} + \sum_i a_i \log a_i. \quad (16)$$

Similarly, using $P^\top \mathbf{1}_m = b$,

$$H_{\text{col}}(P) = - \sum_{i,j} P_{ij} \log P_{ij} + \sum_j b_j \log b_j. \quad (17)$$

Therefore,

$$H_{\text{row}}(P) + H_{\text{col}}(P) = 2 \left(- \sum_{i,j} P_{ij} \log P_{ij} \right) + \sum_i a_i \log a_i + \sum_j b_j \log b_j, \quad (18)$$

i.e., L_{sp} is equivalent to minimizing the entropy of the transport plan and is thus non-constant in P .

Why this encourages sparsity? For each fixed row i , the Shannon entropy satisfies $H(p_{\cdot|i}) \geq 0$ with equality iff $p_{\cdot|i}$ is a point mass. Hence minimizing $H_{\text{row}}(P) = \sum_i a_i H(p_{\cdot|i})$ encourages each row to concentrate mass on few columns; analogously minimizing $H_{\text{col}}(P)$ encourages concentration within each column.

Moreover, the feasible set $\mathcal{U}(a, b) = \{P \geq 0 : P\mathbf{1}_n = a, P^\top \mathbf{1}_m = b\}$ is a convex polytope, and $H(P) = - \sum_{i,j} P_{ij} \log P_{ij}$ is concave in P . Therefore, minimizing $H(P)$ over $\mathcal{U}(a, b)$ attains a minimum at an extreme point of $\mathcal{U}(a, b)$, which is a feasible solution with at most $m + n - 1$ nonzero entries, i.e., a sparse transport plan.

Plausibility loss via degree and account-age alignment. We also regularize the OT plan to preserve social and temporal plausibility. Large numbers of high-degree, long-lived humans following very new, low-degree bots are unlikely in realistic networks.

We define a per-pair plausibility cost that penalizes degree and account-age mismatches between matched neighbors:

$$\phi_{\text{pl}}(i, j) = \left(\alpha_{\text{deg}} |\deg(i) - \deg(j)| + \alpha_{\text{age}} |\text{age}(i) - \text{age}(j)| \right)$$

with $\alpha_{\text{deg}}, \alpha_{\text{age}} \geq 0$, where $\deg(i)$ and $\deg(j)$ are the degrees of neighbors u_i and u'_j , and $\text{age}(\cdot)$ is their normalized account age. Larger α_{deg} (resp. α_{age}) increases the penalty for degree (resp. account-age) mismatches, biasing P_{vh}^* toward structurally and temporally aligned neighbor matches.

The plausibility loss aggregates these per-pair costs under the OT plan:

$$L_{\text{pl}}(v; \theta) = \sum_{i,j} P_{vh,ij}^* \phi_{\text{pl}}(i, j). \quad (19)$$

$L_{\text{pl}}(v; \theta)$ measures how much transport mass is placed on socially implausible matches where high-degree, long-lived humans are aligned with very new, low-degree bots (or vice versa). $\phi_{\text{pl}}(i, j)$ increases as the degree/age mismatch increases, and $L_{\text{pl}}(v; \theta)$ is large when the OT plan puts mass on implausible matches. Under this loss, neighbor pairs that both (i) receive substantial transport mass $P_{vh,ij}^*$ and (ii) exhibit large degree or age gaps contribute more strongly to the penalty, encouraging OT plans that align neighbors with similar degree and temporal profiles.

Multi-task Objective. Putting the pieces together, we train the OT geometry by minimizing

$$\mathcal{L}_{\text{BOCLOAK}}(\theta) = \mathbb{E}_{v \in \mathcal{V}_{\text{train}}} [\lambda_{\text{BCE}} L_{\text{BCE}}(v; \theta) + \lambda_{\text{sp}} L_{\text{sp}}(v; \theta) + \lambda_{\text{pl}} L_{\text{pl}}(v; \theta)] \quad (20)$$

with non-negative weights λ_{BCE} , λ_{sp} , and λ_{pl} .

We treat $(\varepsilon, \text{Top}K)$ as OT hyperparameters: ε is the entropic regularization used in Sinkhorn during both training and inference, while K is used only at attack time to select the Top K OT-salient neighbors in Algorithm 6. Algorithm 1 details the offline optimization procedure used to learn the OT geometry $(\theta, \mathbf{M})$.

C.7. Step 7: OT Margins and Boundary Bots in BOCLOAK

Once the OT geometry is trained (using Algorithm 1), we precompute $D_\theta(v, w)$ for bot-human pairs, obtain margins $m(v)$ for all bots, and identify boundary bots ($\mathcal{B}_{\text{bdry}}$) and misclassified bots ($\mathcal{B}_{\text{mis}}$), as described in §C.5. Training learns a neural ground cost c_θ (with Mahalanobis metric $\mathbf{M}$) by differentiating through a Sinkhorn OT solver with entropic regularization ε . For each training bot, the procedure mines its nearest human and nearest-bot neighbors under the current OT geometry, and forms an OT margin $m(v) = d_H(v) - d_B(v)$. It then updates $(\theta, \mathbf{M})$ to shape this margin via the margin threshold (τ_{bdry}) and sigmoid temperature (τ_{BCE}) while regularizing the transport plan with sparsity and plausibility losses. Given the learned OT boundary, Algorithm 2 performs OT-guided cloaking that pushes correctly classified bots across the decision boundary, causing them to be misclassified as human and thus evading the bot detector.

C.8. Step 8: OT-Guided BOCLOAK Attack

Algorithm 2 summarizes the BOCLOAK OT-guided pipeline for bot editing attacks (with helper routines detailed in §E.1, §E.2, §E.3, §E.4 and Algorithm 7). For a given target bot v_{tar} , once OT margins and boundary bots are precomputed, BOCLOAK selects a small set of OT bot cloak templates $b \in \mathcal{B}_{\text{bdry}} \cap \mathcal{B}_{\text{mis}}$, as described in §E.1. Rather than choosing templates uniformly, BOCLOAK samples each template bot b from an importance-weighted distribution $p(b)$ that favors (i) structurally cheaper templates (fewer required edges) and (ii) templates closer to the OT boundary (higher boundary priority). To avoid overusing a single blueprint, we enforce a reuse cap: each bot template can be selected at most three times while any template remains under the cap; once all templates reach this cap, we reset the counters and continue sampling under the same policy. This captures a practical constraint: if a successful template bot is reused too many times and later detected, then its cloaked bots may be flagged as well. Therefore, we cap template reuse to reduce correlated detection risk across generated bots.

Before applying any edge edits, we initialize the non-temporal node features of v_{tar} to match those of the current template bot b , while keeping temporal features such as account age consistent with a newly created account. For each template b , we then retrieve the OT plan between b and its closest human, interpret high-mass entries in the plan as *critical neighbor pairs*,

Algorithm 1 TRAINOTGEOMETRY($G, \mathcal{V}_{\text{train}}, \mathcal{V}, \mathcal{V}', \varepsilon, T_{\text{sink}}, \tau_{\text{BCE}}, \tau_{\text{bdry}}, \lambda_{\text{BCE}}, \lambda_{\text{sp}}, \lambda_{\text{pl}}$)

```

1: Input: Graph  $G = (\mathcal{V}, \mathcal{E})$ ; ground-truth labels  $\mathcal{V} = \mathcal{H} \cup \mathcal{B}$ ; predicted labels  $\mathcal{V} = \mathcal{H}' \cup \mathcal{B}'$ ; training node set  $\mathcal{V}_{\text{train}} \subseteq \mathcal{V}$ 
   (typically bots); entropic OT parameter  $\varepsilon > 0$ ; Sigmoid temperature  $\tau_{\text{BCE}} > 0$ ; Boundary margin  $\tau_{\text{bdry}} > 0$ ; Sinkhorn
   iterations  $T_{\text{sink}}$ ; loss weights  $\lambda_{\text{BCE}}, \lambda_{\text{sp}}, \lambda_{\text{pl}} \geq 0$ .
2: Output: trained OT geometry  $(\theta, \mathbf{M})$  where  $\mathbf{M} = \mathbf{L}^\top \mathbf{L} \succeq 0$ .
3: Initialize neural network parameters  $\theta$  and matrix factor  $\mathbf{L}$  (so  $\mathbf{M} = \mathbf{L}^\top \mathbf{L}$ ).
4: Precompute neighbor features  $\phi_v(u)$  and importance weights  $w_v(u)$  for all  $v \in \mathcal{V}_{\text{train}}$ .
5: for each training epoch do
6:   for each minibatch  $\mathcal{S} \subseteq \mathcal{V}_{\text{train}} \cap \mathcal{B}$  do
7:      $L \leftarrow 0$ 
8:     for each bot  $v \in \mathcal{S}$  do
9:       Build weighted neighborhood measure  $\mu_v = \sum_{u \in \mathcal{N}(v)} w_v(u) \delta_{\phi_v(u)}$  // Equation 9
10:      Set misclassification label  $y_v \leftarrow \mathbb{1}[v \in \mathcal{H}']$  // true bot predicted human
11:      Find nearest human:  $h^*(v) \leftarrow \arg \min_{h \in \mathcal{H}} D_\theta(v, h)$  // Equation 11
12:      Find nearest other bot:  $b^*(v) \leftarrow \arg \min_{b \in \mathcal{B} \setminus \{v\}} D_\theta(v, b)$ 
13:      Compute margin:  $m(v) \leftarrow D_\theta(v, h^*(v)) - D_\theta(v, b^*(v))$  // §C.5
14:      Compute  $L_{\text{BCE}}(v; \theta)$  from margin and  $\tau_{\text{bdry}}$  and  $\tau_{\text{BCE}}$  // Equation 13
15:      Compute entropic OT plan  $P_{v, h^*(v)}^*$  via Sinkhorn under  $(\theta, \mathbf{M}, \varepsilon)$ 
16:      Compute sparsity loss  $L_{\text{sp}}(v; \theta)$  from plan entropies  $P_{v, h^*(v)}^*$  // Equation 15
17:      Compute plausibility loss  $L_{\text{pl}}(v; \theta)$  from plan entropies  $P_{v, h^*(v)}^*$  // Equation 19
        for  $T_{\text{sink}}$  iterations // §C.4, Equation 29–Equation 30
18:      Accumulate:  $L \leftarrow L + \lambda_{\text{BCE}} L_{\text{BCE}}(v; \theta) + \lambda_{\text{sp}} L_{\text{sp}}(v; \theta) + \lambda_{\text{pl}} L_{\text{pl}}(v; \theta)$ 
19:    end for
20:    Gradient step on  $(\theta, \mathbf{L})$  using  $L$  // backprop through Sinkhorn (§C.4).
21:  end for
22: end for
23: return  $(\theta, \mathbf{M} = \mathbf{L}^\top \mathbf{L})$ .
    
```

and translate those pairs into concrete edge edits between v_{tar} and the neighbors of b , subject to a strict edge budget and plausibility constraints. In other words, the OT plan tells us *which edges matter most* for making b appear human-like, and BOCLOAK reuses this blueprint for v_{tar} to construct sparse, realistic neighborhoods that cross the decision boundary.

In summary, OT provides two key signals for BOCLOAK: (i) OT margins identify boundary bots whose neighborhoods are already close to the human manifold and hence are promising candidate cloaks; and (ii) the OT plan between a template and its nearest human identifies which neighbors are most critical to clone, enabling sparse, plausible edge edits that still effectively push new or existing correctly classified bots across the decision boundary.

Special Case: Perfect Bot-Detector. In rare cases, the underlying bot detector may be “perfect” on the current graph, so that no bots are misclassified, and the OT margin test fails to identify any boundary or misclassified templates, i.e., $\mathcal{B}_{\text{bdry}} \cup \mathcal{B}_{\text{mis}} = \emptyset$. In this scenario, BOCLOAK falls back to a purely human behavior-guided strategy that still exploits the learned OT geometry. For each bot v_{tar} in an allowed structural category, we first identify its most human-like account: $h^*(v_{\text{tar}}) = \arg \min_{h \in \mathcal{H}} D_\theta(v_{\text{tar}}, h)$, and treat $h^*(v_{\text{tar}})$ as a *human template*. We then instantiate a new synthetic bot node v_{new} for node injection or update the target node for node editing by perturbing the non-temporal features and mimicking follow edges of $h^*(v_{\text{tar}})$, subject to the same budget and plausibility constraints.

Human templates whose cloned neighborhood would exceed the budget are skipped. This human-template fallback creates highly human-like synthetic bots whose neighborhoods lie close to the human manifold in OT space, and can still yield evading examples even when no misclassified or boundary bot templates are available. This method greedily pulls the neighborhood mass of v_{tar} toward the human manifold in OT space using only edge additions that progressively shrinking $D_\theta(v_{\text{tar}}, h)$ eventually pushes the target across the decision boundary and yields an evading bot even when no misclassified bot cloak templates are available.

Algorithm 2 BOCLOAK ($G, \mathcal{V}, \mathcal{V}', \theta, M, \varepsilon, TopK, \Delta, R, \text{flag_hb}, \tau_{\text{bdry}}, \text{degree}_{\text{node}}, T$)

```

1: Input: Graph  $G = (\mathcal{V}, \mathcal{E})$ ; ground-truth labels  $\mathcal{V} = \mathcal{H} \cup \mathcal{B}$ ; predictions labels  $\mathcal{V}' = \mathcal{H}' \cup \mathcal{B}'$ ; trained OT geometry
   ( $\theta, M$ ) (obtained via Algorithm 1), target node  $v_{\text{tar}}$  OT params  $(\varepsilon, TopK)$ ; budget  $\Delta$ ; max reuse bot  $R$ ; flag  $\text{flag\_hb}$ 
   to not allow human follow back; boundary threshold  $\tau_{\text{bdry}}$ ; max cloak bot node degree  $\text{degree}_{\text{node}}$ ; and number of
   trials  $T$ .
2: Output: attack statistics and successful bot cloaks.
3:  $\mathcal{B}_{\text{candidate}} \leftarrow \text{GETBOUNDARYBOTCANDIDATES}(G, \mathcal{V}, \mathcal{V}', \theta, M, \tau_{\text{bdry}}, \text{degree}_{\text{node}})$  // using Algorithm 3
4:  $w_{\text{candidate}} \leftarrow \text{GETIMPORTANCEWIGHTS}(\mathcal{B}_{\text{candidate}})$  // using Algorithm 4
5: Initialize template use-counts  $u(t) \leftarrow 0 : \forall t \in \mathcal{B}_{\text{candidate}}$ 
6: Initialize successful misclassified bot cloaks  $B_{\text{successful}} \leftarrow \emptyset$ 
7: for  $\_ \leftarrow 1$  to  $T$  do
8:    $c \leftarrow \text{SAMPLECLOAK}(\mathcal{B}_{\text{candidate}}, w_{\text{candidate}}, u, R)$  // using Algorithm 5
9:    $\mathcal{U} \leftarrow \text{OTGUIDEDNEIGHBORS}(\theta, M, \varepsilon, TopK, G, c)$  // using Algorithm 6
10:   $G', E_{\text{add}}, v_{\text{new}} \leftarrow \text{CLONECLOAK}(G, v_{\text{tar}}, c, \mathcal{U}, \text{flag\_hb})$  // using Algorithm 7
11:  if  $|E_{\text{add}}| > \Delta$  then
12:    record budget exceeded
13:    continue
14:  end if
15:  if  $v_{\text{new}} \neq \text{Bot}$  then
16:    record success,  $c, E_{\text{add}}$ , and  $v_{\text{new}}$ 
17:     $B_{\text{successful}} \leftarrow B_{\text{successful}} \cup \{[c, E_{\text{add}}, v_{\text{new}}]\}$ 
18:     $u(t) \leftarrow u(t) + 1$ 
19:  end if
20: end for
21: return  $B_{\text{successful}}$ .
    
```

D. Entropic Optimal Transport, Sinkhorn Scaling, and Complexity

This section presents the mathematical derivation of entropic Optimal Transport in BOCLOAK, the associated Sinkhorn scaling method, and its computational complexity.

D.1. Entropic OT objective and KL form

Recall the entropic OT problem in §C.4:

$$\text{OT}_{\varepsilon}(\mu_v, \mu_w) = \min_{P \in \mathcal{U}(a, b)} \left[\sum_{i,j} P_{ij} C_{ij} + \varepsilon \sum_{i,j} P_{ij} (\log P_{ij} - 1) \right], \quad (21)$$

where $\mathcal{U}(a, b)$ is the transportation polytope $P \mathbf{1}_n = a, P^{\top} \mathbf{1}_m = b, P_{ij} \geq 0$. We introduce the Gibbs kernel

$$K_{ij} = \exp(-C_{ij}/\varepsilon). \quad (22)$$

Using the identity

$$\sum_{i,j} P_{ij} C_{ij} + \varepsilon \sum_{i,j} P_{ij} (\log P_{ij} - 1) = \varepsilon \sum_{i,j} P_{ij} \log \frac{P_{ij}}{\tilde{K}_{ij}} + \text{const}, \quad (23)$$

where $\tilde{K}_{ij} = \exp(-C_{ij}/\varepsilon)$ and the constant does not depend on P , we can rewrite [Equation 21](#) as

$$\text{OT}_{\varepsilon}(\mu_v, \mu_w) = \varepsilon \min_{P \in \mathcal{U}(a, b)} \text{KL}(P \parallel K) + \text{const}, \quad (24)$$

where $\text{KL}(P \parallel K) = \sum_{i,j} P_{ij} \log(P_{ij}/K_{ij})$. Thus entropic OT selects, among all matrices with the desired marginals, the plan P that is closest (in KL divergence) to the Gibbs kernel K .

Previously, we defined the OT distance between nodes v and w as the transport term under the entropic optimal plan:

$$D_\theta(v, w) \triangleq \langle P_{vw}^*, C_{vw} \rangle.$$

The full entropic objective value in Equation 21 satisfies, for $\varepsilon > 0$,

$$\text{OT}_\varepsilon(\mu_v, \mu_w) = D_\theta(v, w) + \varepsilon \sum_{i,j} (P_{vw}^*)_{ij} (\log(P_{vw}^*)_{ij} - 1),$$

so $\text{OT}_\varepsilon(\mu_v, \mu_w)$ and $D_\theta(v, w)$ are generally not equal except when $\varepsilon = 0$ or when the entropic regularizer evaluates to zero at P_{vw}^* .

D.2. Lagrangian and scaling form

The constrained problem Equation 21 admits a dual. Consider the Lagrangian

$$\mathcal{L}(P, \alpha, \beta) = \sum_{i,j} P_{ij} C_{ij} + \varepsilon \sum_{i,j} P_{ij} (\log P_{ij} - 1) + \sum_i \alpha_i \left(a_i - \sum_j P_{ij} \right) + \sum_j \beta_j \left(b_j - \sum_i P_{ij} \right) \quad (25)$$

where $\alpha \in \mathbb{R}^m$, $\beta \in \mathbb{R}^n$ are dual variables for the marginal constraints. Differentiating w.r.t. P_{ij} and setting the derivative to zero yields

$$C_{ij} + \varepsilon (\log P_{ij}) - \alpha_i - \beta_j = 0,$$

so that

$$P_{ij} = \exp\left(\frac{\alpha_i}{\varepsilon}\right) \exp\left(-\frac{C_{ij}}{\varepsilon}\right) \exp\left(\frac{\beta_j}{\varepsilon}\right) = u_i K_{ij} v_j, \quad (26)$$

where we define the scaling vectors $u_i = \exp(\alpha_i/\varepsilon)$ and $v_j = \exp(\beta_j/\varepsilon)$. In matrix form,

$$P^* = \text{diag}(u) K \text{diag}(v). \quad (27)$$

The marginal constraints $P^* \mathbf{1}_n = a$ and $(P^*)^\top \mathbf{1}_m = b$ translate to

$$u \odot (Kv) = a, \quad v \odot (K^\top u) = b, \quad (28)$$

where $\odot$ denotes element-wise multiplication.

D.3. Sinkhorn iterations

The Sinkhorn algorithm (Cuturi, 2013) iteratively rescales u and v to satisfy Equation 28. Starting from $v^{(0)} = \mathbf{1}_n$, a typical scheme is:

$$u^{(t+1)} = a \oslash (Kv^{(t)}), \quad (29)$$

$$v^{(t+1)} = b \oslash (K^\top u^{(t+1)}), \quad (30)$$

where $\oslash$ is element-wise division. After T_{sink} iterations, we obtain approximate scalings $u^{(T_{\text{sink}})}$ and $v^{(T_{\text{sink}})}$ and hence an approximate optimal plan, $P^{(T_{\text{sink}})}$,

$$P^{(T_{\text{sink}})} = \text{diag}(u^{(T_{\text{sink}})}) K \text{diag}(v^{(T_{\text{sink}})}).$$

Under mild conditions, the iterates converge geometrically and the resulting plan satisfies the marginal constraints up to a small numerical tolerance.

D.4. Computational and Time Complexity

Let $m = |\mathcal{N}(v)|$ and $n = |\mathcal{N}(w)|$ denote the neighborhood sizes of μ_v and μ_w , respectively. Constructing the cost matrix $C \in \mathbb{R}^{m \times n}$ with entries $C_{ij} = c_\theta(z_i, z'_j)$ and the Gibbs kernel $K = \exp(-C/\varepsilon)$ requires $O(mn)$ ground-cost evaluations (§C.3) and exponentials, and stores $O(mn)$ numbers.

Each Sinkhorn iteration updates the scaling vectors via the matrix–vector products Kv and $K^\top u$, each costing $O(mn)$ operations. After T_{sink} iterations, one entropic OT solve between μ_v and μ_w therefore costs

$$\text{time} = O(T_{\text{sink}}mn), \quad \text{space} = O(mn),$$

where the space accounts for storing C and K (and an additional $O(m+n)$ for the scaling vectors). Consequently, the OT computation that dominates one iteration of BOCLOAK for finding one cloak is $O(T_{\text{sink}}mn)$ in time and $O(mn)$ in space complexity.

Computing $d_H(v)$ and $d_B(v)$ for all bots requires OT distances between each $v \in \mathcal{B}$ and all $h \in \mathcal{H}$, as well as all $b \in \mathcal{B} \setminus \{v\}$. If $|\mathcal{B}| = N_B$ and $|\mathcal{H}| = N_H$ and typical neighborhood sizes are m and n , then the number of Sinkhorn solves is $N_B N_H + N_B(N_B - 1)$, and the total time for a naïve full pairwise computation is $O(T_{\text{sink}}mn(N_B N_H + N_B^2))$ with per-solve memory $O(mn)$.

Caching. In the BOCLOAK setting, we must evaluate OT distances between many pairs of bots and humans when computing margins §C.5. To make this tractable, we (i) restrict OT computation to a candidate set of humans and bots selected via only calculating OT for bots with node degree less than or equal to budget provided (e.g., $\Delta = 1$), (ii) reuse the same kernel K across multiple solves whenever C depends only on distance in the learned embedding space, and (iii) cache $D_\theta(v, w)$ for frequently queried pairs. For higher-degree nodes, we can additionally truncate each empirical measure to its top-weighted neighbors before solving OT, and for larger graphs we restrict cloak search to the top- K boundary candidates. These approximations preserve the structure of the OT margin while keeping the precomputation stage within our computational budget.

E. Algorithm 2: Extended Discussion

Helper Functions. We define the helper routines invoked by Algorithm 2 (Node Editing Attack). We (i) construct a boundary-ranked cloak set via OT geometry (§E.1), (ii) derive importance sampling distributions (§E.2), (iii) sample cloaks under a reuse cap (§E.3), (iv) compute OT-guided neighbor restrictions (§E.4), including the nearest-human, and (v) clone the selected cloak (§E.5). Underlying OT definitions (neighborhood measures, ground cost, entropic OT, and OT-margin) are detailed in §C.3–§C.5 and Appendix D.

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a directed social graph. Ground-truth labels induce a partition $\mathcal{V} = \mathcal{H} \cup \mathcal{B}$ (humans/bots), predicted labels induce $\mathcal{V} = \mathcal{H}' \cup \mathcal{B}'$ and target node v_{tar} . The trained OT geometry is denoted by (θ, M) , and OT hyperparameters are (ε, K) where K is the neighbor budget used for OT-guided restriction.

E.1. GETBOUNDARYBOTCANDIDATES($G, \mathcal{V}, \mathcal{V}', \theta, M, \tau_{\text{bdry}}, \text{degree}_{\text{node}}$)

Algorithm 2 calls GETBOUNDARYBOTCANDIDATES Algorithm 3 to identify the candidate bot cloaks, which is the intersection set of (i) *boundary bots* (true bots that lie close to the human/bot decision boundary under the OT geometry) and (ii) *misclassified bots* (true bots predicted as human), while also computing the OT margin $m(b)$ for each $b \in \mathcal{B}$, so that the candidate bots can be sorted by margin. Algorithm 3 gives the exact procedure.

Inputs and outputs. It takes the socail graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$; the ground-truth label $\mathcal{V} = \mathcal{H} \cup \mathcal{B}$; the predicted label $\mathcal{V} = \mathcal{H}' \cup \mathcal{B}'$; the trained OT geometry (θ, M) (trained via Algorithm 1, and defines neighborhood measures and OT-based distances); thresholds τ_{bdry} (boundary proximity); and max boundary bot degree $\text{degree}_{\text{node}}$. It outputs the intersection set between the boundary candidate set and the misclassified-bot set, $\mathcal{B}_{\text{bdry}} \cap \mathcal{B}_{\text{mis}}$, sorted by margin $\{-m(b)\}_{b \in \mathcal{B}}$ in descending order.

Neighborhood measure and OT distances. For each bot $b \in \mathcal{B}$, we form a neighborhood measure μ_b (as in §4.1), using the OT geometry (θ, M) to embed and compare neighborhoods. We then compute two OT-geometry distances: (i) $d_H(b)$, the distance from b to the human side (e.g., to a reference human neighborhood/aggregate or nearest-human prototype under (θ, M)), and (ii) $d_B(b)$, the distance from b to the bot side (analogously defined within the bot class). These quantities are exactly those referenced in Algorithm 3. The routine iterates over true bots $b \in \mathcal{B}$ that have degree less than $\text{degree}_{\text{node}}$, builds the neighborhood measure μ_b (per §4.1), computes OT-geometry distances $d_H(b)$ and $d_B(b)$ under (θ, M) , and calculates the OT margin, $m(b) = d_H(b) - d_B(b)$ (refer to §C.5 for the formal definition and interpretation).

It then forms two sets: boundary bots $\mathcal{B}_{\text{bdry}} = \{b : 0 < m(b) \leq \tau_{\text{bdry}}\}$ and misclassified bots $\mathcal{B}_{\text{mis}} = \{b : b \notin \mathcal{B}'\}$. Finally, it returns the candidate bot cloaks, $\mathcal{T} = \mathcal{B}_{\text{bdry}} \cap \mathcal{B}_{\text{mis}}$, sorted by margin $-m(\cdot)$ in descending order (Algorithm 3) to rank candidate bots.

Algorithm 3 GETBOUNDARYBOTCANDIDATES($G, \mathcal{V}, \mathcal{V}', \theta, M, \tau_{\text{bdry}}, \text{degree}_{\text{node}}$)

```

1: Input: Graph  $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ ; ground-truth labels  $\mathcal{V} = \mathcal{H} \cup \mathcal{B}$  (humans and bots); predicted labels  $\mathcal{V} = \mathcal{H}' \cup \mathcal{B}'$  (predicted humans and bots); trained OT geometry  $(\theta, M)$  (trained via Algorithm 1); its boundary threshold  $\tau_{\text{bdry}}$  and max boundary bot node degree  $\text{degree}_{\text{node}}$ .
2: Output: boundary bot candidates  $\mathcal{B}_{\text{candidate}}$ .
3:  $\mathcal{B}_{\text{bdry}} \leftarrow \emptyset$ 
4:  $\mathcal{B}_{\text{mis}} \leftarrow \emptyset$ 
5: for each bot  $b \in \mathcal{B}$  do
6:   if GETNODEDEGREE( $b$ )  $\leq \text{degree}_{\text{node}}$  then
7:     build neighborhood measure  $\mu_b$ 
8:     compute  $d_H(b)$  and  $d_B(b)$ 
9:     set  $m(b) \leftarrow d_H(b) - d_B(b)$ 
10:    if  $m(b) \leq \tau_{\text{bdry}}$  then
11:       $\mathcal{B}_{\text{bdry}} \leftarrow \mathcal{B}_{\text{bdry}} \cup \{b\}$ 
12:    end if
13:    if  $b \notin \mathcal{B}'$  then
14:       $\mathcal{B}_{\text{mis}} \leftarrow \mathcal{B}_{\text{mis}} \cup \{b\}$ 
15:    end if
16:  end if
17: end for
18: return SORTED( $\mathcal{B}_{\text{bdry}} \cap \mathcal{B}_{\text{mis}}$ , key =  $-m(\cdot)$ , order = descending)
    
```

Algorithm 4 GETIMPORTANCEWEIGHTS($\mathcal{T}$)

```

1: Input: cloak records  $\mathcal{T}$  (containing rank( $t$ ) and access to category( $t$ ),  $in_h, in_b, out_h, out_b$ )
2: Output:  $p_{\text{category}}(\cdot)$  and  $\{p_{\text{cloak}}(\cdot | c)\}_c$ 
3: For each  $t \in \mathcal{T}$ , compute  $e(t)$  and  $\eta(t)$  // using Equation 31-Equation 33
4: Group cloaks into buckets  $\mathcal{T}_c$  by  $c = \text{category}(t)$ 
5: Compute  $p_{\text{category}}(\cdot)$  from // using Equation 34-Equation 39
6: For each category  $c$ , compute  $p_{\text{cloak}}(\cdot | c)$  from // using Equation 40-Equation 44
7: return ( $p_{\text{category}}(\cdot)$  and  $\{p_{\text{cloak}}(\cdot | c)\}_c$ )
    
```

E.2. GETIMPORTANCEWEIGHTS($\mathcal{T}$)

Algorithm 2 calls GETIMPORTANCEWEIGHTS Algorithm 4 with candidate bot cloaks $\mathcal{B}_{\text{candidate}}$ to build a category distribution $p_{\text{category}}(\cdot)$ and per-category cloak distributions $p_{\text{cloak}}(\cdot | c)$.

Per-cloak structural quantities. For each cloak $t \in \mathcal{T}$, we can obtain directed neighbor-counts: $in_h(t)$, $in_b(t)$, $out_h(t)$, $out_b(t)$, and a node category label from neighbor signature (details refer to Table 24), $\text{category}(t) \in \mathcal{C}$ (e.g., follow_bot_followed_by_bot_and_human, follow_node_followed_by_bot). There can be sixteen categories, as bots and humans can follow and be followed by each other separately, together, or not at all.

We define the edge-cost proxy used for weighting:

$$\text{in_eff}(t) = in_h(t) + in_b(t), \quad (31)$$

$$e(t) = out_h(t) + out_b(t) + \text{in_eff}(t), \quad (32)$$

$$\eta(t) = \mathbb{1}[in_h(t) > 0]. \quad (33)$$

Category-level weights. Let $\mathcal{T}_c = \{t \in \mathcal{T} : \text{category}(t) = c\}$ and $M = |\mathcal{T}|$.

Define per-category averages:

$$\bar{e}_c = \frac{1}{|\mathcal{T}_c|} \sum_{t \in \mathcal{T}_c} e(t), \quad (34)$$

$$\bar{r}_c = \frac{1}{|\mathcal{T}_c|} \sum_{t \in \mathcal{T}_c} \text{rank}(t). \quad (35)$$

Let $e_{\min} = \min_{c: |\mathcal{T}_c| > 0} \bar{e}_c$.

Define two factors and the category weight:

$$w_{\text{edges}}(c) = \left(\frac{e_{\min}}{\max(\bar{e}_c, 10^{-6})} \right)^2, \quad (36)$$

$$w_{\text{rank}}(c) = \frac{1}{1 + \bar{r}_c/M}, \quad (37)$$

$$w_{\text{category}}(c) = w_{\text{edges}}(c) w_{\text{rank}}(c). \quad (38)$$

Normalize to obtain:

$$p_{\text{category}}(c) = \frac{w_{\text{category}}(c)}{\sum_{c'} w_{\text{category}}(c')}. \quad (39)$$

Cloak-level weights within a category. Let $r_{\max} = \max_{t \in \mathcal{T}} \text{rank}(t)$ with $r_{\max} \geq 1$. Define:

$$w_{\text{edges}}(t) = \frac{1}{1 + e(t)}, \quad (40)$$

$$w_{\text{rank}}(t) = \frac{1}{1 + \text{rank}(t)/r_{\max}}, \quad (41)$$

$$w_{\text{human}}(t) = \begin{cases} 0.5, & \eta(t) = 1, \\ 1, & \eta(t) = 0, \end{cases} \quad (42)$$

and combine:

$$w_{\text{cloak}}(t) = w_{\text{edges}}(t) w_{\text{rank}}(t) w_{\text{human}}(t). \quad (43)$$

Normalize within category c :

$$p_{\text{cloak}}(t | c) = \frac{w_{\text{cloak}}(t)}{\sum_{t' \in \mathcal{T}_c} w_{\text{cloak}}(t')}. \quad (44)$$

E.3. SAMPLECLOAK($\mathcal{B}_{\text{candidate}}, w_{\text{candidate}}, u, R$)

Algorithm 2 calls SAMPLECLOAK **Algorithm 5** to sample a cloak while enforcing a reuse cap R whenever possible. Here $u(t)$ counts cloak uses (e.g., successful uses).

Cap enforcement. Let $\mathcal{T}_c$ be the cloaks in category c and

$$\mathcal{T}_c^{(<R)} = \{t \in \mathcal{T}_c : u(t) < R \wedge t \in \mathcal{B}_{\text{bdry}}\}. \quad (45)$$

If any category has $\mathcal{T}_c^{(<R)} \neq \emptyset$, we sample only among under-cap cloaks; otherwise we fall back to the original distributions.

E.4. OTGUIDEDNEIGHBORS($\theta, M, \varepsilon, \text{TopK}, G, t$)

Algorithm 2 calls OTGUIDEDNEIGHBORS **Algorithm 6** to restrict which *outgoing* neighbors of cloak t are cloned. In the updated design, this helper also computes the nearest human reference.

Nearest-human reference. Let $d_{\theta, M}(\cdot, \cdot)$ be the OT-geometry distance (or embedding-induced distance) between nodes. The nearest human to cloak t is

$$h^*(t) = \arg \min_{h \in \mathcal{H}} d_{\theta, M}(t, h). \quad (46)$$

Algorithm 5 SAMPLECLOAK($\mathcal{B}_{\text{candidate}}, w_{\text{candidate}}, u, R$)

```

1: Input: Boundary bots  $\mathcal{B}_{\text{candidate}}$ ;  $w_{\text{candidate}} = (\text{category distribution } p_{\text{category}}; \text{conditional cloak distributions } p_{\text{cloak}}(\cdot | c))$ , use-counts  $u(\cdot)$  and reuse cap  $R$ .
2: Output: sampled cloak  $t$ 
3: For each category  $c$ , form  $\mathcal{T}_c^{(<R)}$  using  $\mathcal{B}_{\text{candidate}}$  // using Equation 45
4: Compute  $\mathcal{B}_{\text{candidate}}$  mass  $\alpha(c) = \sum_{t \in \mathcal{T}_c^{(<R)}} p_{\text{cloak}}(t | c)$ 
5: if  $\sum_c p_{\text{category}}(c)\alpha(c) > 0$  then
6:   Sample  $c$  with probability proportional to  $p_{\text{category}}(c)\alpha(c)$ 
7:   Sample  $t$  from  $p_{\text{cloak}}(\cdot | c)$  restricted to  $\mathcal{T}_c^{(<R)}$ 
8: else
9:   Sample  $c \sim p_{\text{category}}$  and then  $t \sim p_{\text{cloak}}(\cdot | c)$ 
10: end if
11: return  $t$ 
    
```

Algorithm 6 OTGUIDEDNEIGHBORS($\theta, M, \varepsilon, \text{TopK}, G, t$)

```

1: Input: OT geometry  $(\theta, M)$ ; OT params  $(\varepsilon, \text{TopK})$ ; graph  $G$  and cloak  $t$ 
2: Output: restriction set  $\mathcal{U}$  (or  $\emptyset$  meaning no restriction)
3: if  $\varepsilon \leq 0$  or  $K \leq 0$  then
4:   return  $\emptyset$ 
5: end if
6: Compute nearest human  $h^*(t)$  via // using Equation 46
7: Compute entropic OT plan  $P_{t, h^*(t)}^*$  under  $(\theta, M, \varepsilon)$  between neighborhoods of  $t$  and  $h^*(t)$  // using §C.4
8: For each row  $i$  (neighbor  $u_i \in N^{\text{out}}(t)$ ), compute row mass  $\rho_i$  // using Equation 47
9: Let  $\mathcal{U} \leftarrow \{u_i : i \in \text{TopK}(\rho, K)\}$ 
10: return  $\mathcal{U}$ 
    
```

OT plan and row-mass scoring. Let $N^{\text{out}}(t)$ be the outgoing neighbors of t in G . When $\varepsilon > 0$, we compute the Sinkhorn plan between the neighborhood measures of t and $h^*(t)$ as described in the §C.4. Let rows of $P_{t, h^*(t)}^*$ correspond to neighbors $u_i \in N^{\text{out}}(t)$.

Define row masses:

$$\rho_i = \sum_j (P_{t, h^*(t)}^*)_{ij}. \quad (47)$$

Then select the OT-guided neighbor restriction set as, $\mathcal{U} = \{u_i : i \in \text{TopK}(\rho, K)\}$.

E.5. CLONECLOAK($G, v_{\text{tar}}, t, \mathcal{U}, \text{flag_hb}$)

Algorithm 2 calls CLONECLOAK Algorithm 7 to edit a target node v_{tar} by cloning the cloak’s directed edges, perturbing the cloak’s features and restricting outgoing edges to the OT-guided subset $\mathcal{U}$. The flag `flag_hb` disables human follow-back (incoming edges from humans).

Outgoing cloning with restriction. Let $N^{\text{out}}(t)$ be outgoing neighbors of t . Define the allowed outgoing set:

$$\tilde{N}^{\text{out}}(t) = \begin{cases} N^{\text{out}}(t), & \mathcal{U} = \emptyset, \\ N^{\text{out}}(t) \cap \mathcal{U}, & \text{otherwise.} \end{cases} \quad (48)$$

Directed edge cloning rule. Let v_{tar} be the target node. For each outgoing edge $(t \rightarrow x) \in \mathcal{E}$ with $x \in \tilde{N}^{\text{out}}(t)$, add $(t \rightarrow x) \in \mathcal{E} \implies (v_{\text{tar}} \rightarrow x) \in \mathcal{E}'$. For incoming edges $(x \rightarrow t) \in \mathcal{E}$, we add $(x \rightarrow t) \in \mathcal{E} \implies (x \rightarrow v_{\text{tar}}) \in \mathcal{E}'$, but if $x \in \mathcal{H}$ and `flag_hb` is enabled, we *skip* cloning that incoming human edge.

Algorithm 7 CLONECLOAK($G, v_{tar}, t, \mathcal{U}, \text{flag_hb}$)

```

1: Input: graph  $G = (\mathcal{V}, \mathcal{E})$ ; target node  $v_{tar}$ ; cloak  $t$ ; restriction set  $\mathcal{U}$  and flag  $\text{flag\_hb}$ 
2: Output: updated graph  $G'$ , added edges  $E_{add}$ , and target node  $v_{tar}$ 
3: Copy the non-temporal node features of  $t$  to  $v_{tar}$ .
4: Perturb the temporal features of  $v_{tar}$  (e.g., age/timestamp-related features) for plausibility
5: Determine allowed outgoing neighbors  $\tilde{N}^{out}(t)$  via // using Equation 48
6: for each outgoing edge  $(t \rightarrow x) \in \mathcal{E}$  with  $x \in \tilde{N}^{out}(t)$  do
7:   Add  $(v_{tar} \rightarrow x)$  to  $G'$  and to  $E_{add}$ 
8: end for
9: for each incoming edge  $(x \rightarrow t) \in \mathcal{E}$  do
10:   if  $\text{flag\_hb}$  is enabled and  $x \in \mathcal{H}$  then
11:     skip (do not add human follow-back)
12:   else
13:     Add  $(x \rightarrow v_{tar})$  to  $G'$  and to  $E_{add}$ 
14:   end if
15: end for
16: return  $G', E_{add}, v_{tar}$ 

```

F. Bot Detectors (Victim Models)

The underlying bot detectors attacked by BOCLOAK are treated as frozen black-box classifiers. Our victim set includes:

- **GCN** (Kipf & Welling, 2017) standard homogeneous GNNs that operate on the social graph with node features.
- **GAT** (Veličković et al., 2018) an attention-based model for heterogeneous graphs that operate on the social graph with node features.
- **BotRGCN** (Feng et al., 2021b), a heterogeneous GNN where a standard GCN is enhanced using relation-aware message propagation for bot detection.
- **Simple-HGNN (S-HGN)** (Lv et al., 2021), a heterogeneous GNN similar to BotRGCN, but complex relation-specific modules are replaced with lightweight type-wise attention and relation-aware message propagation.
- **RGT** (Relational Graph Transformer) (Feng et al., 2022a), a heterogeneity-aware bot detector that uses relation-specific transformer blocks.

G. GNN Adversarial Defense

Although our primary focus is on evaluating BOCLOAK against *vanilla* bot detectors, we also study how GNN robustness interventions affect BOCLOAK’s performance (more details in §I.1):

- **GNNGuard** (Zhang & Zitnik, 2020), which reweights and prunes edges based on feature similarity scores, suppressing adversarial shortcuts while preserving informative neighbors.
- **GRAND** (Feng et al., 2020), a graph random neural network that uses random propagation and consistency regularization across stochastic augmentations to improve both generalization and robustness.
- **RobustGCN** (Zhu et al., 2019), which replaces deterministic neighborhood aggregation with a Gaussian-based formulation to downweight noisy or adversarial neighbors.
- **NoisyGNN** (Ennadir et al., 2024), an additional adversarial-training/architecture-noise control that injects stochasticity into hidden representations while adding minimal complexity.

H. Adversarial Attacks

We compare BOCLOAK against four SOTA adversarial attack frameworks:

- **Nettack** (Zügner et al., 2018), a targeted attack that greedily perturbs edges and features to maximally decrease the victim model’s logit margin while respecting a budget.

- **FGA** (Fast Gradient Attack) (Chen et al., 2018), which uses a first-order (locally linear) approximation of the victim model and gradient-based importance score of each edge to rank candidate edge perturbations.
- **PR-BCD** (Projected Randomized Block Coordinate Descent) (Geisler et al., 2021), a scalable white-box topology attack that optimizes a sparse adjacency-perturbation matrix over randomized edge blocks while enforcing a global edge budget.
- **Gottack** (Alom et al., 2025), a universal attack that learns graph orbit representations and optimizes a single perturbation pattern that generalizes across many target nodes and even across different victim architectures.

I. Implementation

The bot detector code was adapted from an up-to-date codebase² maintained by the authors of (Feng et al., 2022b). This codebase has been used by numerous studies and recent work (Qiao et al., 2025). We implement BOCLOAK in PyTorch and PyTorch Geometric. SOTA adversarial attack code bases³ have been adopted by running the original author’s code from their shared repository and only modifying the dataset loaders to execute on large social graphs such as TwiBot-22, which contains 1M nodes.

I.1. GNN Adversarial Defenses: Extended Discussion

We integrated three primary GNN defenses⁴: GNNGuard (Zhang & Zitnik, 2020), GRAND (Feng et al., 2020), and RobustGCN (Zhu et al., 2019), with our victim GNN bot detector models. We also include NoisyGNN (Ennadir et al., 2024) as an additional robustness control on the strongest TwiBot-22/BotRGCN setting. We faithfully reused the original defense implementations and extended them to a heterogeneous bot detector without violating any defense’s modeling assumptions.

Common interface and invariants. All victim models share a unified forward signature, so we map the raw node features into a shared hidden space using the same projection as the vanilla models. Each GNN defense then modifies either the adjacency used by the detector (GNNGuard) or appends a defense head that operates on the detector’s hidden representation (GRAND, RobustGCN). This design ensures: (i) the defended and vanilla models are comparable (same input projection, same label space), and (ii) the defense-specific changes are localized and auditable.

Summary: The *only* changes relative to the original implementation when extending to heterogeneous GNN-based bot detectors: (i) GNNGuard: relation-wise edge pruning; (ii) GRAND: a post-detector propagation head; and (iii) RobustGCN: a post-detector Gaussian aggregation head.

I.1.1. GNNGUARD

GNNGuard computes an attention score per edge via feature cosine similarity, removes edges below a threshold, and returns the pruned edges along with a renormalized edge weight vector α (Zhang & Zitnik, 2020). Crucially, the output edge list is a *subset* of the input edges, and the returned weights align one-to-one with the kept edges.

Why a relational adaptation is necessary. Victim models operate on a typed edge set, with $\text{edge_type} \in \{0, \dots, R-1\}^E$ indicating relation identity. The original GNNGuard implementation is defined for a single (homogeneous) adjacency. Applying it naively to a concatenated multi-relation edge list would (a) mix relation semantics when pruning and normalization are computed, and (b) potentially break the invariant that edge_type aligns with the filtered edges.

Our extension: per-relation filtering with type-safe reconstruction. We implement a helper that: (1) partitions edges by relation r using $\text{mask}(\text{edge_type} = r)$; (2) runs the original GNNGuard primitive *independently* on each relation-specific edges; and (3) concatenates the surviving edges across relations. This preserves two correctness properties: (i) no cross-relation edges are introduced (only deletions), and (ii) edge_type remains perfectly aligned after filtering. As in the original GNNGuard setting, the defense is realized through edge pruning and neighborhood renormalization.

Implementation Details.

- **GCN** and **BotRGCN**. We apply GNNGuard filtering per relation, then run the standard BotRGCN on the filtered edges.

²<https://github.com/LuoUndergradXJTU/TwiBot-22>

³<https://github.com/danielzuegner/nettack>, https://deeprobust.readthedocs.io/en/latest/_modules/deeprobust/graph/targeted_attack/fga.html, https://github.com/sigeisler/robustness_of_gnns_at_scale, <https://github.com/cakcora/Gottack>

⁴<https://github.com/mims-harvard/GNNGuard>, <https://github.com/THUDM/GRAND>, <https://github.com/ZW-ZHANG/RobustGCN>, <https://github.com/sennadir/noisyggn>

- **GAT.** We apply GNNGuard filtering per relation and then run the GAT on the filtered adjacency matrix.
- **S-HGN.** S-HGN uses `pre_alpha` to pass attention information from layer 1 to layer 2. This creates an important constraint: the two layers must see a *fixed* edge set so that the attention tensor dimensions match. Therefore, we *guard once per forward pass* and reuse the same filtered adjacency in both S-HGN layers. This is a correctness-critical choice: guarding separately per layer could change E between layers and invalidate the `pre_alpha` alignment.

GNNGuard Correctness. Across all three detectors, our implementation satisfies: (i) *soundness of edits*: the defense only removes edges, never fabricating new inter-node connections; (ii) *type preservation*: for typed graphs, the relation id of each retained edge is preserved and reattached consistently; (iii) *shape invariants*: in S-HGN, the edge set is fixed across layers to ensure `pre_alpha` tensor compatibility.

I.1.2. GRAND

GRAND improves robustness through stochastic random propagation (e.g., node dropout) and consistency regularization across multiple augmented views of the graph (Feng et al., 2020). The GRAND implementations expose an inference/encoder path that returns (log-)softmax probabilities rather than raw logits. We expect our detector models to return logits, from which we compute CrossEntropy (CE) loss. Passing normalized probabilities (or `log_softmax` outputs) into this logits-based loss would be inconsistent with the intended objective. We therefore introduce a lightweight wrapper, *FixedGRAND*, a wrapper around the original GRAND that preserves GRAND’s hyperparameters (drop-node rate, propagation order, number of samples), implements GRAND’s normalized propagation, and returns raw logits.

Implementation Details. GRAND’s random propagation is defined on an adjacency; it does not require typed message functions. Our design keeps relation awareness in the detectors and applies GRAND as a *regularizing smoothing-and-dropout head* on the resulting representation. Our implementation reproduces the practice of applying GRAND-style consistency regularization on top of learned embeddings while avoiding the redefinition of GRAND for every relation type (Zhang et al., 2021). The three victim models: GCN, GAT, BotRGCN, and S-HGN produce hidden states $\mathbf{H}$, then *FixedGRAND* performs GRAND propagation on $\mathbf{H}$ and outputs logits.

GRAND Correctness. Our integration ensures that: (i) the implementation preserves GRAND’s propagation mechanism (normalized adjacency + order- K averaging + node dropout), (ii) outputs are logits compatible with a CE loss, and (iii) the detector computations are unchanged, ensuring the defense layer is isolated to the intended GRAND regularization layer.

I.1.3. ROBUSTGCN

RobustGCN replaces deterministic aggregation with Gaussian-distributed hidden states that propagate both the mean and variance, and samples node representations for classification (Zhu et al., 2019). It additionally provides a KL regularizer to constrain the latent distribution.

Implementation Details. Similar to GRAND, the three victim models: GCN, GAT, BotRGCN, and S-HGN produce a hidden representation $\mathbf{H}$, then apply a RobustGCN head on $\mathbf{H}$.

RobustGCN Correctness. Our implementation ensures that we reuses the original RobustGCN forward pass (mean/variance propagation + sampling), and integrates as a modular head without changing the victim’s architecture.

Table 6. **Hyperparameters of Bot Detectors:** Hyperparameters for vanilla bot detectors and their defense variants.

Model	Variant	Layer	Number Heads	Hidden Units	Learning Rate	Batch Size	Dropout	Weight Decay	Epochs	Patience	Optimizer	Neighbour Sampling	Neighbour Aggregation	Num Edge Types	Temporal Smoothing
GCN	Vanilla	2	-	128	1×10^{-3}	128	0.5	3×10^{-5}	1000	50	Adam	256	mean	-	-
	+GNNGuard	2	-	128	1×10^{-3}	128	0.5	3×10^{-5}	1000	50	Adam	256	mean	-	-
	+GRAND	2	-	128	1×10^{-3}	128	0.5	3×10^{-5}	1000	50	Adam	256	sum	-	-
	+RobustGCN	2	-	128	1×10^{-3}	128	0.5	3×10^{-5}	1000	50	Adam	256	mean	-	-
	+NoisyGNN	2	-	128	1×10^{-3}	256	0.5	3×10^{-5}	1000	50	Adam	256	mean	2	-
GAT	Vanilla	4	2	128	1×10^{-3}	256	0.5	3×10^{-5}	1000	50	Adam	256	mean	-	-
	+GNNGuard	4	2	128	1×10^{-3}	256	0.5	3×10^{-5}	1000	50	Adam	256	mean	-	-
	+GRAND	4	2	128	1×10^{-3}	256	0.5	3×10^{-5}	1000	50	Adam	256	sum	-	-
	+RobustGCN	4	2	128	1×10^{-3}	256	0.5	3×10^{-5}	1000	50	Adam	256	mean	-	-
BotRGCN	Vanilla	2	-	128	1×10^{-3}	256	0.5	3×10^{-5}	1000	50	Adam	256	mean	2	-
	+GNNGuard	2	-	128	1×10^{-3}	256	0.5	3×10^{-5}	1000	50	Adam	256	mean	2	-
	+GRAND	2	-	128	1×10^{-3}	256	0.5	3×10^{-5}	1000	50	Adam	256	sum	2	-
	+RobustGCN	2	-	128	1×10^{-3}	256	0.5	3×10^{-5}	1000	50	Adam	256	mean	2	-
S-HGN	Vanilla	2	-	128	1×10^{-3}	256	0.5	3×10^{-5}	1000	50	Adam	256	mean	2	0.05
	+GNNGuard	2	-	128	1×10^{-2}	256	0.5	3×10^{-5}	1000	50	Adam	256	mean	2	0.05
	+GRAND	2	-	128	1×10^{-3}	256	0.5	3×10^{-5}	1000	50	Adam	256	sum	2	0.05
	+RobustGCN	2	-	128	1×10^{-2}	256	0.5	3×10^{-5}	1000	50	Adam	256	mean	2	0.05
RGT	Vanilla	4	2	256	1×10^{-3}	512	0.5	3×10^{-5}	500	50	Adam	256	-	2	-

Table 7. **Hyperparameters of GNN-based Defenses.**

GNN Defense	Hyperparameter	Value
GNNGuard	Threshold	0.1
	Add self-loops	False
GRAND	Node drop rate	0.1
	Propagation order	1
	Temperature	0.1
	Consistency Weight (λ)	0.5
	Monte Carlo Samples	5
RobustGCN	Robustness Parameter (γ)	1.0
	KL regularization	5×10^{-4}
NoisyGNN	Backbone	BotRGCN
	Noise type	Additive Gaussian hidden-state noise
	Noise scale (noise_ratio_1)	0.05; training only, disabled during inference

Table 8. Hyperparameters of Adversarial Attack Frameworks.

Attack	Setup	Scaling Safeguards	Attack Specific	Surrogate Specifics
Nettack			LR cutoff 0.004 Stop rule early stop if no admissible structural edits	
FGA			Structure edits enabled Feature edits disabled	
PR-BCD			Epochs 125 Resample 100 Loss probability margin loss Step size 1000 Directed Graph false Block size $\max(B+1, 50k)$	Architecture 2-layer GCN Hidden units 64 Epochs 200 Optimizer Adam Learning rate 1×10^{-3} Weight decay 5×10^{-4}
GOttack	$B \in \{1, 3, 5\}$; targets=50. Targeted a specific node prediction.	If $ V \geq 10^5$: local k -hop, $k=2$, max nodes=50k. Else: full graph.	Seed 720 Orbit type 1518 Orbit table precomputed per dataset.	
BoCLOAK			OT Regularizer ϵ 0.2 Sinkhorn iters 30 Cost MLP (hid, emb) (128, 256) Top boundary bots 50 Max reuse 3 Edit policy Reset per trial Add/del only for target bot Forbid human $\rightarrow$ bot follow-back Train C_θ $\lambda_{\text{BCE}}=2$, $\lambda_{\text{sp}}=0.05$, $\lambda_{\text{pl}}=0.10$ $\alpha_{\text{degree}}=0.8$, $\alpha_{\text{time}}=0.2$ Boundary margin $\tau_{\text{bdry}}=0.1$ Sigmoid temperature $\tau_{\text{BCE}}=0.01$ $\gamma=1 \times 10^{-3}$, Batch=128 Neighbor Sampling 256 k -hop neighborhood $k=1$ or 1-hop ego neighborhood	Surrogate none Guidance signal learned OT cost C_θ

J. Hyper-parameters

Table 6 and Table 7 report detector and defense hyperparameters, across GCN/GAT/BotRGCN/S-HGN based bot detectors, we largely standardize optimization (Adam, dropout 0.5, weight decay 3×10^{-5} , neighbor sampling 256, patience 50) and train to convergence with large epochs (typically 1000 epochs), so robustness comparisons are not driven by under-training. Architectural differences then reflect model capacity rather than tuning artifacts (e.g., GAT uses 4 layers with 2 heads; RGT increases hidden size to 256 and batch size to 512 while using fewer epochs, 500, to balance compute). We selected $\varepsilon = 0.2$ and $\tau_{\text{dry}} = 0.1$ from the validation sensitivity analysis in Table 22; the same settings are then fixed across datasets unless otherwise stated.

The aggregation choice also aligns with the defense mechanism: GRAND variants use sum aggregation (consistent with stochastic node dropping and consistency regularization), whereas most other settings keep mean for stability under neighbor sampling. Defense-specific hyperparameters are intentionally lightweight, GNNGuard’s similarity threshold (0.1) without self-loops, GRAND’s node drop rate (0.1) with $\lambda = 0.5$ and 5 Monte Carlo samples, RobustGCN’s $\gamma = 1.0$ with KL regularization 5×10^{-4} , and NoisyGNN injects additive Gaussian hidden-state noise with `noise_ratio_1` = 0.05 after the first GCN layer during training only; so any remaining vulnerability is less attributable to fragile tuning and more to the underlying constraints of the adversarial setting.

Table 8 summarizes the hyperparameters used to instantiate each adversarial attack baseline under a common evaluation protocol: we attack 50 target nodes per run with a small per-target edit budget $B \in \{1, 3, 5\}$. For scalability, all attacks fall back to a localized k -hop subgraph (with $k = 2$ and at most 50k nodes) when $|V| \geq 10^5$, otherwise operating on the full graph, which keeps runtime and memory comparable across datasets. Nettack uses a conservative learning-rate cutoff (0.004) and an early-stop rule when no admissible structural edits remain, while FGA is configured for edge rewiring only; PR-BCD follows its standard randomized block-coordinate optimization with 125 epochs, 100 resamples, and a probability-margin loss. LR-BCD follows the local-budget attack of Gosch et al. (2023); in our targeted setting, its node-wise local budget is set equal to the per-target budget. FGA-mod and PR-BCD-mod use the same hyperparameters as FGA and PR-BCD but project each candidate update onto the admissible bot→human edge set before the update is committed. In contrast, BOCLOAK replaces surrogate gradients entirely with a learned OT guidance signal (no surrogate model), using moderate entropic regularization $\epsilon = 0.2$ with 30 Sinkhorn iterations and a compact cost network (128, 256). The additional domain constraint guided edit-policy constraints (e.g., max reuse 3, and prohibiting human→bot follow-backs) bias perturbations toward sparse and operationally plausible edits under social-graph constraints.

K. Dataset

We evaluated BOCLOAK on three widely used social bot detection datasets: TwiBot-22 (Feng et al., 2022b), Cresci-2015 (Cresci et al., 2015), and BotSim-24 (Qiao et al., 2025), with summary statistics reported in Table 9. TwiBot-22 is by far the largest benchmark in our study (1M nodes and 170M edges), and its scale comes with heterogeneous social signals (multiple relation types and rich user metadata) that more closely resemble real-world detection settings. Consistent with this realism, the degree gap between bots and humans is present (Avg Deg. 3.56 vs. 7.00), which reduces the effectiveness of trivial heuristics and incentivizes methods that integrate both attribute and relational evidence. Overall, the three datasets span (i) large-scale heterogeneous graphs (TwiBot-22), (ii) sparse early-era spam graphs with sharp structural cues (Cresci-2015), and (iii) modern, dense, human-like interaction simulations where structural context is essential (BotSim-24), enabling a more comprehensive assessment of robustness across realistic operating conditions.

Dataset	# of Nodes	# of Edges	Avg Deg.	# of Bots	Avg Deg. (Bots)	# of Humans	Avg Deg. (Humans)
Cresci-15	5,301	14,220	2.05	3,351	0.22	1,950	5.18
TwiBot-22	1,000,000	170,185,937	6.67	139,943	3.56	860,057	7.00
BotSim-24	2,907	46,518	45.39	1,000	19.23	1,907	59.12

Table 9. **Dataset Statistics:** Detailed bot detection dataset summaries.

In contrast, Cresci-2015 is substantially smaller and markedly sparse (Avg Deg. 2.05), with bots exhibiting very low connectivity on average (0.22) compared to humans (5.18), reflecting early-generation spam/fake-follower behaviors and producing a clearer structural separation. Together, these two datasets provide complementary regimes: a large, heterogeneous and sparse graph (TwiBot-22) versus a small, sparse graph with strong degree-based signals (Cresci-2015), for stress-testing both effectiveness and scalability.

BotSim-24 is the newest dataset and is designed to simulate highly human-like bot interactions, making the *node features* alone more challenging to distinguish from genuine users; in such cases, leveraging graph structure becomes crucial. This dataset is also notably dense despite its small size (Avg Deg. 45.39), and the connectivity patterns are different across classes: humans have very high average degree (59.12) while bots remain substantially connected (19.23), showcasing that bots are embedded in interaction neighborhoods rather than being isolated. Therefore, it increases redundancy in local neighborhoods (potentially stabilizing message passing) while simultaneously reducing the space of plausible structural perturbations available to an adversary.

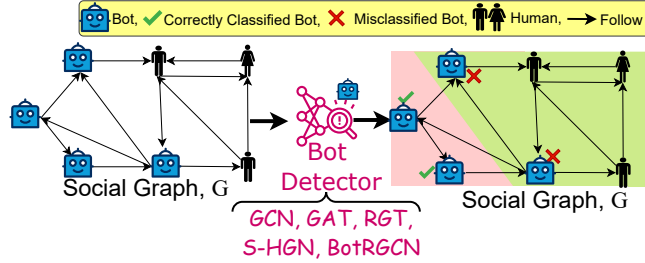


Figure 6. Overview of bot detector detection pipeline.

Dataset	Victim	Vanilla ($\uparrow$)	GNN Defense		
			+GNNGuard ($\uparrow$)	+GRAND ($\uparrow$)	+RobustGCN ($\uparrow$)
Cresci-15	GCN	0.93 ± 0.01	0.93 ± 0.05	0.93 ± 0.01	0.92 ± 0.06
	GAT	0.97 ± 0.03	0.97 ± 0.05	0.98 ± 0.01	0.97 ± 0.07
	BotRGCN	0.98 ± 0.08	0.98 ± 0.05	0.98 ± 0.03	0.98 ± 0.02
	S-HGN	0.97 ± 0.02	0.98 ± 0.06	0.96 ± 0.02	0.97 ± 0.07
	RGT	0.97 ± 0.02	0.97 ± 0.01	0.97 ± 0.03	<u>0.98 ± 0.04</u>
TwiBot-22	GCN	0.52 ± 0.05	0.50 ± 0.05	0.48 ± 0.03	0.51 ± 0.04
	GAT	0.52 ± 0.02	0.50 ± 0.10	0.53 ± 0.00	<u>0.53 ± 0.01</u>
	BotRGCN	0.58 ± 0.08	0.52 ± 0.05	0.56 ± 0.01	0.51 ± 0.06
	S-HGN	0.58 ± 0.02	0.50 ± 0.08	0.56 ± 0.02	0.55 ± 0.03
BotSim-24	RGT	0.42 ± 0.04	0.40 ± 0.01	0.41 ± 0.02	0.38 ± 0.01
	GCN	0.90 ± 0.01	0.92 ± 0.03	0.93 ± 0.01	<u>0.93 ± 0.02</u>
	GAT	0.95 ± 0.05	0.90 ± 0.01	0.92 ± 0.02	0.89 ± 0.03
	BotRGCN	0.96 ± 0.03	0.91 ± 0.01	0.90 ± 0.06	0.91 ± 0.14
	S-HGN	0.92 ± 0.09	0.94 ± 0.01	0.93 ± 0.03	0.92 ± 0.20
	RGT	0.93 ± 0.05	<u>0.94 ± 0.03</u>	0.92 ± 0.01	0.94 ± 0.03

Table 10. **Node Classification Performance (F1 score):** Bot detection performance of different SOTA bot detector architectures with GNN-based adversarial defenses. The best performance is shown in bold, and the second best is underlined.

L. Bot Detection Performance

Table 10 reports the F1 scores of vanilla bot detectors and their adversarially defended variants, detection pipeline illustrated in Figure 6. Across datasets, **BotRGCN** is the strongest overall detector: it attains the top vanilla performance on Cresci-15 (0.98) and BotSim-24 (0.96), and remains among the top performers on TwiBot-22 (0.58, tied for best), suggesting that explicitly modeling heterogeneous relation types is consistently beneficial when interaction semantics matter. GAT and S-HGN form a close second and third best bot detector, frequently appearing as runner-up (e.g., S-HGN is second on Cresci-15 vanilla and GAT is second on BotSim-24 vanilla), indicating that attention or heterophily-aware designs can match BotRGCN when the relational signal is strong. A consistent pattern as noted by other works (Qiao et al., 2025) also, is that absolute F1 scores on TwiBot-22 are lower for all models than on Cresci-15 and BotSim-24, reflecting the larger scale and higher ambiguity of TwiBot-22 where graph size, diverse behaviors, and heterogeneous relations makes behavioral modeling challenging.

Defense variants exhibit mixed behaviors such as on Cresci-15, defenses largely preserve already vanilla performance, with small gains such as GAT+GRAND, consistent with defenses acting as mild regularizers when the classification boundary is easy. In contrast, on TwiBot-22 the best defended results are achieved by vanilla backbones rather than by defenses alone (e.g., BotRGCN remains best), suggesting that robustness mechanisms cannot compensate for a backbone that under-utilizes heterogeneous structure.

But, several defenses *decrease* performance on BotSim-24 for the strongest vanilla models, which is showcases that

Attack	Unconstrained Vanilla	Constrained Vanilla	+NoisyGNN
Nettack	79.77 $\pm$ 6.77	9.33 $\pm$ 3.06	4.34 $\pm$ 8.09
FGA	71.90 $\pm$ 4.23	4.00 $\pm$ 2.00	3.65 $\pm$ 1.11
PR-BCD	75.77 $\pm$ 6.05	8.67 $\pm$ 2.31	5.45 $\pm$ 0.13
Gottack	84.29 $\pm$ 11.63	6.89 $\pm$ 3.33	4.11 $\pm$ 1.11
BOCLOAK (ours)	86.67$\pm$2.31	86.67$\pm$2.31	85.12$\pm$2.23

Table 11. NoisyGNN robustness results on TwiBot-22 against the strongest BotRGCN detector with node-editing budget $\Delta = 1$: NoisyGNN injects Gaussian noise into the hidden state after the first relational message-passing layer during training only. Even with this stochastic hidden-state defense, BOCLOAK remains substantially stronger than constrained generic graph attacks.

aggressive perturbation filtering or stochastic regularization can remove informative dense-neighborhood signals in human-like interaction graphs. Overall, Table 10 suggests that (i) choosing a relation-aware backbone (BotRGCN) provides the most reliable gains, while (ii) defenses behave like regularizers whose benefit depends on its defense procedure.

M. NoisyGNN Adversarial Robustness

Table 11 reports an additional robustness-control experiment using NoisyGNN (Ennadir et al., 2024) on the strongest BotRGCN detector setting. Unlike GNNGuard, GRAND, and RobustGCN, which are evaluated as general defense variants, NoisyGNN is instantiated here as a targeted control on the relation-aware BotRGCN backbone. The defense injects Gaussian noise into the hidden representation after the first GCN layer during training and disables this noise during inference.

The results show that NoisyGNN further reduces the success of generic constrained attacks, with Nettack, FGA, PR-BCD, and Gottack remaining below 6% misclassification. In contrast, BOCLOAK maintains an 85.12% misclassification rate under the same constrained setting.

This suggests that stochastic hidden-state regularization alone does not remove the vulnerability exploited by BOCLOAK, because BOCLOAK operates by constructing domain-feasible neighborhood distributions rather than relying on fragile gradient perturbations.

Dataset	Attack	GAT				BotRGCN				S-HGN			
		Vanilla (↑)	+GNNGuard (↑)	+GRAND (↑)	+RobustGCN (↑)	Vanilla (↑)	+GNNGuard (↑)	+GRAND (↑)	+RobustGCN (↑)	Vanilla (↑)	+GNNGuard (↑)	+GRAND (↑)	+RobustGCN (↑)
Δ = 1													
Cresci-15	Random	5.34±1.65	3.75±2.12	3.80±2.85	3.11±2.57	8.67±5.03	7.33±2.33	6.67±3.43	8.00±0.20	5.11±xxx	4.46±xxx	4.18±xxx	3.56±xxx
	Nettack	8.17±3.23	6.59±3.40	7.54±2.07	7.28±3.19	4.32±2.10	3.78±1.95	3.21±1.88	2.67±1.72	6.04±3.00	4.81±3.18	4.51±3.60	4.88±2.71
	FGA	8.52±1.92	5.74±2.39	4.73±3.67	7.93±2.91	3.95±2.44	3.40±2.12	2.98±1.76	2.15±1.60	5.22±3.17	4.96±3.09	4.31±2.07	3.44±2.01
	PR-BCD	8.79±1.91	6.85±2.35	7.48±3.27	7.92±3.24	4.10±3.05	3.60±2.80	2.85±2.20	1.90±1.45	7.57±3.84	6.91±2.43	6.70±2.84	5.58±2.45
	Gottack	7.61±2.91	7.14±3.65	5.08±3.06	6.20±1.94	3.55±2.34	3.41±2.52	2.28±1.22	2.66±1.67	8.10±2.83	7.57±2.04	7.92±1.87	6.39±2.12
	BOCLOAK (ours)	93.10±2.34	94.20±1.89	92.45±3.11	91.88±4.56	99.34±0.29	98.91±0.54	99.06±0.41	99.72±0.18	92.18±4.73	91.60±2.15	91.92±3.08	91.40±1.67
Twibot-22	Random	6.67±2.45	3.12±2.11	3.87±2.84	4.48±1.01	5.12±6.11	2.00±3.46	5.33±13.61	3.00±6.00	4.55±1.69	3.50±1.48	3.11±2.83	3.76±2.55
	Nettack	13.51±8.15	11.92±9.38	8.59±7.73	12.91±8.91	9.33±0.06	8.67±4.62	4.67±5.03	8.00±5.20	5.03±4.72	3.24±9.20	3.72±10.81	4.73±7.85
	FGA	12.32±8.10	8.59±4.52	9.94±6.41	5.57±10.78	4.00±2.00	7.33±2.31	6.67±4.16	1.33±6.43	5.79±6.52	4.57±4.96	3.41±6.87	3.19±9.89
	PR-BCD	13.51±4.52	12.52±9.38	11.32±6.41	12.24±4.52	8.67±2.31	8.67±5.03	6.23±4.16	7.33±2.31	4.58±5.29	3.74±3.34	3.61±7.20	3.20±7.92
	Gottack	11.41±7.59	10.50±6.38	9.78±3.93	10.33±5.34	6.89±3.33	7.15±4.25	6.01±2.48	5.32±2.58	5.55±7.98	4.04±8.93	4.44±7.82	4.95±8.39
	BOCLOAK (ours)	94.38±3.41	93.72±1.66	93.90±4.28	92.26±2.13	86.67±2.31	84.67±11.02	83.67±3.06	85.30±2.00	95.66±2.27	94.88±4.93	94.41±1.62	94.30±3.55
BotSim-24	Random	3.50±1.04	2.10±1.45	2.67±1.50	1.55±2.30	3.03±2.06	0.67±3.06	2.33±4.16	1.33±4.16	2.87±1.50	2.09±1.33	2.50±1.87	2.35±2.47
	Nettack	5.52±1.92	3.74±2.39	4.73±3.67	4.93±8.10	0.00±0.00	0.00±0.00	0.00±0.00	3.00±2.00	4.71±2.19	3.44±3.09	3.35±2.67	3.65±7.44
	FGA	5.79±1.91	4.85±2.35	4.48±2.37	4.92±3.24	0.00±0.00	0.00±0.00	0.00±0.00	0.00±0.00	5.72±2.56	4.63±2.12	5.06±3.15	4.54±3.73
	PR-BCD	7.32±8.10	6.59±4.52	6.65±7.59	7.14±3.65	2.00±2.00	5.33±2.31	0.67±1.15	0.00±0.00	5.20±8.04	4.24±4.63	4.43±4.99	4.26±1.88
	Gottack	7.08±3.06	6.20±1.94	6.67±3.09	5.87±2.27	0.00±0.00	0.00±0.00	0.00±0.00	0.00±0.00	4.42±1.83	3.25±3.38	3.65±3.12	3.21±2.45
	BOCLOAK (ours)	88.74±3.12	90.05±1.47	89.30±4.66	88.40±2.29	58.22±7.20	52.68±1.15	54.98±2.45	55.10±3.98	89.16±4.25	90.44±2.11	89.78±1.63	88.90±3.74
Δ = 3													
Cresci-15	Random	6.67±9.08	5.78±1.26	3.88±3.25	3.50±6.84	8.00±10.58	8.67±3.06	3.33±1.15	2.00±9.17	6.33±12.08	5.56±4.86	5.78±3.25	3.50±11.50
	Nettack	10.94±6.10	9.71±4.26	7.72±3.19	5.74±3.99	13.98±10.50	12.10±1.70	12.98±1.10	11.10±1.74	9.44±4.06	8.18±8.08	6.62±2.69	6.75±2.41
	FGA	11.73±3.67	7.93±2.91	7.79±1.91	7.85±2.35	12.65±3.20	10.95±2.85	9.10±2.31	7.60±1.68	8.79±2.89	6.48±3.76	4.24±2.19	4.90±3.57
	PR-BCD	12.48±2.37	8.92±3.24	6.61±2.91	7.14±3.65	11.98±1.02	10.40±1.92	8.90±1.70	9.35±1.46	8.10±3.69	7.67±2.77	7.49±1.75	5.69±2.46
	Gottack	9.08±3.06	6.20±1.94	8.67±3.09	5.87±2.27	12.88±2.48	11.10±2.20	9.60±1.84	9.70±1.50	9.34±2.95	8.75±7.06	8.18±8.01	7.39±3.19
	BOCLOAK (ours)	95.67±3.02	96.40±2.41	94.90±1.77	93.76±5.12	99.18±0.37	98.47±0.96	99.59±0.21	99.01±0.44	94.55±2.84	95.10±4.12	93.80±1.39	92.66±5.58
Twibot-22	Random	7.34±0.81	4.00±2.65	4.22±0.21	4.50±2.33	5.67±2.31	3.33±1.15	3.67±2.31	2.00±0.20	5.00±3.81	4.66±2.65	3.12±4.41	3.50±2.33
	Nettack	14.21±8.12	14.09±7.80	10.38±5.54	10.98±8.55	10.22±8.83	11.78±1.41	5.10±1.41	5.01±4.24	10.80±5.05	9.65±4.64	9.69±3.15	9.56±7.92
	FGA	14.51±3.15	12.52±9.38	14.23±3.09	6.65±1.32	8.15±2.83	8.66±2.83	9.87±7.07	2.10±5.66	9.08±4.79	8.97±6.62	8.30±4.92	7.70±11.67
	PR-BCD	18.38±1.54	19.98±9.38	15.54±3.12	9.94±1.00	15.10±7.07	16.50±2.83	10.00±5.66	4.64±2.83	9.45±3.83	8.77±2.29	8.28±5.54	8.56±6.54
	Gottack	14.10±7.59	13.83±5.57	10.90±7.46	11.22±9.65	3.12±1.11	8.48±3.65	5.17±2.59	5.68±5.28	7.64±2.74	6.70±2.23	7.93±3.01	7.16±3.07
	BOCLOAK (ours)	96.15±2.58	95.40±4.62	95.80±1.49	95.12±3.33	94.50±2.00	86.20±4.59	84.67±3.06	87.33±1.15	97.42±1.39	96.15±3.74	95.90±2.06	95.11±4.28
BotSim-24	Random	4.20±3.09	3.40±0.30	3.57±2.16	1.72±1.86	2.00±5.29	2.00±2.00	2.67±4.16	8.67±4.16	5.80±7.49	3.60±3.70	4.77±6.16	4.62±6.46
	Nettack	6.73±3.67	5.93±2.91	5.79±1.91	5.90±7.09	0.00±0.00	0.00±0.00	0.00±0.00	5.23±0.01	6.78±2.62	5.54±2.22	4.24±2.33	4.07±1.22
	FGA	7.48±2.37	6.92±3.24	6.61±2.91	7.14±3.65	0.00±0.00	0.00±0.00	0.00±0.00	0.00±0.00	6.75±1.76	4.68±1.93	5.14±2.22	5.07±1.75
	PR-BCD	8.38±8.12	8.32±7.09	7.93±9.91	6.67±1.83	1.00±1.41	4.00±0.00	2.00±0.00	0.00±0.00	6.33±4.68	6.02±2.12	5.83±2.26	5.49±3.74
	Gottack	8.08±3.06	6.20±1.94	8.67±3.09	5.87±2.27	0.00±0.00	0.00±0.00	0.00±0.00	0.00±0.00	5.78±2.01	4.90±2.20	5.12±8.07	5.26±3.44
	BOCLOAK (ours)	91.28±2.06	92.60±3.55	90.85±1.33	89.92±4.41	93.33±1.15	66.74±3.45	88.63±0.87	64.32±3.33	92.03±1.92	93.40±4.08	91.55±2.36	90.62±5.11
Δ = 5													
Cresci-15	Random	10.10±3.49	9.30±1.00	9.12±2.56	9.42±1.96	12.00±5.29	12.00±2.00	12.67±4.16	18.67±4.16	12.90±2.09	10.70±4.10	10.22±5.76	9.92±6.36
	Nettack	14.38±8.12	12.09±7.80	12.16±7.47	13.42±8.23	22.35±3.18	20.80±2.85	18.95±2.30	16.40±1.62	15.50±8.01	12.11±5.83	12.04±3.67	14.60±4.68
	FGA	10.61±2.91	9.14±3.65	9.08±3.06	9.20±1.94	21.90±2.74	20.15±2.40	18.40±2.05	15.95±1.48	15.65±2.63	14.03±2.57	15.53±2.01	14.42±3.51
	PR-BCD	14.67±3.09	13.87±2.27	12.67±1.83	12.74±3.19	20.75±1.96	19.30±1.74	17.85±1.60	15.40±1.36	16.64±2.60	14.49±2.74	14.77±2.26	15.91±3.68
	Gottack	11.61±3.07	10.38±2.98	10.74±2.39	11.17±3.23	22.50±3.55	21.05±3.10	19.10±2.65	16.85±1.70	14.68±2.22	12.65±2.56	13.84±2.02	12.65±3.12
	BOCLOAK (ours)	97.82±1.58	97.30±2.96	96.55±3.44	95.90±4.01	99.67±0.20	98.88±0.58	99.11±0.39	99.52±0.24	96.93±1.44	95.50±3.66	95.70±2.09	94.88±4.37
Twibot-22	Random	10.93±5.98	9.63±2.47	9.72±2.23	5.83±3.13	11.33±8.08	11.33±9.02	12.67±6.43	7.33±5.03	13.73±10.18	9.03±1.62	11.62±6.83	8.83±3.93
	Nettack	18.32±7.09	19.98±7.09	10.90±7.46	11.22±9.65	14.22±20.03	14.70±10.00	5.77±2.24	7.78±4.24	18.17±6.05	15.38±6.65	10.80±7.51	10.93±10.03
	FGA	15.54±3.09	13.16±1.32	11.23±8.12	13.51±18.86	11.30±7.07	10.80±5.66	6.21±2.83	9.22±12.73	16.65±10.19	13.38±7.99	10.81±0.96	14.32±19.53
	PR-BCD	19.98±11.32	16.43±3.09	15.54±1.32	12.51±8.12	24.73±5.66	21.53±7.07	14.45±5.66	19.00±2.83	18.18±11.78	13.98±11.75	12.78±11.10	12.12±5.83
	Gottack	16.54±7.59	13.83±9.56	14.87±7.47	15.55±8.23	10.58±2.15	18.25±3.65	12.89±1.02	13.32±3.03	14.06±6.30	13.26±7.04	11.61±4.93	11.20±4.60
	BOCLOAK (ours)	99.70±1.37	98.20±2.91	97.85±3.76	98.90±4.15	94.00±2.00	90.00±4.82	93.67±3.06	88.33±1.15	99.12±1.88	98.48±2.35	98.36±3.19	97.40±5.02
BotSim-24	Random	3.25±1.86	1.27±0.11	2.00±2.65	2.90±2.10	5.00±3.46	2.67±2.31	3.33±1.15	0.00±0.00	5.75±5.06	3.07±4.51	2.86±2.65	2.90±2.10
	Nettack	8.79±1.91	7.85±2.35	7.48±2.37	6.92±3.24	0.00±0.00	0.00±0.00	0.00±0.00	0.00±0.00	8.00±3.19	7.24±2.87	7.65±1.96	7.42±2.01
	FGA	8.61±2.91	8.65±2.59	5.08±3.06	7.59±7.80	0.00±0.00	0.67±1.15	0.00±0.00	2.67±1.15	8.91±2.63	7.78±3.36	6.77±3.09	7.36±4.99
	PR-BCD	9.38±8.12	8.32±1.32	7.93±2.91	5.74±2.39	2.67±1.15	4.00±2.83	1.00±1.41	0.00±0.00	8.53±2.68	7.10±7.99	5.48±4.34	8.24±2.65
	Gottack	10.67±9.09	7.59±3.40	6.67±1.83	6.74±3.19	2.00±0.00	2.00±2.00	0.00±0.00	0.00±0.00	7.09±6.59	6.00±6.57	5.86±2.54	5.91±6.28
	BOCLOAK (ours)	94.90±1.41	95.22±2.94	93.75±3.67	92.40±5.09	99.33±1.15	79.21±1.02	92.28±0.33	88.20±3.58	95.44±1.29	95.50±2.87	94.10±3.88	93.25±4.63

N. Bot Editing Results against SOTA Bot Detectors

Table 12 shows the bot editing results using BOCLOAK against the second- and third- best SOTA bot detectors and their adversarial defense variants. BOCLOAK remains the strongest attacker across datasets, budgets, and defenses, achieving consistently **80.13%** high misclassification rate and substantially outperforming SOTA adversarial attacks.

A consistent trend in Table 12 is that domain-constrained SOTA adversarial attacks remain in the single-digit to low-teen misclassification range across both GAT and S-HGN and their defenses, even as the edit budget increases. In contrast, BOCLOAK produces large-scale flips under the same constraints, staying in the high-success regime on Cresci-15 and TwiBot-22 (typically 90–99%+) across vanilla and defended detector variants. This gap indicates that BOCLOAK is not merely exploiting unconstrained rewiring freedom; it is effective even when edits must remain operationally feasible under domain constraints.

BotSim-24 consistently behaves as the most difficult dataset. Unlike Cresci-15 and TwiBot-22 (where BOCLOAK saturates quickly), BotSim-24 shows noticeably lower misclassification at smaller budgets and exhibits larger sensitivity to defense choice. This suggests that, in BotSim-24, producing a human-like neighborhood with sparse feasible edits is intrinsically harder, and robust training/denoising defenses can partially suppress boundary transfer. Importantly, however, the same setting also highlights how limited constrained SOTA attacks are: they frequently remain near-zero to low-single-digit success, while BOCLOAK still achieves substantially higher flip rates, preserving a clear best performer even in the most resistant dataset.

Dataset	GCN (↑)	GAT (↑)	BotRGCN (↑)	S-HGN (↑)	RGT (↑)
budget $\Delta = 1$					
Cresci-15	<u>99.12$\pm$0.45</u>	98.76 $\pm$ 0.62	97.94 $\pm$ 1.10	98.31 $\pm$ 0.88	99.43$\pm$0.29
TwiBot-22	97.85 $\pm$ 1.34	96.92 $\pm$ 1.87	<u>98.14$\pm$0.97</u>	97.26 $\pm$ 1.55	98.67$\pm$0.73
BotSim-24	18.30$\pm$1.50	<u>15.20$\pm$1.50</u>	14.50 $\pm$ 2.50	12.33 $\pm$ 1.15	13.50 $\pm$ 1.25
budget $\Delta = 3$					
Cresci-15	98.95 $\pm$ 0.58	99.27$\pm$0.41	97.63 $\pm$ 1.22	98.40 $\pm$ 0.91	<u>99.08$\pm$0.53</u>
TwiBot-22	96.74 $\pm$ 2.05	97.58 $\pm$ 1.43	<u>98.02$\pm$1.01</u>	96.89 $\pm$ 1.96	98.55$\pm$0.79
BotSim-24	36.50$\pm$1.75	31.33 $\pm$ 2.50	32.00 $\pm$ 1.00	<u>33.33$\pm$1.25</u>	33.15 $\pm$ 2.11
budget $\Delta = 5$					
Cresci-15	99.51$\pm$0.24	99.04 $\pm$ 0.54	98.59 $\pm$ 0.76	<u>99.21$\pm$0.33</u>	98.62 $\pm$ 0.71
TwiBot-22	97.11 $\pm$ 1.92	98.73$\pm$0.69	97.24 $\pm$ 1.88	98.11 $\pm$ 1.05	<u>98.41$\pm$0.86</u>
BotSim-24	46.22 $\pm$ 0.07	44.67 $\pm$ 3.77	<u>47.49$\pm$1.36</u>	58.88$\pm$0.61	43.22 $\pm$ 1.02
budget $\Delta = 20$					
Cresci-15	99.76$\pm$0.18	99.04 $\pm$ 0.54	98.59 $\pm$ 0.76	99.21 $\pm$ 0.33	<u>99.58$\pm$0.22</u>
TwiBot-22	97.68 $\pm$ 1.57	<u>98.73$\pm$0.69</u>	97.24 $\pm$ 1.88	98.11 $\pm$ 1.05	99.02$\pm$0.48
BotSim-24	<u>68.50$\pm$0.75</u>	64.67 $\pm$ 3.77	67.49 $\pm$ 1.36	68.88$\pm$0.61	65.15 $\pm$ 0.30

Table 13. **Node Injection Attack:** Misclassification rate (in %) for successfully injecting fifty target nodes by BOCLOAK against vanilla SOTA bot detectors. For attacks against defended SOTA bot detector refer to Table 15. The best performance per dataset is shown in bold, and the second best is underlined.

Dataset	GCN (↑)	GAT (↑)	BotRGCN (↑)	S-HGN (↑)	RGT (↑)
budget $\Delta = 1$					
Cresci-15	100.00$\pm$0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	<u>100.00$\pm$0.00</u>
TwiBot-22	100.00$\pm$0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	<u>100.00$\pm$0.00</u>
BotSim-24	12.75 $\pm$ 1.00	10.33 $\pm$ 1.33	11.75 $\pm$ 1.00	<u>12.30$\pm$1.50</u>	16.30$\pm$1.25
budget $\Delta = 3$					
Cresci-15	100.00$\pm$0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	<u>100.00$\pm$0.00</u>
TwiBot-22	100.00$\pm$0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	<u>100.00$\pm$0.00</u>
BotSim-24	32.50$\pm$1.00	30.00 $\pm$ 1.33	31.00 $\pm$ 1.00	<u>31.30$\pm$1.00</u>	26.30 $\pm$ 1.50
budget $\Delta = 5$					
Cresci-15	100.00$\pm$0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	<u>100.00$\pm$0.00</u>
TwiBot-22	100.00$\pm$0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	<u>100.00$\pm$0.00</u>
BotSim-24	56.30$\pm$1.50	51.00 $\pm$ 2.50	50.33 $\pm$ 1.50	<u>55.30$\pm$1.33</u>	56.30 $\pm$ 1.33
budget $\Delta = 20$					
Cresci-15	100.00$\pm$0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	<u>100.00$\pm$0.00</u>
TwiBot-22	100.00$\pm$0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	<u>100.00$\pm$0.00</u>
BotSim-24	100.00$\pm$0.00	64.67 $\pm$ 3.77	71.33 $\pm$ 1.50	<u>68.50$\pm$1.33</u>	100.00 $\pm$ 0.00

Table 14. **Node Injection Attack Reusing Same Bot Cloak:** Misclassification rate (in %) for successfully injecting fifty target nodes by BOCLOAK against vanilla SOTA bot detectors, *reusing* the same bot cloak. The best performance per dataset is shown in bold, and the second best is underlined.

O. Bot Injection Results against SOTA Bot Detectors

Table 13 illustrates the bot injection results using BOCLOAK against SOTA bot detectors. Table 15 shows the bot injection results using BOCLOAK against the best three SOTA bot detectors and their adversarial defense variants. Unlike node editing, there is no established *node-injection* attack for social bot detection under realistic constraints; accordingly, we compare BOCLOAK to a constrained random injection baseline.

Across Cresci-15 and TwiBot-22, BOCLOAK achieves near-ceiling injection success even at very small budgets, consistently reaching close to 99% misclassification across *all* vanilla detectors in Table 13. We demonstrate that BOCLOAK is (to our knowledge) the first framework to explicitly evaluate and succeed at *domain-constrained node injection* on social bot detectors, rather than restricting attention to perturbing existing nodes. Injection here is constrained to *only* add edges for a new bot, without forcing human follow-backs, so success depends on constructing a locally human-like ego-neighborhood under a tight feasibility budget.

BOCLOAK : Optimal Transport-Guided Adversarial Attacks on Graph Neural Network-Based Bot Detection

BotRGCN					
Dataset	Attack	Vanilla (↑)	+GNNGuard (↑)	+GRAND (↑)	+RobustGCN (↑)
Cresci-15	Random, $\Delta = 1$	12.84 $\pm$ 0.13	11.33 $\pm$ 1.15	10.67 $\pm$ 1.15	10.52 $\pm$ 0.27
	Random, $\Delta \leq 3$	22.00 $\pm$ 2.10	21.00 $\pm$ 2.22	22.67 $\pm$ 1.15	21.33 $\pm$ 3.06
	Random, $\Delta \leq 5$	33.33 $\pm$ 2.31	33.67 $\pm$ 1.15	31.11 $\pm$ 0.74	32.67 $\pm$ 2.31
	Random, $\Delta = 20$	96.67 $\pm$ 1.15	95.33 $\pm$ 1.89	95.32 $\pm$ 1.98	94.74 $\pm$ 0.18
	BOCLOAK, $\Delta \leq 5$	98.59$\pm$0.76	98.23$\pm$0.63	98.77$\pm$0.89	99.61$\pm$0.31
TwiBot-22	Random, $\Delta = 1$	13.33 $\pm$ 1.15	11.33 $\pm$ 1.15	16.00 $\pm$ 3.46	10.33 $\pm$ 4.62
	Random, $\Delta \leq 3$	25.33 $\pm$ 4.62	23.33 $\pm$ 2.31	23.67 $\pm$ 2.41	20.33 $\pm$ 1.15
	Random, $\Delta \leq 5$	36.67 $\pm$ 3.06	34.67 $\pm$ 3.06	33.00 $\pm$ 2.89	34.00 $\pm$ 2.22
	Random, $\Delta = 20$	96.67 $\pm$ 3.06	96.67 $\pm$ 1.15	95.33 $\pm$ 4.16	95.33 $\pm$ 1.15
	BOCLOAK, $\Delta \leq 5$	97.24$\pm$1.88	97.86$\pm$0.71	97.47$\pm$0.26	98.12$\pm$1.84
BotSim-24	Random, $\Delta = 1$	10.00 $\pm$ 3.46	15.33 $\pm$ 6.11	11.33 $\pm$ 4.16	19.33 $\pm$ 9.87
	Random, $\Delta \leq 3$	4.00 $\pm$ 2.00	0.67 $\pm$ 1.15	5.33 $\pm$ 1.15	11.23 $\pm$ 5.03
	Random, $\Delta \leq 5$	3.33 $\pm$ 1.15	3.33 $\pm$ 2.31	6.67 $\pm$ 2.31	8.67 $\pm$ 2.31
	Random, $\Delta = 20$	22.00 $\pm$ 6.93	11.33 $\pm$ 7.02	9.33 $\pm$ 3.06	12.67 $\pm$ 4.62
	BOCLOAK, $\Delta \leq 5$	47.49$\pm$1.36	46.12$\pm$0.52	42.41$\pm$0.98	40.68$\pm$0.21
GAT					
Dataset	Attack	Vanilla (↑)	+GNNGuard (↑)	+GRAND (↑)	+RobustGCN (↑)
Cresci-15	Random, $\Delta = 1$	14.00 $\pm$ 2.00	15.33 $\pm$ 3.06	13.00 $\pm$ 3.46	10.67 $\pm$ 2.31
	Random, $\Delta \leq 3$	29.00 $\pm$ 2.00	25.67 $\pm$ 2.31	23.67 $\pm$ 1.15	29.33 $\pm$ 1.15
	Random, $\Delta \leq 5$	35.33 $\pm$ 4.16	35.03 $\pm$ 1.15	30.00 $\pm$ 2.00	31.33 $\pm$ 4.16
	Random, $\Delta = 20$	97.33 $\pm$ 1.15	96.67 $\pm$ 3.06	94.00 $\pm$ 6.93	92.00 $\pm$ 3.46
	BOCLOAK, $\Delta \leq 5$	99.04$\pm$0.54	98.41$\pm$0.28	98.93$\pm$0.57	99.72$\pm$0.09
TwiBot-22	Random, $\Delta = 1$	19.03 $\pm$ 2.00	15.33 $\pm$ 3.06	16.00 $\pm$ 3.46	10.67 $\pm$ 2.31
	Random, $\Delta \leq 3$	25.00 $\pm$ 2.00	25.67 $\pm$ 2.31	24.67 $\pm$ 1.15	23.33 $\pm$ 1.15
	Random, $\Delta \leq 5$	36.33 $\pm$ 4.16	35.33 $\pm$ 1.15	36.00 $\pm$ 2.00	31.33 $\pm$ 4.16
	Random, $\Delta = 20$	92.33 $\pm$ 1.15	86.67 $\pm$ 3.06	89.00 $\pm$ 6.93	83.00 $\pm$ 3.46
	BOCLOAK, $\Delta \leq 5$	98.73$\pm$0.69	91.58$\pm$0.92	90.11$\pm$0.44	87.76$\pm$1.36
BotSim-24	Random, $\Delta = 1$	11.13 $\pm$ 6.11	10.67 $\pm$ 4.62	17.31 $\pm$ 5.03	11.13 $\pm$ 5.03
	Random, $\Delta \leq 3$	2.00 $\pm$ 1.15	8.67 $\pm$ 3.06	24.00 $\pm$ 2.00	10.00 $\pm$ 0.00
	Random, $\Delta \leq 5$	12.00 $\pm$ 8.72	5.33 $\pm$ 4.16	25.33 $\pm$ 5.03	4.67 $\pm$ 1.15
	Random, $\Delta = 20$	14.67 $\pm$ 6.43	32.00 $\pm$ 5.29	8.00 $\pm$ 5.29	6.67 $\pm$ 6.43
	BOCLOAK, $\Delta \leq 5$	44.67$\pm$3.77	46.67$\pm$3.06	46.67$\pm$7.74	49.36$\pm$0.51
S-HGN					
Dataset	Attack	Vanilla (↑)	+GNNGuard (↑)	+GRAND (↑)	+RobustGCN (↑)
Cresci-15	Random, $\Delta = 1$	14.33 $\pm$ 1.15	14.40 $\pm$ 0.12	15.85 $\pm$ 0.09	10.23 $\pm$ 0.10
	Random, $\Delta = 3$	27.10 $\pm$ 0.11	26.75 $\pm$ 0.07	25.20 $\pm$ 0.13	27.93 $\pm$ 3.15
	Random, $\Delta = 5$	36.65 $\pm$ 0.14	33.90 $\pm$ 0.10	36.33 $\pm$ 1.15	32.33 $\pm$ 8.17
	Random, $\Delta = 20$	99.33 $\pm$ 1.15	92.80 $\pm$ 0.08	99.33 $\pm$ 1.15	98.00 $\pm$ 3.46
	BOCLOAK, $\Delta \leq 5$	99.21$\pm$0.33	99.35$\pm$0.12	99.78$\pm$0.05	99.10$\pm$0.28
TwiBot-22	Random, $\Delta = 1$	15.33 $\pm$ 3.06	12.27 $\pm$ 1.15	12.00 $\pm$ 3.46	10.67 $\pm$ 9.24
	Random, $\Delta = 3$	24.00 $\pm$ 2.00	21.67 $\pm$ 2.31	22.67 $\pm$ 4.16	24.77 $\pm$ 2.31
	Random, $\Delta = 5$	38.00 $\pm$ 0.00	33.67 $\pm$ 5.31	36.67 $\pm$ 1.15	37.33 $\pm$ 6.11
	Random, $\Delta = 20$	98.67 $\pm$ 2.31	98.67 $\pm$ 2.31	92.67 $\pm$ 2.31	91.33 $\pm$ 6.11
	BOCLOAK, $\Delta \leq 5$	98.11$\pm$1.05	99.86$\pm$0.03	99.18$\pm$0.22	99.70$\pm$0.06
BotSim-24	Random, $\Delta = 1$	24.30 $\pm$ 0.18	19.75 $\pm$ 0.22	7.10 $\pm$ 0.12	15.60 $\pm$ 0.27
	Random, $\Delta = 3$	21.40 $\pm$ 0.25	18.00 $\pm$ 0.00	12.85 $\pm$ 0.09	14.00 $\pm$ 4.00
	Random, $\Delta = 5$	17.95 $\pm$ 0.30	18.00 $\pm$ 3.46	19.33 $\pm$ 1.15	16.00 $\pm$ 5.29
	Random, $\Delta = 20$	19.33 $\pm$ 1.15	18.67 $\pm$ 1.15	14.00 $\pm$ 6.00	16.67 $\pm$ 3.06
	BOCLOAK, $\Delta \leq 5$	58.88$\pm$0.61	49.21$\pm$0.14	49.88$\pm$0.02	52.67$\pm$13.01

Table 15. Node Injection Attack Against Best Bot Detectors With Adversarial Defenses: Misclassification rate (in %) for successfully injecting fifty target nodes by BOCLOAK against the three best SOTA bot detectors (BotRGCN, GAT, and S-HGN) with adversarial defense. The best performance is shown in bold, and the second best is underlined.

Table 15 highlights a consistent efficiency gap. On BotRGCN (and its defense variants), random injection remains modest at $\Delta \leq 5$ (e.g., ≈ 31 – 37% on Cresci-15/TwiBot-22), but climbs to the mid-90% range only when allowed a large budget of 20 edges. In contrast, BOCLOAK reaches 100% with $\Delta \leq 5$ across all BotRGCN defense variants on both Cresci-15 and TwiBot-22. This is similar for GAT and S-HGN based bot detectors, describing, when the attacker is constrained to realistic edits, BOCLOAK provides a *high-success, low-budget* injection mechanism that random baselines cannot replicate without substantially more connectivity.

BotSim-24 does not saturate at small budgets. In Table 13, injection success increases steadily with budget (from ~ 10 – 16% at $\Delta = 1$ to ~ 50 – 56% at $\Delta = 5$), and only approaches high success at very large budgets for some detectors. Under defended settings (Table 15), BotSim-24 remains the most resistant: BOCLOAK is still far above random at $\Delta \leq 5$, but absolute rates are lower and defenses hurt more than on the other datasets. BotSim-24 has a sharper or sparser “human manifold” for injected neighborhoods, so a few arbitrary edges rarely land in the right region, whereas BOCLOAK’s OT-guided template selection yields meaningful gains that scale with budget.

On the easier benchmarks (Cresci-15/TwiBot-22), the boundary appears relatively easy to cross once a minimally human-like neighborhood is formed, so BOCLOAK reaches ceiling quickly and additional edges give diminishing returns. Meanwhile, when the attacker is allowed to connect broadly (random with $\Delta = 20$), the gap to BOCLOAK can shrink, because brute-force

BotRGCN					
Dataset	Attack	Vanilla (↑)	+GNNGuard (↑)	+GRAND (↑)	+RobustGCN (↑)
Cresci-15	Random, $\Delta = 1$	12.84 $\pm$ 0.13	11.33 $\pm$ 1.15	10.67 $\pm$ 1.15	10.52 $\pm$ 0.27
	Random, $\Delta \leq 3$	22.00 $\pm$ 2.10	21.00 $\pm$ 2.22	22.67 $\pm$ 1.15	21.33 $\pm$ 3.06
	Random, $\Delta \leq 5$	33.33 $\pm$ 2.31	33.67 $\pm$ 1.15	31.11 $\pm$ 0.74	32.67 $\pm$ 2.31
	Random, $\Delta = 20$	96.67 $\pm$ 1.15	95.33 $\pm$ 1.89	95.32 $\pm$ 1.98	94.74 $\pm$ 0.18
	BoCLOAK, $\Delta \leq 5$	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00
TwiBot-22	Random, $\Delta = 1$	13.33 $\pm$ 1.15	11.33 $\pm$ 1.15	16.00 $\pm$ 3.46	10.33 $\pm$ 4.62
	Random, $\Delta \leq 3$	25.33 $\pm$ 4.62	23.33 $\pm$ 2.31	23.67 $\pm$ 2.41	20.33 $\pm$ 1.15
	Random, $\Delta \leq 5$	36.67 $\pm$ 3.06	34.67 $\pm$ 3.06	33.00 $\pm$ 2.89	34.00 $\pm$ 2.22
	Random, $\Delta = 20$	96.67 $\pm$ 3.06	96.67 $\pm$ 1.15	95.33 $\pm$ 4.16	95.33 $\pm$ 1.15
	BoCLOAK, $\Delta \leq 5$	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00
BotSim-24	Random, $\Delta = 1$	10.00 $\pm$ 3.46	15.33 $\pm$ 6.11	11.33 $\pm$ 4.16	19.33 $\pm$ 9.87
	Random, $\Delta \leq 3$	4.00 $\pm$ 2.00	0.67 $\pm$ 1.15	5.33 $\pm$ 1.15	11.23 $\pm$ 5.03
	Random, $\Delta \leq 5$	3.33 $\pm$ 1.15	3.33 $\pm$ 2.31	6.67 $\pm$ 2.31	8.67 $\pm$ 2.31
	Random, $\Delta = 20$	22.00 $\pm$ 6.93	11.33 $\pm$ 7.02	9.33 $\pm$ 3.06	12.67 $\pm$ 4.62
	BoCLOAK, $\Delta \leq 5$	74.43 $\pm$ 1.56	70.11 $\pm$ 2.87	71.44 $\pm$ 2.45	72.66 $\pm$ 1.31
GAT					
Dataset	Attack	Vanilla (↑)	+GNNGuard (↑)	+GRAND (↑)	+RobustGCN (↑)
Cresci-15	Random, $\Delta = 1$	14.00 $\pm$ 2.00	15.33 $\pm$ 3.06	13.00 $\pm$ 3.46	10.67 $\pm$ 2.31
	Random, $\Delta \leq 3$	29.00 $\pm$ 2.00	25.67 $\pm$ 2.31	23.67 $\pm$ 1.15	29.33 $\pm$ 1.15
	Random, $\Delta \leq 5$	35.33 $\pm$ 4.16	35.03 $\pm$ 1.15	30.00 $\pm$ 2.00	31.33 $\pm$ 4.16
	Random, $\Delta = 20$	97.33 $\pm$ 1.15	96.67 $\pm$ 3.06	94.00 $\pm$ 6.93	92.00 $\pm$ 3.46
	BoCLOAK, $\Delta \leq 5$	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00
TwiBot-22	Random, $\Delta = 1$	19.03 $\pm$ 2.00	15.33 $\pm$ 3.06	16.00 $\pm$ 3.46	10.67 $\pm$ 2.31
	Random, $\Delta \leq 3$	25.00 $\pm$ 2.00	25.67 $\pm$ 2.31	24.67 $\pm$ 1.15	23.33 $\pm$ 1.15
	Random, $\Delta \leq 5$	36.33 $\pm$ 4.16	35.33 $\pm$ 1.15	36.00 $\pm$ 2.00	31.33 $\pm$ 4.16
	Random, $\Delta = 20$	92.33 $\pm$ 1.15	86.67 $\pm$ 3.06	89.00 $\pm$ 6.93	83.00 $\pm$ 3.46
	BoCLOAK, $\Delta \leq 5$	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00
BotSim-24	Random, $\Delta = 1$	11.13 $\pm$ 6.11	10.67 $\pm$ 4.62	17.31 $\pm$ 5.03	11.13 $\pm$ 5.03
	Random, $\Delta \leq 3$	16.67 $\pm$ 6.11	8.67 $\pm$ 3.06	24.00 $\pm$ 2.00	10.00 $\pm$ 0.00
	Random, $\Delta \leq 5$	12.00 $\pm$ 8.72	5.33 $\pm$ 4.16	25.33 $\pm$ 5.03	4.67 $\pm$ 1.15
	Random, $\Delta = 20$	14.67 $\pm$ 6.43	32.00 $\pm$ 5.29	8.00 $\pm$ 5.29	6.67 $\pm$ 6.43
	BoCLOAK, $\Delta \leq 5$	77.56 $\pm$ 3.30	75.00 $\pm$ 1.50	73.44 $\pm$ 2.30	71.32 $\pm$ 2.11
S-HGN					
Dataset	Attack	Vanilla (↑)	+GNNGuard (↑)	+GRAND (↑)	+RobustGCN (↑)
Cresci-15	Random, $\Delta = 1$	14.33 $\pm$ 1.15	14.40 $\pm$ 0.12	15.85 $\pm$ 0.09	10.23 $\pm$ 0.10
	Random, $\Delta = 3$	27.10 $\pm$ 0.11	26.75 $\pm$ 0.07	25.20 $\pm$ 0.13	27.93 $\pm$ 3.15
	Random, $\Delta = 5$	36.65 $\pm$ 0.14	33.90 $\pm$ 0.10	36.33 $\pm$ 1.15	32.33 $\pm$ 8.17
	Random, $\Delta = 20$	99.33 $\pm$ 1.15	92.80 $\pm$ 0.08	99.33 $\pm$ 1.15	98.00 $\pm$ 3.46
	BoCLOAK, $\Delta \leq 5$	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00
TwiBot-22	Random, $\Delta = 1$	15.33 $\pm$ 3.06	12.27 $\pm$ 1.15	12.00 $\pm$ 3.46	10.67 $\pm$ 9.24
	Random, $\Delta = 3$	24.00 $\pm$ 2.00	21.67 $\pm$ 2.31	22.67 $\pm$ 4.16	24.77 $\pm$ 2.31
	Random, $\Delta = 5$	38.00 $\pm$ 0.00	33.67 $\pm$ 5.31	36.67 $\pm$ 1.15	37.33 $\pm$ 6.11
	Random, $\Delta = 20$	98.67 $\pm$ 2.31	98.67 $\pm$ 2.31	92.67 $\pm$ 2.31	91.33 $\pm$ 6.11
	BoCLOAK, $\Delta \leq 5$	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00
BotSim-24	Random, $\Delta = 1$	24.30 $\pm$ 0.18	19.75 $\pm$ 0.22	7.10 $\pm$ 0.12	15.60 $\pm$ 0.27
	Random, $\Delta = 3$	21.40 $\pm$ 0.25	18.00 $\pm$ 0.00	12.85 $\pm$ 0.09	14.00 $\pm$ 4.00
	Random, $\Delta = 5$	17.95 $\pm$ 0.30	18.00 $\pm$ 3.46	19.33 $\pm$ 1.15	16.00 $\pm$ 5.29
	Random, $\Delta = 20$	19.33 $\pm$ 1.15	18.67 $\pm$ 1.15	14.00 $\pm$ 0.00	16.67 $\pm$ 3.06
	BoCLOAK, $\Delta \leq 5$	76.11 $\pm$ 2.11	72.01 $\pm$ 2.87	72.23 $\pm$ 1.11	74.00 $\pm$ 1.10

BOCLOAK : Optimal Transport-Guided Adversarial Attacks on Graph Neural Network-Based Bot Detection

Dataset	Attack	BotRGCN			
		Vanilla (†)	+GNNGuard (†)	+GRAND (†)	+RobustGCN (†)
$\Delta = 1$					
Cresci-15	Nettack	4.32 $\pm$ 2.10	3.78 $\pm$ 1.95	3.21 $\pm$ 1.88	2.67 $\pm$ 1.72
	FGA	3.95 $\pm$ 2.44	3.40 $\pm$ 2.12	2.98 $\pm$ 1.76	2.15 $\pm$ 1.60
	PR-BCD	4.10 $\pm$ 3.05	3.60 $\pm$ 2.80	2.85 $\pm$ 2.20	1.90 $\pm$ 1.45
	GOTack	3.55 $\pm$ 2.34	3.41 $\pm$ 2.52	2.28 $\pm$ 1.22	2.66 $\pm$ 1.67
	BoCLOAK	99.34 $\pm$ 0.29	98.91 $\pm$ 0.54	99.06 $\pm$ 0.41	99.72 $\pm$ 0.18
TwiBot-22	Nettack	9.33 $\pm$ 3.06	8.67 $\pm$ 4.62	4.67 $\pm$ 5.03	8.00 $\pm$ 5.29
	FGA	4.00 $\pm$ 2.00	7.33 $\pm$ 2.31	6.67 $\pm$ 4.16	1.33 $\pm$ 6.43
	PR-BCD	8.67 $\pm$ 2.31	8.67 $\pm$ 5.03	6.23 $\pm$ 4.16	7.33 $\pm$ 2.31
	GOTack	6.89 $\pm$ 3.33	7.15 $\pm$ 4.25	6.01 $\pm$ 2.48	5.32 $\pm$ 2.58
	BoCLOAK	86.67 $\pm$ 2.31	84.67 $\pm$ 11.02	83.67 $\pm$ 3.06	85.30 $\pm$ 2.00
BotSim-24	Nettack	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	3.00 $\pm$ 2.00
	FGA	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00
	PR-BCD	2.00 $\pm$ 2.00	5.33 $\pm$ 2.31	0.67 $\pm$ 1.15	0.00 $\pm$ 0.00
	GOTack	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00
	BoCLOAK	58.22 $\pm$ 7.20	52.68 $\pm$ 1.15	54.98 $\pm$ 2.45	55.10 $\pm$ 3.98
$\Delta = 3$					
Cresci-15	Nettack	13.98 $\pm$ 10.50	12.10 $\pm$ 1.70	12.98 $\pm$ 1.10	11.10 $\pm$ 1.74
	FGA	12.65 $\pm$ 3.20	10.95 $\pm$ 2.85	9.10 $\pm$ 2.31	7.60 $\pm$ 1.68
	PR-BCD	11.98 $\pm$ 2.10	10.40 $\pm$ 1.92	8.90 $\pm$ 1.70	9.35 $\pm$ 1.46
	GOTack	12.88 $\pm$ 2.48	11.10 $\pm$ 2.20	9.60 $\pm$ 1.84	9.70 $\pm$ 1.50
	BoCLOAK	99.18 $\pm$ 0.37	98.47 $\pm$ 0.96	99.59 $\pm$ 0.21	99.01 $\pm$ 0.44
TwiBot-22	Nettack	10.22 $\pm$ 2.83	11.78 $\pm$ 1.41	5.10 $\pm$ 1.41	5.01 $\pm$ 4.24
	FGA	8.15 $\pm$ 2.83	8.66 $\pm$ 2.83	9.87 $\pm$ 7.07	2.10 $\pm$ 5.66
	PR-BCD	15.10 $\pm$ 7.07	16.50 $\pm$ 2.83	10.00 $\pm$ 5.66	4.64 $\pm$ 2.83
	GOTack	3.12 $\pm$ 1.11	8.48 $\pm$ 3.65	5.17 $\pm$ 2.59	5.68 $\pm$ 5.28
	BoCLOAK	94.50 $\pm$ 2.00	86.20 $\pm$ 4.59	84.67 $\pm$ 3.06	87.33 $\pm$ 1.15
BotSim-24	Nettack	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	5.23 $\pm$ 0.01
	FGA	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00
	PR-BCD	1.00 $\pm$ 1.41	4.00 $\pm$ 0.00	2.00 $\pm$ 0.00	0.00 $\pm$ 0.00
	GOTack	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00
	BoCLOAK	93.33 $\pm$ 1.15	66.74 $\pm$ 3.45	88.63 $\pm$ 0.87	64.32 $\pm$ 3.33
$\Delta = 5$					
Cresci-15	Nettack	22.35 $\pm$ 3.18	20.80 $\pm$ 2.85	18.95 $\pm$ 2.30	16.40 $\pm$ 1.62
	FGA	21.90 $\pm$ 2.74	20.15 $\pm$ 2.40	18.40 $\pm$ 2.05	15.95 $\pm$ 1.48
	PR-BCD	20.75 $\pm$ 1.96	19.30 $\pm$ 1.74	17.85 $\pm$ 1.60	15.40 $\pm$ 1.36
	GOTack	22.50 $\pm$ 3.55	21.05 $\pm$ 3.10	19.10 $\pm$ 2.65	16.85 $\pm$ 1.70
	BoCLOAK	99.67 $\pm$ 0.20	98.88 $\pm$ 0.58	99.11 $\pm$ 0.39	99.52 $\pm$ 0.24
TwiBot-22	Nettack	14.22 $\pm$ 2.03	14.10 $\pm$ 0.00	5.77 $\pm$ 4.24	7.78 $\pm$ 4.24
	FGA	11.30 $\pm$ 7.07	10.80 $\pm$ 5.66	6.21 $\pm$ 2.83	9.22 $\pm$ 12.73
	PR-BCD	24.73 $\pm$ 5.66	21.53 $\pm$ 7.07	20.45 $\pm$ 5.66	19.00 $\pm$ 2.83
	GOTack	10.58 $\pm$ 2.15	18.25 $\pm$ 3.65	12.89 $\pm$ 1.02	13.32 $\pm$ 3.03
	BoCLOAK	94.00 $\pm$ 2.00	90.00 $\pm$ 4.82	93.67 $\pm$ 3.06	88.33 $\pm$ 1.15
BotSim-24	Nettack	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00
	FGA	0.00 $\pm$ 0.00	0.67 $\pm$ 1.15	0.00 $\pm$ 0.00	2.67 $\pm$ 1.15
	PR-BCD	2.67 $\pm$ 1.15	4.00 $\pm$ 2.83	1.00 $\pm$ 1.41	0.00 $\pm$ 0.00
	GOTack	2.00 $\pm$ 0.00	2.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00
	BoCLOAK	99.33 $\pm$ 1.15	79.21 $\pm$ 1.02	92.28 $\pm$ 0.33	66.20 $\pm$ 3.50

Table 17. **Node Editing Attack:** Misclassification rate (in %) for flipping fifty existing correctly classified bots by BOCLOAK and SOTA adversarial attacks against best SOTA bot detector, BotRGCN with adversarial defenses. For results against other bot detectors, refer to Table 12, and for results when no domain constraints are enforced, refer to Table 19. The best performance is shown in bold, and the second best is underlined.

Dataset	Attack	BotRGCN			
		Vanilla (↑)	+GNNGuard (↑)	+GRAND (↑)	+RobustGCN (↑)
Δ = 1					
Cresci-15	Nettack	4.32±2.10	3.78±1.95	3.21±1.88	2.67±1.72
	FGA	3.95±2.44	3.40±2.12	2.98±1.76	2.15±1.60
	PR-BCD	4.10±3.05	3.60±2.80	2.85±2.20	1.90±1.45
	GOTack	3.55±2.34	3.41±2.52	2.28±1.22	2.66±1.67
	BoCLOAK	100.00±0.00	100.00±0.00	100.00±0.00	100.00±0.00
TwiBot-22	Nettack	9.33±3.06	8.67±4.62	4.67±5.03	8.00±5.29
	FGA	4.00±2.00	7.33±2.31	6.67±4.16	1.33±6.43
	PR-BCD	8.67±2.31	8.67±5.03	6.23±4.16	7.33±2.31
	GOTack	6.89±3.33	7.15±4.25	6.01±2.48	5.32±2.58
	BoCLOAK	86.67±2.31	88.67±11.02	88.67±3.06	52.12±14.05
BotSim-24	Nettack	0.00±0.00	0.00±0.00	0.00±0.00	3.00±2.00
	FGA	0.00±0.00	0.00±0.00	0.00±0.00	0.00±0.00
	PR-BCD	2.00±2.00	5.33±2.31	0.67±1.15	0.00±0.00
	GOTack	0.00±0.00	0.00±0.00	0.00±0.00	0.00±0.00
	BoCLOAK	0.00±0.00	0.00±0.00	0.00±0.00	0.00±0.00
Δ = 3					
Cresci-15	Nettack	13.98±10.50	12.10±1.70	12.98±1.10	11.10±1.74
	FGA	12.65±3.20	10.95±2.85	9.10±2.31	7.60±1.68
	PR-BCD	11.98±2.10	10.40±1.92	8.90±1.70	9.35±1.46
	GOTack	12.88±2.48	11.10±2.20	9.60±1.84	9.70±1.50
	BoCLOAK	100.00±0.00	100.00±0.00	100.00±0.00	100.00±0.00
TwiBot-22	Nettack	10.00±2.83	11.00±1.41	5.00±1.41	5.00±4.24
	FGA	8.00±2.83	8.00±2.83	9.00±7.07	2.00±5.66
	PR-BCD	15.00±7.07	16.00±2.83	10.00±5.66	4.00±2.83
	GOTack	3.12±1.11	8.48±3.65	5.17±2.59	5.68±5.28
	BoCLOAK	94.00±2.00	96.00±4.00	94.67±3.06	99.33±1.15
BotSim-24	Nettack	0.00±0.00	0.00±0.00	0.00±0.00	5.23±0.01
	FGA	0.00±0.00	0.00±0.00	0.00±0.00	0.00±0.00
	PR-BCD	1.00±1.41	4.00±0.00	2.00±0.00	0.00±0.00
	GOTack	0.00±0.00	0.00±0.00	0.00±0.00	0.00±0.00
	BoCLOAK	99.33±1.15	74.50±2.50	100.00±0.00	74.12±1.50
Δ = 5					
Cresci-15	Nettack	22.35±3.18	20.80±2.85	18.95±2.30	16.40±1.62
	FGA	21.90±2.74	20.15±2.40	18.40±2.05	15.95±1.48
	PR-BCD	20.75±1.96	19.30±1.74	17.85±1.60	15.40±1.36
	GOTack	22.50±3.55	21.05±3.10	19.10±2.65	16.85±1.70
	BoCLOAK	100.00±0.00	100.00±0.00	100.00±0.00	100.00±0.00
TwiBot-22	Nettack	14.22±2.03	14.10±0.00	5.77±4.24	7.78±4.24
	FGA	11.30±7.07	10.80±5.66	6.21±2.83	9.22±12.73
	PR-BCD	24.73±5.66	21.53±7.07	20.45±5.66	19.00±2.83
	GOTack	10.58±2.15	18.25±3.65	12.89±1.02	13.32±3.03
	BoCLOAK	94.00±2.00	96.00±4.00	94.67±3.06	99.33±1.15
BotSim-24	Nettack	0.00±0.00	0.00±0.00	0.00±0.00	0.00±0.00
	FGA	0.00±0.00	0.67±1.15	0.00±0.00	2.67±1.15
	PR-BCD	2.67±1.15	4.00±2.83	1.00±1.41	0.00±0.00
	GOTack	2.00±0.00	2.00±0.00	0.00±0.00	0.00±0.00
	BoCLOAK	99.33±1.15	95.21±5.02	100.00±0.00	74.00±1.50

Table 18. **Node Editing Attack Reusing Same Bot Cloak:** Misclassification rate (in %) for flipping fifty existing correctly classified bots by BOCLOAK and SOTA adversarial attacks against best SOTA bot detector, BotRGCN with adversarial defenses and *reusing* the same bot cloak. The best performance is shown in bold, and the second best is underlined.

linking can eventually approximate the needed neighborhood structure. On BotSim-24, random remains weak even at large budgets in defended settings, indicating that naive linking often fails to reach the human region; here, OT-guided selection provides the largest advantage by systematically steering injections toward plausible, boundary-adjacent neighborhoods.

P. Bot Editing and Injection Results When the Best Bot Template is Reused Every Time

Table 14, Table 16, and Table 18 illustrate the experimentation results when the best bot cloak template is selected every time for node editing and injection. The template-reuse setting effectively approximates an attacker who *cherry-picks* the most human-like bot and mass-produces clones of its neighborhood signature. However, this comes with an important realism caveat: reuse creates a single point of failure. If the best template (or its characteristic neighborhood “fingerprint”) is ever detected, the same signature can be blacklisted, causing all its clones (and any targets edited toward it) to become detectable as well.

Dataset	Attack	BotRGCN			
		Vanilla ($\uparrow$)	+GNNGuard ($\uparrow$)	+GRAND ($\uparrow$)	+RobustGCN ($\uparrow$)
$\Delta = 1$					
Cresci-15	Nettack	94.67 $\pm$ 6.55	90.10 $\pm$ 9.22	90.30 $\pm$ 3.75	92.58 $\pm$ 1.05
	FGA	96.89 $\pm$ 4.81	92.21 $\pm$ 3.48	93.75 $\pm$ 5.64	91.44 $\pm$ 10.00
	PR-BCD	96.36 $\pm$ 8.26	93.15 $\pm$ 8.06	92.91 $\pm$ 1.59	93.16 $\pm$ 0.69
	Gottack	98.54 $\pm$ 12.40	96.57 $\pm$ 8.98	94.04 $\pm$ 1.68	93.79 $\pm$ 1.67
	BoCLOAK (ours)	99.34 $\pm$ 0.29	98.91 $\pm$ 0.54	99.06 $\pm$ 0.41	99.72 $\pm$ 0.18
TwiBot-22	Nettack	79.77 $\pm$ 6.77	41.72 $\pm$ 3.23	68.68 $\pm$ 18.75	84.54 $\pm$ 22.74
	FGA	71.90 $\pm$ 4.23	47.03 $\pm$ 4.80	80.60 $\pm$ 4.92	74.40 $\pm$ 10.08
	PR-BCD	75.77 $\pm$ 6.05	49.69 $\pm$ 19.23	73.99 $\pm$ 14.54	61.81 $\pm$ 7.41
	Gottack	84.29 $\pm$ 11.63	54.90 $\pm$ 10.57	80.73 $\pm$ 8.34	78.42 $\pm$ 13.55
	BoCLOAK (ours)	86.67 $\pm$ 2.31	84.67 $\pm$ 11.02	83.67 $\pm$ 3.06	85.30 $\pm$ 2.00
BotSim-24	Nettack	56.57 $\pm$ 6.63	51.22 $\pm$ 7.08	55.36 $\pm$ 4.80	52.57 $\pm$ 1.70
	FGA	55.67 $\pm$ 6.96	52.35 $\pm$ 13.91	55.95 $\pm$ 3.67	58.04 $\pm$ 25.09
	PR-BCD	50.62 $\pm$ 8.04	53.19 $\pm$ 8.01	53.53 $\pm$ 4.46	52.71 $\pm$ 3.69
	Gottack	57.18 $\pm$ 2.32	50.48 $\pm$ 12.18	58.43 $\pm$ 5.58	56.50 $\pm$ 10.64
	BoCLOAK (ours)	58.22 $\pm$ 7.20	52.68 $\pm$ 1.15	54.98 $\pm$ 2.45	55.10 $\pm$ 3.98
$\Delta = 5$					
Cresci-15	Nettack	99.45 $\pm$ 9.40	98.02 $\pm$ 12.41	95.78 $\pm$ 7.20	95.45 $\pm$ 17.53
	FGA	97.56 $\pm$ 12.43	80.11 $\pm$ 9.65	94.65 $\pm$ 3.63	94.88 $\pm$ 12.32
	PR-BCD	98.65 $\pm$ 5.25	96.12 $\pm$ 15.73	94.55 $\pm$ 14.93	92.54 $\pm$ 6.27
	Gottack	99.33 $\pm$ 17.24	99.58 $\pm$ 3.58	90.98 $\pm$ 5.49	90.36 $\pm$ 7.36
	BoCLOAK (ours)	99.67 $\pm$ 0.20	98.88 $\pm$ 0.58	99.11 $\pm$ 0.39	99.52 $\pm$ 0.24
TwiBot-22	Nettack	94.15 $\pm$ 1.05	58.54 $\pm$ 8.34	86.73 $\pm$ 21.07	96.78 $\pm$ 19.05
	FGA	96.13 $\pm$ 7.15	60.52 $\pm$ 8.20	92.42 $\pm$ 1.48	88.58 $\pm$ 5.33
	PR-BCD	94.08 $\pm$ 2.68	66.14 $\pm$ 21.72	89.85 $\pm$ 12.04	87.45 $\pm$ 4.14
	Gottack	99.55 $\pm$ 6.16	75.96 $\pm$ 7.77	93.53 $\pm$ 4.10	92.52 $\pm$ 18.37
	BoCLOAK (ours)	94.00 $\pm$ 2.00	90.00 $\pm$ 4.00	93.67 $\pm$ 3.06	88.33 $\pm$ 1.15
BotSim-24	Nettack	97.78 $\pm$ 8.96	74.79 $\pm$ 2.64	78.58 $\pm$ 7.48	89.08 $\pm$ 4.04
	FGA	98.23 $\pm$ 2.98	82.77 $\pm$ 8.82	88.12 $\pm$ 7.35	84.02 $\pm$ 20.30
	PR-BCD	92.45 $\pm$ 3.58	85.63 $\pm$ 10.46	82.61 $\pm$ 1.42	85.12 $\pm$ 6.51
	Gottack	96.86 $\pm$ 7.70	82.87 $\pm$ 15.75	89.54 $\pm$ 3.56	90.72 $\pm$ 4.93
	BoCLOAK (ours)	99.33 $\pm$ 1.15	89.21 $\pm$ 1.02	92.28 $\pm$ 0.33	44.00 $\pm$ 19.16

Table 19. **Node Editing Attack Without Domain Constraints:** Misclassification rate (in %) for flipping fifty existing correctly classified bots by BoCLOAK and SOTA adversarial attacks with budget Δ of five against best SOTA bot detector, BotRGCN with adversarial defenses and *without* enforcing domain constraints on attacks. For results when domain constraints are enforced, refer to Table 17 and for results when the same bot cloak is reused, refer to Table 18. The best performance is shown in bold, and the second best is underlined.

Because the reused template is intentionally chosen from the most human-like region of the detector’s decision boundary, it acts as a highly transferable cloak: once we decode its neighborhood structure into a feasible set of edits, the resulting perturbations repeatedly push many different targets into the human region. This effect is especially pronounced on Cresci-15 and TwiBot-22, where injection becomes essentially solved in this setting: Table 14 shows 100% misclassification across all vanilla detectors even at small budgets, and Table 16 shows the same near-perfect behavior persists against the best detectors equipped with adversarial defenses.

A similar trend holds for node editing when the cloak is reused: Table 18 indicates that selecting a single strong cloak can sustain very high flip rates, often remaining best even under standard defenses, consistent with the idea that BoCLOAK learns a transferable “bot cloak” rather than a target-specific perturbation.

Unlike Cresci-15/TwiBot-22, BotSim-24 does *not* saturate at tiny budgets when reusing one template. In Table 14, BotSim-24 injection rises gradually with budget (from low success at $\Delta=1$ to higher success at larger budgets), and only reaches ceiling at very large budget for some detectors. For node editing, Table 18 similarly reveals that while reuse is generally powerful, some defense combinations can still suppress performance (an example is a pronounced drop under RobustGCN in at least one configuration).

Q. Bot Editing Results When No Domain Constraints are Enforced

Table 19 reports node-editing performance when we *remove* all domain constraints and allow SOTA attacks to freely rewire incident edges around the target bot under the same per-subgraph budgets. This setting is intentionally unrealistic for social platforms, but it is useful as it reveals how much of the previously reported *attack strength* is driven by infeasible edits. In

Attack	Unconstrained	Constrained
FGA	71.90 $\pm$ 4.23	4.00 $\pm$ 2.00
FGA-mod	70.44 $\pm$ 1.34	7.11 $\pm$ 2.23
PR-BCD	75.77 $\pm$ 6.05	8.67 $\pm$ 2.31
PR-BCD-mod	74.93 $\pm$ 5.90	13.21 $\pm$ 1.45
LR-BCD	82.33 $\pm$ 2.13	9.86 $\pm$ 4.65
Homophily	60.11 $\pm$ 3.69	13.12 $\pm$ 1.01
BOCLOAK (ours)	86.67$\pm$2.31	86.67$\pm$2.31

Table 20. **Control study on TwiBot-22/BotRGCN for node editing with $\Delta = 1$:** FGA-mod and PR-BCD-mod add in-loop projection onto the admissible bot $\rightarrow$ human edit set; Homophily replaces OT-guided coupling with embedding/homophily matching while keeping the same feature set and local context.

our threat model, SOTA attacks collapse to single-digit/low-teen flip rates once we enforce social-domain constraints, which the main paper highlights as a core realism gap in prior graph-attack evaluations.

Across all datasets and defenses, unconstrained variants of adversarial attacks become highly effective, often approaching ceiling misclassification. Suggesting that if an attacker could violate domain rules, the detector is extremely brittle. This aligns with the paper’s observation that unconstrained attacks can appear strong, but their effectiveness does not survive once feasibility constraints are imposed.

While unconstrained SOTA attacks inflate substantially, BOCLOAK stays consistently high and is frequently the top performer across datasets/defenses. On BotSim-24 at $\Delta=1$, BOCLOAK is best on vanilla and competitive elsewhere (58.22% on vanilla), indicating that our cloak-transfer mechanism is not relying on *illegal* rewiring to be effective; it remains potent even when we *allow* that extra flexibility. The unconstrained setting can make SOTA attacks seem overwhelmingly strong, but this success is largely driven by edits that are not feasible in real social systems. In contrast, BOCLOAK is designed around domain-faithful constraints, yet still remains highly effective even when those constraints are lifted and, crucially, the gap between constrained vs. unconstrained SOTA performance underscores why evaluations that ignore domain rules can substantially mischaracterize real-world robustness.

R. Constrained Baseline Controls

Table 20 reports additional controls to showcase that the constrained baseline comparison is fair. All attacks are evaluated on TwiBot-22 against BotRGCN in the node-editing setting with budget $\Delta = 1$, using the same target set and the same constrained feasible-edit set as the main experiments.

The mod variants test whether stronger constraint integration inside the baseline optimization closes the gap to BOCLOAK. Specifically, FGA-mod and PR-BCD-mod add an in-loop projection step so that candidate perturbations are restricted to admissible bot $\rightarrow$ human edits before they are committed. This improves the constrained baselines modestly: FGA increases from 4.00% to 7.11%, and PR-BCD increases from 8.67% to 13.21%. However, both remain far below BOCLOAK, which achieves 86.67% under the same budget and feasibility rules.

We also include LR-BCD as a newer local-budget graph attack. Although LR-BCD is stronger than PR-BCD in the unconstrained setting, its constrained performance remains low because it is still not designed around social-bot feasibility rules such as target-incident-only edits, no unrelated rewiring, temporal plausibility, and no forced human $\rightarrow$ bot follow-backs.

Finally, the Homophily control tests whether the observed gain can be explained by simple neighborhood similarity or embedding matching. Although the Homophily baseline performs better than several constrained generic attacks, it still reaches only 13.12% in the constrained setting. This supports our claim that OT is useful not merely as a scalar similarity score, but because its transport plan identifies which local neighbor correspondences should be decoded into sparse, feasible edits.

S. Ablation Study

Table 21 describes the ablation study of BOCLOAK against BotRGCN with adversarial defenses on three bot datasets, with a budget Δ of 5. We measure the impact of the different loss (e.g., L_{BCE} , L_{sp} , L_{pl}) in BOCLOAK’s performance. We selectively deactivate each loss by setting λ_{BCE} , λ_{sp} , λ_{pl} to zero. We also measure the impact of node degree and node age on L_{pl} by setting α_{deg} , α_{age} to zero.

Loss Component	BotRGCN			
	Vanilla ($\uparrow$)	+GNNGuard ($\uparrow$)	+GRAND ($\uparrow$)	+RobustGCN ($\uparrow$)
Cresci-15				
$-\lambda_{\text{BCE}}$	38.21 $\pm$ 5.12	32.98 $\pm$ 0.22	33.34 $\pm$ 2.21	35.76 $\pm$ 1.80
$-\lambda_{\text{sp}}$	96.88 $\pm$ 5.89	95.12 $\pm$ 5.41	95.34 $\pm$ 7.13	94.23 $\pm$ 0.71
$-\lambda_{\text{pl}}$	57.99 $\pm$ 3.38	52.67 $\pm$ 0.26	50.71 $\pm$ 1.76	55.45 $\pm$ 4.05
$-\alpha_{\text{deg}}$	73.23 $\pm$ 0.23	70.23 $\pm$ 1.60	71.11 $\pm$ 5.20	68.54 $\pm$ 4.36
$-\alpha_{\text{age}}$	87.23 $\pm$ 1.78	85.77 $\pm$ 4.72	84.82 $\pm$ 6.47	86.76 $\pm$ 0.07
BoCLOAK	99.67$\pm$0.20	98.88$\pm$0.58	99.11$\pm$0.39	99.52$\pm$0.24
Twibot-22				
$-\lambda_{\text{BCE}}$	22.45 $\pm$ 6.44	20.12 $\pm$ 5.58	17.23 $\pm$ 2.73	19.87 $\pm$ 1.26
$-\lambda_{\text{sp}}$	92.12 $\pm$ 7.65	87.34 $\pm$ 2.70	91.22 $\pm$ 0.76	84.12 $\pm$ 0.79
$-\lambda_{\text{pl}}$	55.23 $\pm$ 6.77	50.12 $\pm$ 4.83	52.55 $\pm$ 6.45	52.98 $\pm$ 5.84
$-\alpha_{\text{deg}}$	70.98 $\pm$ 4.29	68.12 $\pm$ 7.78	65.22 $\pm$ 3.04	68.79 $\pm$ 4.42
$-\alpha_{\text{age}}$	83.88 $\pm$ 6.63	81.54 $\pm$ 4.95	80.02 $\pm$ 6.89	78.35 $\pm$ 4.62
BoCLOAK	94.00$\pm$2.00	90.00$\pm$4.00	93.67$\pm$3.06	88.33$\pm$1.15
BotSim-24				
$-\lambda_{\text{BCE}}$	32.45 $\pm$ 5.64	25.34 $\pm$ 0.39	29.43 $\pm$ 1.84	10.11 $\pm$ 2.33
$-\lambda_{\text{sp}}$	96.55 $\pm$ 0.66	85.73 $\pm$ 1.88	90.21 $\pm$ 0.82	38.45 $\pm$ 2.24
$-\lambda_{\text{pl}}$	60.34 $\pm$ 5.09	58.17 $\pm$ 2.93	53.13 $\pm$ 2.97	25.33 $\pm$ 1.69
$-\alpha_{\text{deg}}$	69.22 $\pm$ 2.15	65.23 $\pm$ 7.49	64.67 $\pm$ 5.18	32.32 $\pm$ 4.87
$-\alpha_{\text{age}}$	85.34 $\pm$ 1.38	83.78 $\pm$ 5.83	82.44 $\pm$ 1.32	37.23 $\pm$ 3.04
BoCLOAK	99.33$\pm$1.15	79.21$\pm$1.02	92.28$\pm$0.33	66.20$\pm$3.50

Table 21. **Ablation Study:** Ablation results for misclassification rate (in %) for node editing. BoCLOAK results against BotRGCN with standard adversarial defenses with budget Δ of 5. The best performance is shown in bold, and the second best is underlined.

The OT-margin BCE surrogate is the key driver: setting λ_{BCE} to zero causes a sharp collapse in flip rates across all datasets and defense combinations, consistent with its role in explicitly shaping the OT margin so boundary/misclassified templates remain near (or inside) the human region in OT space. The plausibility loss is the next most important, dropping λ_{pl} yields only mid-range success, because it discourages transport mass on socially implausible neighbor matches (*e.g.*, large degree/age gaps), which otherwise produces unrealistic alignments that do not transfer into effective edge edits. Finally, within plausibility, degree alignment matters more than age alignment (ablating α_{deg} deteriorates more than ablating α_{age}), indicating that preserving local connectivity is a stronger constraint on realistic template transfer than temporal calibration alone. This is reflected in the real world, where new accounts tend to follow trusted old accounts, but new accounts do not always have diverse interests, so their follow accounts differ considerably.

Hyperparameter	Cresci-15	Twibot-22	BotSim-24
OT Regularizer(ϵ)			
$\epsilon = 0.01$	98.11 $\pm$ 4.23	84.00 $\pm$ 6.00	84.00 $\pm$ 1.56
$\epsilon = 0.1$	<u>99.30$\pm$2.36</u>	90.67 $\pm$ 3.06	<u>98.00$\pm$3.88</u>
$\epsilon = 1$	53.33 $\pm$ 17.67	<u>92.67$\pm$2.31</u>	18.00 $\pm$ 3.03
$\epsilon = 10$	33.33 $\pm$ 5.66	32.00 $\pm$ 2.00	22.65 $\pm$ 2.25
$\epsilon = 100$	13.83 $\pm$ 5.57	22.00 $\pm$ 1.03	18.23 $\pm$ 1.20
BoCLOAK ($\epsilon = 0.2$)	99.67$\pm$0.20	94.00$\pm$2.00	99.33$\pm$1.15
BCE Threshold (τ_{bdry})			
$\tau_{\text{bdry}} = 0.00$	92.50 $\pm$ 2.50	85.33 $\pm$ 1.55	83.05 $\pm$ 1.33
$\tau_{\text{bdry}} = 0.05$	97.00 $\pm$ 2.00	91.50 $\pm$ 1.00	97.20 $\pm$ 1.15
$\tau_{\text{bdry}} = 0.15$	<u>99.05$\pm$1.50</u>	93.25 $\pm$ 1.25	<u>97.33$\pm$1.50</u>
$\tau_{\text{bdry}} = 0.30$	98.89 $\pm$ 1.07	<u>93.67$\pm$1.76</u>	95.15 $\pm$ 1.33
$\tau_{\text{bdry}} = 0.50$	98.15 $\pm$ 1.67	90.11 $\pm$ 2.50	90.67 $\pm$ 1.33
BoCLOAK ($\tau_{\text{bdry}} = 0.10$)	99.67$\pm$0.20	94.00$\pm$2.00	99.33$\pm$1.15

Table 22. **Sensitivity Study:** Misclassification rate (in %) for flipping fifty existing correctly classified bots as the OT regularizer (ϵ) and BCE Threshold (τ_{bdry}) is varied, impacting the human-bot decision boundary and the strength of the OT margin constraint required for a successful cloak. BoCLOAK results against vanilla BotRGCN with budget Δ of 5. Best is highlighted and second best is underlined within each block.

T. Sensitivity Study

Table 22 describes the sensitivity of BOCLOAK against vanilla BotRGCN on three bot datasets with budget Δ of 5. We measure the impact of the entropic regularizer, ε that encourages OT decision boundary plans. It controls the plan sharpness: smaller ε yields near-permutation matchings, whereas larger ε produces more diffuse couplings. We also vary the margin threshold (τ_{bdry}), which sets the required OT margin for a successful cloak: increasing τ_{bdry} imposes a stronger constraint (requiring bots to be closer to human templates by a larger margin), which can improve boundary crossing up to a point but may eventually over-constrain optimization.

When ε is too small (e.g., 0.01), small neighborhood mismatches can cause unstable, high-variance boundary behavior despite occasional strong flips, yielding only 83-85% flip rates on TwiBot-22 dataset against BotRGCN, transport remains sharp and diffuse enough to capture multiple plausible neighbor correspondences, producing the strongest and most consistent boundary crossings, with near-ceiling 98-100% flip rates on Cresci-15 dataset against BotRGCN. For large ε (≥ 10), the plan washes out toward diffuseness, blurring class structure in the OT-induced margin, so flip rates collapse (e.g., 53% to 14% on Cresci-15 as ε increases 1 to 100), indicating a weakened ability to target the boundary.

Across τ_{bdry} , Cresci-15 benefits from moderate margins: moving from $\tau_{\text{bdry}}=0.00$ to 0.10 lifts flips from 92.50% to 99% while tightening variance, suggesting reduced “lucky flips” and more consistent boundary crossings. In contrast, TwiBot-22 and BotSim-24 exhibit diminishing returns at large margins: after peaking around $\tau_{\text{bdry}}=0.10$, further increasing τ_{bdry} to 0.30-0.50 reduces success (TwiBot-22 down to 90.11%, BotSim-24 down to 90.67%) and can increase each trail variability, consistent with a harder constraint that leaves optimization persistently penalized and yields plateauing or dropping flip rates.

System Resource ($\downarrow$)	Attack	Cresci-15	TwiBot-22	BotSim-24
Time (sec)	Random	0.01 $\pm$ 0.01	0.18 $\pm$ 0.02	0.03 $\pm$ 0.01
	Nettack	20.3 $\pm$ 0.22	1.4 $\pm$ 1.33	19.7 $\pm$ 1.65
	FGA	0.3 $\pm$ 0.14	1.1 $\pm$ 0.06	0.3 $\pm$ 1.57
	PR-BCD	1.4 $\pm$ 0.13	2.9 $\pm$ 0.47	2.1 $\pm$ 0.75
	Gottack	1.9 $\pm$ 3.56	853.9 $\pm$ 19.95	7.5 $\pm$ 1.75
	BOCLOAK (ours)	0.1$\pm$0.13	0.5$\pm$0.14	0.1$\pm$0.13
RAM (MB)	Random	10 $\pm$ 2	8237 $\pm$ 11	1991 $\pm$ 10
	Nettack	2496 $\pm$ 43	21102 $\pm$ 12133	2973 $\pm$ 628
	FGA	2307 $\pm$ 0	9036 $\pm$ 72	1720 $\pm$ 13
	PR-BCD	2307 $\pm$ 0	9037 $\pm$ 72	1857 $\pm$ 21
	Gottack	2430 $\pm$ 13	10121 $\pm$ 214	2069 $\pm$ 623
	BOCLOAK (ours)	1455$\pm$8	7974$\pm$25	1450$\pm$2
GPU (MB)	Random	116 $\pm$ 11	20812 $\pm$ 2	146 $\pm$ 6
	Nettack	288 $\pm$ 58	5982 $\pm$ 4	156 $\pm$ 40
	FGA	812 $\pm$ 26	9222 $\pm$ 28	286 $\pm$ 2
	PR-BCD	812 $\pm$ 26	6178 $\pm$ 4	286 $\pm$ 1
	Gottack	324 $\pm$ 2	5982 $\pm$ 15	240 $\pm$ 2
	BOCLOAK (ours)	9$\pm$1	13$\pm$2	11$\pm$1

Table 23. **System Overhead Study:** System resource usage by BOCLOAK and SOTA adversarial attacks against vanilla BotRGCN with budget Δ of 1. The best performance is shown in bold, and the second best is underlined.

U. System Overhead

We summarize the system-level cost of running BOCLOAK and competing SOTA attacks in Table 23. The results show that BOCLOAK consistently operates with near-negligible GPU usage (9–13MB; **99.8%** lower) and achieves up to 20 $\times$ lower runtime across datasets, primarily because it does not invoke GPU-intensive optimization. Moreover, BOCLOAK remains RAM-efficient by restricting computation to loading the graph and querying the precomputed OT transport plan, whereas several baselines allocate tens of GBs to perform iterative perturbations over graphs and features. This yields predictable, near-constant overhead for BOCLOAK, unlike gradient/optimization-based methods that scale poorly as graphs grow.

Further, we observe a consistent pattern: optimization-driven attacks (e.g., Nettack/PR-BCD/Gottack) incur multi-second runtimes and elevated RAM usage even at a fixed budget, indicating that perturbation search (not mere inference) dominates the end-to-end overhead. This trend becomes more pronounced as dataset scale increases: TwiBot-22 triggers the highest GPU reservations (on the order of multiple GBs) and the largest runtime variability, highlighting strong sensitivity to graph size and iterative optimization dynamics. We noticed that even attacks that appear fast in wall-clock time (e.g., FGA) can still reserve substantial GPU memory, underscoring that runtime alone can understate operational cost; in contrast, BOCLOAK

Outgoing follows (Target Bot →)	Incoming follows (→ Target Bot)
Target Bot → Human	Human → Target Bot
Target Bot → Human	Bots → Target Bot
Target Bot → Human	Both → Target Bot
Target Bot → Human	Nobody → Target Bot
Target Bot → Bots	Human → Target Bot
Target Bot → Bots	Bots → Target Bot
Target Bot → Bots	Both → Target Bot
Target Bot → Bots	Nobody → Target Bot
Target Bot → Both	Human → Target Bot
Target Bot → Both	Bots → Target Bot
Target Bot → Both	Both → Target Bot
Target Bot → Both	Nobody → Target Bot
Target Bot → Nobody	Human → Target Bot
Target Bot → Nobody	Bots → Target Bot
Target Bot → Nobody	Both → Target Bot
Target Bot → Nobody	Nobody → Target Bot

Table 24. **Structural Categories.** Sixteen structural categories used to characterize target bot’s neighborhood using 1-hop ego-graph structure around the target bot. Rows enumerate the *outgoing* follow type (target Bot account following Humans/Bots/Both/Nobody) and the *incoming* follow type (followers of the target Bot are Humans/Bots/Both/Nobody).

reduces attack generation to lightweight OT-based lookups with predictable, bounded memory.



Figure 7. **BOCLOAK induced edits in practice (Twibot-22).** Bars compare (i) the distribution of candidate cloaks across structural categories and (ii) the normalized sampling mass assigned to those categories (*Sampling weight*). (Left) Outgoing follow categories (target bot → others). (Right) Incoming follower categories (others → target bot).

V. BOCLOAK in Practice

Characterizing the edits made by BOCLOAK (Twibot-22). To validate plausibility beyond the constraint-function definition, we summarize what graph edits BOCLOAK makes in a Twibot-22 against BotRGCN for node editing attack using a budget $\Delta=5$ over 50 trials. The attack flips 48/50 targets (96% misclassifications rate) within budget.

Template selection and constraint-induced pruning. BOCLOAK first selects OT-boundary accounts for templating (OT-boundary bots with incident edges ≤ 5 are computed) and then filters misclassified-bot templates. 3,638 templates remain after filtering, while 11,709 are rejected by constraints and filters (*e.g.*, only 23.70% retained; 76.30% pruned). This demonstrates that the constraints substantially narrow the feasible set of candidate cloaks in practice.

Outgoing edges (to humans vs. bots). Using the structural categories Table 24, we characterize the different combinations (*type*) of neighborhood structure BOCLOAK can mimic. Figure 7 shows that the candidate cloaks are dominated by categories with *no outgoing follows* (“Outgoing: nobody”). Moreover, the sampler’s category weights concentrate mostly on templates with no outgoing edges, implying that BOCLOAK typically avoids creating new outgoing follow edges. When outgoing edges are present in the weighted sampler, they are overwhelmingly “to humans” rather than “to bots”.

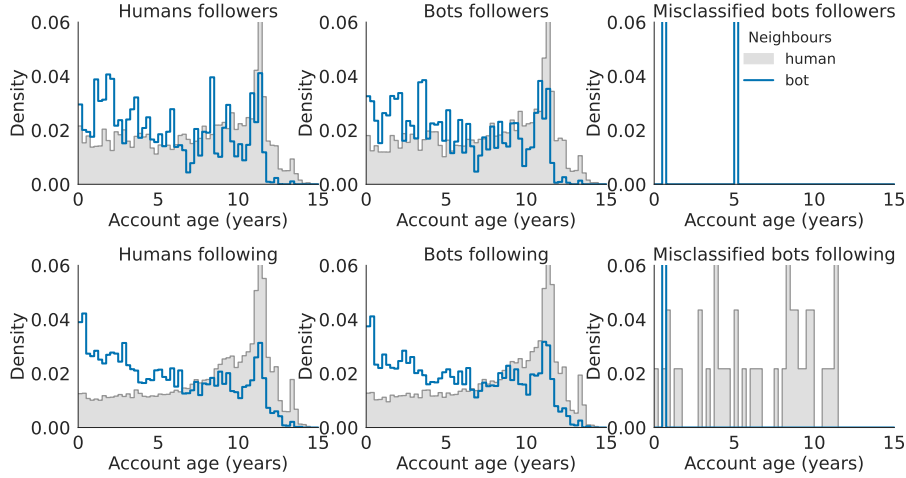


Figure 8. Temporal distribution of neighbor accounts for TwiBot-22. Top row shows followers; bottom row shows following. Shaded histograms represent human neighbors and outlined curves represent bot neighbors.

Incoming edges (from helper bots). The same category decomposition also describes who follows the edited account (incoming neighborhood). In the cloak pool, 85.50% of templates fall in categories where the account is followed by bots or by a mix of bots and humans (“Incoming: from bots” or “Incoming: both”). After applying the sampler’s category weights, 95.80% of the sampling mass lies in these bot-involving incoming categories, indicating that BOCLOAK’s most common successful mechanism is to realize (or match) *bot-driven incoming followers*; assuming helper-bot follows under the constraint that human→bot follow creation is disallowed.

How constraints shape which accounts are chosen. Figure 7 indicates the feasible cloak accounts concentrate in *low-degree* structural regimes. On the outgoing side (left), the vast majority of selected cloaks fall into the *no-outgoing-follows* category (to nobody), with the remainder largely in to both and to humans, and essentially none in the to bots category. This indicates that, under our constraints, BOCLOAK prefers (and is often forced) to avoid creating new outgoing follows and instead rely on edits that minimally perturb the target’s outgoing neighborhood.

On the incoming side (right), the sampling distribution places most of its mass on categories where the target is followed by bots (from bots), yet many candidate cloaks are dominated by from both. This mismatch is consistent with constraint-induced filtering: because we disallow creating human→target bot follows, achieving a from both structure typically requires selecting cloaks that already have some human followers, while the remaining required signal is supplied via helper-bot incoming follows. In other words, the constraints steer BOCLOAK toward accounts whose local structure can be matched with a small number of bot-driven incoming edges without introducing implausible outgoing rewiring.

BOCLOAK temporal alignment for cloak selection. Figure 8 compares neighbor account-age distributions (followers and following) for (i) true humans, (ii) true bots, and (iii) misclassified bots after BOCLOAK attack. The temporal alignment constraint does not only shrink the candidate pool; it also biases *which ages* are preferred when constructing human-looking neighborhoods. In particular, for misclassified bots we observe a consistent directional shift: selected *human* neighbors skew toward *older* accounts, whereas selected *bot* neighbors skew toward *newer* accounts. This effect is most visible in the following distributions, where human-neighbor mass for misclassified bots concentrates at higher ages, while bot neighbors appear closer to the low-age end.

This behavior reflects the interaction between temporal plausibility and the attack objective. Because BOCLOAK can only add edges that satisfy temporal alignment, it leverages temporally consistent accounts to move the target’s neighborhood statistics toward those of humans. Older human accounts contribute stable, human-characteristic temporal signatures that help pull the target toward the human region of the decision boundary, while newer bot accounts are comparatively easier to match under temporal constraints and can satisfy connectivity requirements without introducing implausible temporal patterns.

BOCLOAK degree alignment for cloak selection. Figure 9 compares neighbor degree distributions (followers and following) for (i) true humans, (ii) true bots, and (iii) misclassified bots after the BOCLOAK attack. A consistent separation

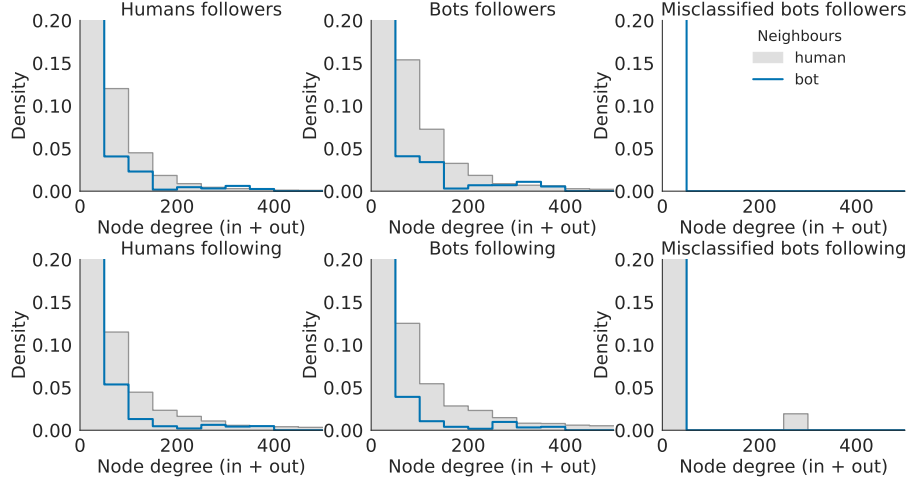


Figure 9. Degree distribution of neighbor accounts for TwiBot-22. Top row shows followers; bottom row shows following. Shaded histograms represent human neighbors and outlined curves represent bot neighbors.

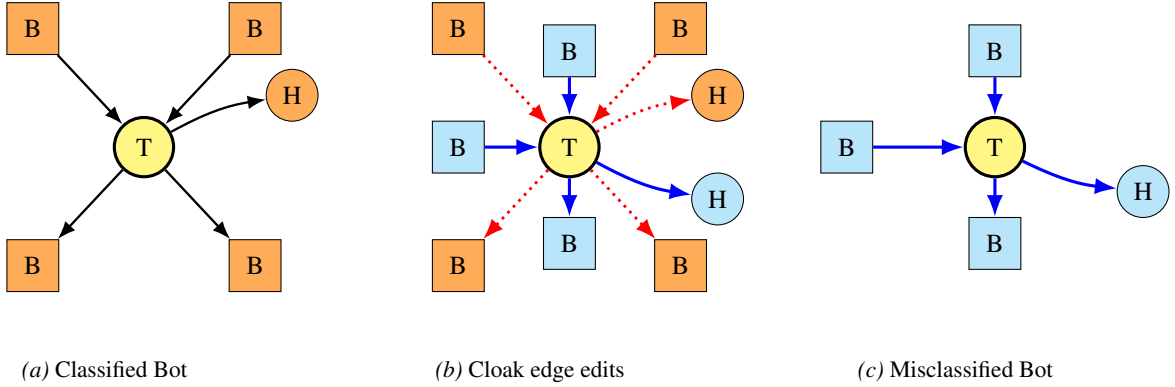


Figure 10. Cloaking target bot (node id=81779) with misclassified boundary bot cloak (node id=41575) in TwiBot-22 dataset. Target bot (T) is yellow; pre-existing neighbors are orange; new neighbors introduced by BOCLOAK are light blue. Red dotted arrows denote deleted incident edges; blue arrows denote newly added edges.

is that true bots are associated with heavier-tailed neighbor degrees than true humans, suggesting that bots are embedded in denser local structures (more high-degree neighbors). For misclassified bots, the distributions shift toward *lower-degree* neighbors, especially in the followers neighbors, indicating that successful cloaking tends to attach the target to accounts with low connectivity that better match the human neighborhood in terms of degree.

We also suggests an asymmetry between followers and following in Figure 9. The follower-side distributions show a clearer move away from the high-degree tail, whereas the following-side distributions are comparatively less shifted, implying that follower cloaks are a more effective (or less constrained) lever for altering degree-based signals at the target. Additionally, the misclassified-bot appear sparser than the true-class panels, consistent with the fact that only a small number of edges are added per target by BOCLOAK; with limited budget, BOCLOAK concentrates its additions on the most degree-informative connections, rather than attempting to reproduce the full human degree distribution. Overall, BOCLOAK succeeds by selectively using neighborhood degree composition, primarily by favoring low-degree follower connections to reduce bot-like structural signatures while maintaining plausible connectivity.

W. BOCLOAK Case Study

Target bot 81779 and (cloak 41575). Case study in Figure 10 illustrated that target bot 81779 is a low-degree node: it has

total degree 5 (in-degree 2, out-degree 3), and its in-neighbors are labeled as bot and two out-neighbors are bots and one is human. Temporally, the target account is relatively old (about 5.59 years), and its average 1-hop neighbors are also mature (about 4.96 years). In contrast, the chosen cloak 41575 1-hop neighbor is sparse (degree 4, dominated by outgoing edges) and younger (about 2.64 years).

Attack outcome and cloak signature. The detector initially predicts the target as bot, but flips to human after the cloak is applied. The cloaking constraints signature shows four edges are added, with the label breakdown indicating two newly added incoming edges from bots and two outgoing edge to a bot and a human. The target’s incident edges are removed upon cloaking, these four edited edges effectively dominate the target’s observable neighborhood, a regime where small absolute edge edits become large relative perturbations resulting in misclassification.

Neighborhood interpretation. Despite the target’s sparsity, it is embedded in a large 2-hop neighborhood: target 81779 hop-2 nodes, which are predominantly human (about 942 humans, 78 bots; *i.e.*, $\approx 92\%$ human and $\approx 7.6\%$ bot). The 2-hop neighbors are also temporally mature (median account age ≈ 7.86 years), suggesting the attack is not exploiting a locally bot-dense region. Instead, this case highlights a brittle operating point for low-degree nodes: when the pre-attack ego signal is effectively a single edge, a small number of constraint-matched additions (combined with an edge wipe) can overwhelm the evidence the classifier uses and induce a label flip.

X. Defense against BOCLOAK

BOCLOAK-inspired style evasion is effective because it reshapes a node’s *neighborhood distribution* (rather than only a few individual edges) to look human-like while keeping edits sparse and socially plausible. A practical defense is therefore to (i) add an explicit *cloak detector* that compares each account’s k -hop ego-neighborhood distribution to human/bot reference distributions, (ii) adversarially harden the detector using *feasible* perturbations that match the attacker constraints (incident-edge edits, budgeted, direction/temporal plausibility), and (iii) for detection complement graph structure with temporally constant behavioral/content signals that are harder to keep consistent under long-lived evasion.